

Big Data

- Big data is a term that refers to the large and complex datasets that are generated by various sources, such as social media, sensors, e-commerce, etc.
- Big data poses challenges for traditional data processing and analysis methods, such as storage, scalability, performance, quality, security, privacy, etc.
- Big data also offers opportunities for discovering new insights, patterns, trends, and value from the data, such as customer behavior, market trends, fraud detection, etc.
- Big data requires new technologies and techniques for data collection, storage, processing, analysis, and visualization, such as distributed systems, cloud computing, parallel computing, machine learning, data mining, etc.
- Big data is characterized by the 5 Vs: volume, velocity, variety, veracity, and value.
 - Volume: the amount of data generated and stored, which can range from terabytes to petabytes and beyond.
 - Velocity: the speed at which data is generated and processed, which can be real-time, near-real-time, or batch.
 - Variety: the diversity of data types and formats, which can be structured, semi-structured, or unstructured, such as text, images, audio, video, etc.
 - Veracity: the quality and reliability of data, which can be affected by noise, inconsistency, incompleteness, ambiguity, etc.
 - Value: the potential usefulness and benefit of data for decision making, problem solving, innovation, etc.

Unit 1 - Introduction to Big Data

- Big data is a term that refers to the large, complex, and diverse sets of data that are generated at high speed from various sources, such as social media, sensors, web logs, etc.
- Big data has four main characteristics: volume, variety, velocity, and veracity.
 - Volume: The amount of data that is generated and stored. Big data can range from terabytes to petabytes or even exabytes of data.
 - Variety: The types and formats of data that are collected and analyzed. Big data can include structured, semi-structured, or unstructured data, such as text, images, audio, video, etc.
 - Velocity: The speed at which data is generated and processed. Big data can be produced and consumed in real time or near real time, requiring fast and efficient processing techniques.
 - Veracity: The quality and reliability of data that are collected and analyzed. Big data can be noisy, incomplete, inconsistent, or inaccurate, requiring methods to ensure data quality and integrity.

- Big data poses many challenges and opportunities for businesses, organizations, and society, such as:
 - How to store, manage, and access big data efficiently and securely?
 - How to analyze, visualize, and extract insights from big data effectively and accurately?
 - How to leverage big data to improve decision making, innovation, and performance?
 - How to balance the benefits and risks of big data, such as privacy, ethics, and social impact?
- Big data requires new technologies, tools, and methods to handle its complexity and scale, such as:
 - Distributed and parallel computing systems, such as Hadoop, Spark, etc., that can process large amounts of data across multiple nodes and clusters.
 - NoSQL and NewSQL databases, such as MongoDB, Cassandra, etc., that can store and query various types of data with high scalability and performance.
 - Machine learning and data mining algorithms, such as classification, clustering, regression, etc., that can discover patterns, trends, and associations from big data.
 - Data visualization and exploration techniques, such as charts, graphs, dashboards, etc., that can present and communicate big data in an understandable and interactive way.

Types of digital data in big data

Big data refers to the large and complex sets of data that are generated from various sources and require advanced techniques and tools to process, analyze and extract insights. Big data can be classified into different types based on the format, structure and source of the data. The following are some of the common types of digital data in big data:

- **Structured data:** This is the data that can be easily organized, stored and processed in a fixed format, such as tables, spreadsheets or databases. Structured data is usually generated from transactional systems, sensors, machines or applications. Examples of structured data are customer records, sales data, product information, sensor readings, etc. Structured data can be easily queried and analyzed using SQL or other programming languages.
- **Unstructured data:** This is the data that has no predefined format or structure and is often in the form of text, images, audio, video or log files. Unstructured data is usually generated from human activities, such as social media posts, emails, blogs, reviews, videos, etc. Examples of unstructured data are tweets, photos, videos, emails, etc. Unstructured data can be difficult to process and analyze using traditional methods and requires specialized tools and techniques, such as natural language

processing, computer vision, sentiment analysis, etc.

- **Semi-structured data:** This is the data that has some elements of structure, such as tags, labels or metadata, but does not conform to a fixed schema or format. Semi-structured data is often in the form of XML, JSON, HTML or CSV files. Semi-structured data is usually generated from web applications, APIs, IoT devices or online platforms. Examples of semi-structured data are web pages, JSON files, XML documents, etc. Semi-structured data can be processed and analyzed using a combination of structured and unstructured methods, such as parsing, querying, indexing, etc.
- **Open data:** This is the data that is publicly available and accessible for anyone to use, reuse and share. Open data is often published by governments, organizations or individuals for the purpose of transparency, accountability, innovation or collaboration. Examples of open data are census data, weather data, traffic data, etc. Open data can be accessed and analyzed using various tools and platforms, such as open data portals, APIs, web services, etc.
- **Dark/lost data:** This is the data that is hidden, inaccessible or unused due to various reasons, such as siloed systems, security issues, privacy concerns or lack of awareness. Dark/lost data is often generated from legacy systems, CCTV cameras, offline devices or confidential sources. Examples of dark/lost data are surveillance footage, medical records, offline transactions, etc. Dark/lost data can be valuable and insightful if it can be retrieved, integrated and analyzed using appropriate methods and technologies, such as data mining, data integration, data visualization, etc.
- **Linked data:** This is the data that is connected and interlinked with other data sources using common identifiers, standards and formats. Linked data is often generated from web data, such as web pages, social networks, online databases or semantic web. Examples of linked data are Wikipedia, DBpedia, Linked Open Data, etc. Linked data can be accessed and analyzed using various methods and tools, such as SPARQL, RDF, OWL, etc.

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write the content in markdown format on the history of Big Data innovation. Here is what I have written:

History of Big Data innovation

- Big Data is the term used to describe the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- The concept of Big Data is not new, as humans have been collecting and analyzing data for centuries. However, the term Big Data was coined

in the late 1990s by John Mashey, a computer scientist and executive at Silicon Graphics, who used it to describe the challenges and opportunities of data-intensive computing.

- The emergence of Big Data innovation was driven by several factors, such as:
 - The exponential growth of data from the internet, social media, mobile devices, sensors, and other digital sources.
 - The development of new technologies and tools, such as cloud computing, distributed systems, parallel processing, machine learning, and artificial intelligence, that enabled the storage, processing, and analysis of Big Data at scale and speed.
 - The recognition of the value and potential of Big Data for various domains and applications, such as business, science, health, education, government, and society.
- Some of the milestones and achievements of Big Data innovation are:
 - In 2003, Google published a paper on MapReduce, a programming model and framework for processing large-scale data sets using a cluster of computers.
 - In 2004, Google published a paper on Bigtable, a distributed storage system for managing structured data at petabyte scale.
 - In 2006, Amazon launched its Elastic Compute Cloud (EC2) and Simple Storage Service (S3), which provided cloud computing and storage services for Big Data applications.
 - In 2008, Yahoo! released Hadoop, an open-source software platform that implemented the MapReduce and distributed file system concepts for Big Data processing.
 - In 2009, IBM launched the Watson project, a cognitive computing system that used natural language processing and machine learning to answer questions from large and diverse data sources.
 - In 2010, Facebook launched the Open Compute Project, an initiative to share the designs and best practices of its data center infrastructure for Big Data processing.
 - In 2012, the Obama administration announced the Big Data Research and Development Initiative, a multi-agency effort to invest in Big Data research and innovation for national priorities.
 - In 2014, Alibaba processed 278 million orders and \$9.3 billion in sales on its Singles' Day online shopping event, using its Big Data platform and analytics.
 - In 2016, AlphaGo, a computer program developed by Google's DeepMind, defeated Lee Sedol, a world champion of Go, a complex board game, using deep neural networks and reinforcement learning trained on Big Data.
 - In 2019, the Event Horizon Telescope project captured the first image of a black hole, using a global network of telescopes and Big Data processing techniques.

Introduction to Big Data Platform

- A big data platform is a system that enables the collection, storage, processing, analysis, and visualization of large and complex data sets.
- A big data platform typically consists of several components, such as:
 - Data sources: These are the origin of the data, such as sensors, web logs, social media, databases, etc.
 - Data ingestion: This is the process of acquiring, transforming, and loading the data from the sources to the platform.
 - Data storage: This is the component that stores the data in a scalable and reliable way, such as distributed file systems, cloud storage, data warehouses, etc.
 - Data processing: This is the component that performs various operations on the data, such as cleaning, filtering, aggregating, joining, etc.
 - Data analysis: This is the component that applies various techniques to extract insights from the data, such as statistics, machine learning, data mining, etc.
 - Data visualization: This is the component that presents the data and the analysis results in a graphical and interactive way, such as charts, dashboards, maps, etc.
- A big data platform can be deployed on-premise, in the cloud, or in a hybrid mode, depending on the requirements and the resources of the organization.
- A big data platform can provide various benefits, such as:
 - Improving decision making by providing data-driven insights and predictions.
 - Enhancing customer experience by personalizing products and services.
 - Increasing operational efficiency by optimizing processes and resources.
 - Reducing costs by leveraging economies of scale and automation.
 - Innovating new products and services by discovering new patterns and opportunities.

Drivers for Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- Some of the drivers for big data are:
 - **The growth of data sources and types:** Data is being generated from various sources such as social media, sensors, mobile devices, web logs, etc. and in various formats such as text, audio, video, images, etc. This creates a diversity and volume of data that traditional

systems cannot handle efficiently.

- **The need for faster and better decision making:** Data can provide valuable insights and opportunities for businesses, governments, and individuals if it is analyzed and utilized effectively. Big data analytics can help in discovering patterns, trends, correlations, and anomalies that can improve decision making and performance in various domains such as marketing, healthcare, education, security, etc.
- **The emergence of new technologies and paradigms:** Technologies such as cloud computing, distributed computing, machine learning, artificial intelligence, etc. have enabled the development of new paradigms and platforms for big data processing and analysis. These technologies can offer scalability, flexibility, reliability, and cost-effectiveness for big data applications.
- **The demand for innovation and competitiveness:** Big data can provide a competitive edge and a source of innovation for organizations and individuals who can leverage it to create new products, services, processes, and business models. Big data can also enable new forms of collaboration, communication, and social interaction among various stakeholders.

Big Data Architecture

- Big data architecture is the logical and/or physical layout/structure of how big data is stored, accessed and managed within a big data or IT environment.
- Big data architecture primarily serves as the key design reference for big data infrastructures and solutions. It is created by big data designers/architects before physically implementing a big data solution.
- A big data architecture is designed to handle the ingestion, processing, and analysis of data that is too large or complex for traditional database systems. The threshold at which organizations enter into the big data realm differs, depending on the capabilities of the users and their tools.
- A data architecture describes how data is managed—from collection through to transformation, distribution, and consumption. It sets the blueprint for data and the way it flows through data storage systems. It is foundational to data processing operations and artificial intelligence (AI) applications.
- A typical big data architecture consists of the following components:
 - Data sources: These are the various sources of data, such as sensors, web logs, social media, etc. that generate or collect data in different formats and volumes.
 - Data ingestion: This is the process of acquiring, importing, and validating data from data sources into a data storage system. Data ingestion can be batch, streaming, or hybrid, depending on the frequency and latency of data delivery.

- Data storage: This is the system that stores and organizes data for further processing and analysis. Data storage can be relational, non-relational, or hybrid, depending on the structure and scalability of data. Data storage can also be on-premises, cloud-based, or hybrid, depending on the cost and availability of resources.
- Data processing: This is the process of transforming, enriching, and aggregating data to make it suitable for analysis. Data processing can be batch, streaming, or hybrid, depending on the speed and complexity of data operations. Data processing can also be done using various frameworks and tools, such as MapReduce, Spark, Storm, etc.
- Data analysis: This is the process of applying analytical techniques, such as machine learning, statistics, visualization, etc. to extract insights and value from data. Data analysis can be descriptive, predictive, or prescriptive, depending on the purpose and scope of data exploration. Data analysis can also be done using various tools and platforms, such as Hadoop, Hive, R, Python, etc.
- Data consumption: This is the process of delivering and presenting data insights and value to the end-users, such as business analysts, decision makers, customers, etc. Data consumption can be done using various methods and formats, such as dashboards, reports, alerts, APIs, etc.

Big data characteristics

Big data is a term that refers to large and complex datasets that are beyond the capabilities of traditional data processing and analysis methods. Big data has the following characteristics:

- Volume: The amount of data generated and stored. Big data can range from terabytes to petabytes or even exabytes of data.
- Variety: The types and sources of data. Big data can include structured, semi-structured, or unstructured data from various domains such as text, images, audio, video, social media, sensor data, etc.
- Velocity: The speed and frequency of data generation and processing. Big data can be produced and consumed in real-time or near-real-time, requiring fast and efficient data processing and analysis methods.
- Veracity: The quality and reliability of data. Big data can be noisy, incomplete, inconsistent, or inaccurate, posing challenges for data cleaning and integration.
- Value: The potential and usefulness of data. Big data can provide valuable insights and opportunities for decision making, innovation, and competitive advantage, if analyzed and utilized properly.

5 Vs of Big Data

Big data is a term that refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze. Big data can be characterized by five Vs: volume, velocity, variety, veracity, and value. These are explained below:

- **Volume:** This refers to the amount or size of data that is generated and collected. Big data can range from terabytes to petabytes or even exabytes of data. The volume of data poses challenges for storage, processing, and analysis, as traditional methods and tools may not be able to handle such large datasets efficiently and effectively.
- **Velocity:** This refers to the speed or rate at which data is generated, collected, and processed. Big data can be generated at a very high velocity, such as real-time data from sensors, social media, streaming services, or online transactions. The velocity of data poses challenges for capturing, processing, and analyzing data in a timely manner, as delays or lags may result in missed opportunities or outdated insights.
- **Variety:** This refers to the diversity or heterogeneity of data types and formats that are generated and collected. Big data can come from various sources and have various structures, such as structured, semi-structured, or unstructured data. The variety of data poses challenges for integration, transformation, and analysis, as different data types and formats may require different methods and tools to handle and process.
- **Veracity:** This refers to the quality or reliability of data that is generated and collected. Big data can have various levels of veracity, such as accurate, incomplete, inconsistent, or noisy data. The veracity of data poses challenges for validation, cleaning, and analysis, as low-quality data may lead to inaccurate or misleading insights or decisions.
- **Value:** This refers to the potential or actual benefits or outcomes that can be derived from data that is generated and collected. Big data can have various values, such as strategic, operational, or financial value. The value of data poses challenges for extraction, interpretation, and visualization, as the data needs to be analyzed and presented in a way that is meaningful and actionable for the users or stakeholders.

Big Data technology components

Big data technology is a collection of tools, frameworks, and techniques that enable the processing, storage, analysis, and visualization of large and complex data sets. Some of the main components of big data technology are:

- **Data sources:** These are the origins of the data, such as sensors, web logs, social media, transactions, etc. Data sources can be structured, semi-structured, or unstructured, and can vary in volume, velocity, variety, and

veracity.

- **Data ingestion:** This is the process of acquiring, transferring, and loading the data from the sources to the destination systems, such as databases, data warehouses, data lakes, etc. Data ingestion can be batch, streaming, or hybrid, depending on the frequency and latency of the data flow.
- **Data storage:** This is the component that provides the physical or logical space for storing the data in a persistent or transient manner. Data storage can be relational, non-relational, or hybrid, depending on the schema, query, and scalability requirements of the data.
- **Data processing:** This is the component that performs the transformation, manipulation, aggregation, and computation of the data using various programming models, such as MapReduce, Spark, Flink, etc. Data processing can be batch, streaming, or hybrid, depending on the timeliness and complexity of the data analysis.
- **Data analysis:** This is the component that applies various analytical techniques, such as statistics, machine learning, data mining, natural language processing, etc., to extract insights, patterns, and knowledge from the data. Data analysis can be descriptive, predictive, or prescriptive, depending on the purpose and scope of the data exploration.
- **Data visualization:** This is the component that presents the data and the analysis results in a graphical or interactive form, such as charts, dashboards, maps, etc., to facilitate the understanding, communication, and decision making of the data. Data visualization can be static, dynamic, or interactive, depending on the level of interactivity and customization of the data display.

Big Data importance

- Big data refers to the large and complex sets of data that are generated from various sources and require advanced techniques and tools to process, analyze and extract insights.
- Big data is important because it can help organizations and individuals to make better decisions, improve performance, enhance customer experience, create new products and services, and solve complex problems.
- Some of the benefits of big data are:
 - It can reveal hidden patterns, trends and correlations that are not visible in traditional data sources.
 - It can enable real-time analysis and feedback, allowing for faster and more accurate responses to changing situations and needs.
 - It can reduce costs and increase efficiency by optimizing processes, resources and operations.
 - It can foster innovation and creativity by enabling new ways of thinking and experimenting with data.
 - It can enhance security and privacy by detecting and preventing

fraud, cyberattacks and other threats.

- Some of the challenges of big data are:
 - It can be difficult to collect, store, manage and process due to its volume, variety, velocity and veracity.
 - It can require specialized skills, technologies and infrastructure that are not widely available or affordable.
 - It can raise ethical, legal and social issues such as data ownership, quality, governance, privacy and security.

Big Data applications

Big data applications are software systems that use big data technologies to process, analyze and derive insights from large and complex data sets. Big data applications can help companies to make better business decisions, improve customer experience, optimize operations, reduce costs and risks, and create new products and services. Some of the domains where big data applications are widely used are:

- **Business and e-commerce:** Big data applications can help businesses to understand customer behavior, preferences, and needs, as well as market trends and opportunities. For example, Amazon uses big data to provide personalized recommendations, dynamic pricing, and targeted advertising to its customers.
- **Healthcare and life sciences:** Big data applications can help healthcare providers and researchers to improve diagnosis, treatment, and prevention of diseases, as well as enhance patient care and safety. For example, IBM Watson uses big data to provide cognitive computing solutions for healthcare, such as analyzing medical records, images, and literature to support clinical decision making.
- **Government and public sector:** Big data applications can help government agencies and organizations to improve public services, security, and transparency, as well as detect and prevent fraud, waste, and abuse. For example, the IRS uses big data to analyze tax data from various sources to identify suspicious behavior and multiple identities.
- **Education and learning:** Big data applications can help educators and learners to enhance teaching and learning outcomes, as well as personalize and optimize learning experiences. For example, Coursera uses big data to provide adaptive learning, feedback, and assessment to its online learners.
- **Transportation and logistics:** Big data applications can help transportation and logistics companies to optimize routes, schedules, and capacities, as well as improve safety, efficiency, and customer satisfaction. For example, UPS uses big data to monitor and analyze its fleet of vehicles, drivers, and packages to reduce fuel consumption, emissions, and delivery time.
- **Media and entertainment:** Big data applications can help media and

entertainment companies to create and distribute engaging and relevant content, as well as understand and target audiences. For example, Netflix uses big data to provide personalized recommendations, streaming quality, and original content to its subscribers.

Big Data features – security, compliance, auditing and protection

- Big data security refers to the measures and tools used to protect big data from unauthorized access, modification, or deletion. Big data security is essential to ensure the confidentiality, integrity, and availability of big data.
- Big data compliance refers to the adherence of big data to the laws, regulations, standards, and policies that govern its collection, processing, storage, and use. Big data compliance is important to ensure the legality, ethics, and accountability of big data.
- Big data auditing refers to the process of examining and verifying big data to ensure its quality, accuracy, completeness, and reliability. Big data auditing is useful to detect and correct errors, anomalies, or fraud in big data.
- Big data protection refers to the preservation and recovery of big data from accidental or intentional loss, damage, or destruction. Big data protection is necessary to ensure the durability, availability, and resilience of big data.

Some of the challenges and solutions for big data features are:

- Big data security challenges include:
 - The large volume, variety, and velocity of big data make it difficult to apply traditional security methods and tools, such as encryption, access control, or firewalls.
 - The distributed and heterogeneous nature of big data sources, platforms, and applications increase the complexity and risk of security breaches and attacks.
 - The lack of standardization and interoperability among big data technologies and vendors hamper the implementation and enforcement of security policies and best practices.
- Big data security solutions include:
 - Developing and adopting new security methods and tools that can scale and adapt to the dynamic and diverse characteristics of big data, such as homomorphic encryption, attribute-based encryption, or blockchain.
 - Implementing and integrating security mechanisms and controls at different layers and stages of the big data lifecycle, such as data collection, processing, storage, and analysis.
 - Establishing and following security standards and frameworks that can provide guidance and recommendations for big data security, such as the NIST Big Data Security and Privacy Guidelines or the CSA Big Data Security and Privacy Handbook.

- Big data compliance challenges include:
 - The global and cross-border nature of big data raises legal and ethical issues regarding the jurisdiction, sovereignty, and ownership of big data, as well as the protection of personal data and privacy rights of data subjects.
 - The dynamic and evolving nature of big data regulations and requirements make it difficult to keep track and comply with the latest and relevant laws, rules, and norms, such as the GDPR, the CCPA, or the HIPAA.
 - The lack of transparency and accountability of big data processes and outcomes make it hard to demonstrate and verify the compliance of big data, as well as to identify and address any non-compliance issues or violations.
- Big data compliance solutions include:
 - Developing and adopting new compliance methods and tools that can enable and automate the compliance of big data, such as data anonymization, data lineage, or data governance.
 - Implementing and integrating compliance mechanisms and controls at different layers and stages of the big data lifecycle, such as data collection, processing, storage, and analysis.
 - Establishing and following compliance standards and frameworks that can provide guidance and recommendations for big data compliance, such as the ISO/IEC 27000 series or the COBIT 5 framework.
- Big data auditing challenges include:
 - The high volume, variety, and velocity of big data make it difficult to audit and validate big data, as well as to ensure its consistency and correctness across different sources, platforms, and applications.
 - The complex and opaque nature of big data algorithms and models make it difficult to audit and explain big data, as well as to ensure its fairness and trustworthiness.
 - The lack of auditing skills and expertise among big data practitioners and stakeholders make it difficult to audit and improve big data, as well as to communicate and collaborate on big data auditing issues and solutions.
- Big data auditing solutions include:
 - Developing and adopting new auditing methods and tools that can enable and automate the auditing of big data, such as data quality assessment, data provenance, or data visualization.
 - Implementing and integrating auditing mechanisms and controls at different layers and stages of the big data lifecycle, such as data collection, processing, storage, and analysis.
 - Establishing and following auditing standards and frameworks that can provide guidance and recommendations for big data auditing, such as the ISACA Big Data Audit Program or the AICPA Big Data Audit Guide.

- Big data protection challenges include:
 - The large volume, variety, and velocity of big data make it difficult to backup and restore big data, as well as to ensure its availability and accessibility in case of failures or disasters.
 - The distributed and heterogeneous nature of big data sources, platforms, and applications increase

Security of Big Data

- Big data refers to the large and complex datasets that are generated from various sources, such as social media, sensors, web logs, etc.
- Big data security is the process of protecting the confidentiality, integrity, and availability of big data from unauthorized access, modification, or disclosure.
- Big data security challenges include:
 - Data volume: The sheer size of big data makes it difficult to store, process, and analyze using traditional security tools and techniques.
 - Data variety: The diverse and heterogeneous nature of big data sources and formats poses challenges for data integration, validation, and quality assurance.
 - Data velocity: The high speed and frequency of big data generation and transmission requires real-time or near-real-time security monitoring and response.
 - Data veracity: The uncertainty and inconsistency of big data quality and reliability raises questions about the trustworthiness and accuracy of the data and the derived insights.
 - Data value: The high value of big data for business, research, and social purposes attracts malicious actors who may attempt to steal, tamper, or destroy the data or the systems that handle it.
- Big data security solutions include:
 - Data encryption: The process of transforming the data into an unreadable form using a secret key, so that only authorized parties can decrypt and access the data.
 - Data masking: The process of hiding or replacing sensitive data elements with fictitious or anonymized values, so that the data can be used for analysis or testing without revealing the original data.
 - Data anonymization: The process of removing or modifying personally identifiable information (PII) or other sensitive data attributes from the data, so that the data cannot be linked to a specific individual or entity.
 - Data access control: The process of defining and enforcing rules and policies that specify who can access, modify, or delete the data, and under what conditions.
 - Data auditing: The process of recording and reviewing the activities and events related to the data, such as data creation, modification, deletion, access, or transfer, to ensure compliance and accountability.

- Data backup and recovery: The process of creating and maintaining copies of the data in a separate location, and restoring the data in case of data loss or corruption due to hardware failure, human error, or cyberattack.

Compliance of Big Data

- Compliance of big data refers to the process of ensuring that the data collected, processed, and stored by an organization meets the standards and regulations of the relevant authorities and stakeholders.
- Compliance of big data is important for several reasons, such as:
 - Protecting the privacy and security of the data subjects and the data owners.
 - Preventing the misuse or abuse of the data for illegal or unethical purposes, such as fraud, money laundering, or discrimination.
 - Enhancing the trust and reputation of the organization among its customers, partners, and regulators.
 - Avoiding the legal and financial risks of non-compliance, such as fines, lawsuits, or sanctions.
- Compliance of big data involves several challenges, such as:
 - Managing the volume, variety, and velocity of the data, which may exceed the capacity and capability of the existing systems and processes.
 - Balancing the trade-off between the value and the risk of the data, which may require different levels of access, protection, and retention.
 - Adapting to the dynamic and evolving nature of the data, which may change in quality, relevance, and context over time.
 - Aligning with the diverse and complex requirements of the data, which may vary across different jurisdictions, domains, and stakeholders.
- Compliance of big data can be achieved by adopting various strategies, such as:
 - Implementing a robust data governance framework, which defines the roles, responsibilities, policies, and procedures for the data lifecycle.
 - Applying a risk-based approach, which identifies, assesses, and mitigates the potential threats and vulnerabilities of the data.
 - Leveraging the power of big data analytics, which enables the detection, prevention, and resolution of the compliance issues and incidents.
 - Engaging with the relevant stakeholders, which ensures the transparency, accountability, and collaboration of the data activities.

Auditing of Big Data

- Big data is the term used to describe large and complex data sets that are generated from various sources and have different characteristics, such as

volume, velocity, variety, veracity, and value.

- Auditing of big data is the process of examining and evaluating the quality, reliability, security, and integrity of big data and the systems that collect, store, process, and analyze it.
- Auditing of big data can help organizations to ensure compliance with regulations, standards, and policies, to identify and mitigate risks, to improve performance and efficiency, and to gain insights and value from big data.
- Auditing of big data can be challenging due to the following factors:
 - The volume and velocity of big data can make it difficult to capture, store, and process all the relevant data in a timely manner.
 - The variety and veracity of big data can make it difficult to ensure the accuracy, completeness, and consistency of the data and to deal with data quality issues, such as missing, incomplete, or inaccurate data.
 - The value of big data can depend on the context, purpose, and methods of analysis, which can vary across different users and stakeholders.
 - The security and privacy of big data can be compromised by unauthorized access, use, disclosure, modification, or deletion of the data or by cyberattacks, data breaches, or data loss.
- Auditing of big data can be performed using various methods and techniques, such as:
 - Data sampling and testing, which involves selecting and examining a representative subset of the data to verify its quality, reliability, and validity.
 - Data profiling and analytics, which involves analyzing the data to discover its structure, content, patterns, trends, and anomalies.
 - Data governance and management, which involves establishing and enforcing policies, standards, and procedures for the collection, storage, processing, and analysis of the data.
 - Data security and privacy, which involves implementing and monitoring controls and measures to protect the data from unauthorized or inappropriate access, use, disclosure, modification, or deletion.
 - Data audit trail and logging, which involves recording and tracking the activities and events related to the data and the systems that handle it.

Protection of Big Data

Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to store, process, analyze, and share. Big data has many potential benefits for businesses, governments, and society, such as improving decision making, enhancing customer experience, increasing efficiency, and creating new value. However, big data also poses significant challenges and risks for data privacy and security, as it

may contain sensitive and personal information that can be misused, breached, or exploited by unauthorized parties.

To protect the privacy and security of big data, it is important to adopt a comprehensive and proactive approach that covers the following aspects:

- **Data collection:** Data collection is the process of acquiring data from various sources, such as sensors, devices, applications, websites, social media, etc. Data collection should be done in a transparent, lawful, and ethical manner, respecting the rights and preferences of the data subjects. Data collection should also be limited to the minimum necessary for the intended purposes, and avoid collecting irrelevant or excessive data that may increase the privacy risks.
- **Data retention and archiving:** Data retention and archiving is the process of storing and preserving data for future use or reference. Data retention and archiving should be done in a secure and efficient manner, ensuring the integrity, availability, and accessibility of the data. Data retention and archiving should also be done in accordance with the applicable laws and regulations, as well as the data subjects' consent and expectations. Data retention and archiving should also be subject to periodic review and deletion, and avoid keeping data longer than necessary or required.
- **Data use:** Data use is the process of applying data for various purposes, such as analysis, processing, sharing, etc. Data use should be done in a responsible and ethical manner, respecting the data subjects' rights and interests, as well as the data quality and accuracy. Data use should also be done in a secure and controlled manner, ensuring the confidentiality, integrity, and availability of the data. Data use should also be done in compliance with the applicable laws and regulations, as well as the data subjects' consent and expectations. Data use should also be subject to audit and oversight, and avoid unauthorized or inappropriate use of data.
- **Data disclosure:** Data disclosure is the process of revealing or transferring data to third parties, such as partners, customers, regulators, etc. Data disclosure should be done in a transparent and accountable manner, informing the data subjects and obtaining their consent when necessary. Data disclosure should also be done in a secure and controlled manner, ensuring the confidentiality, integrity, and availability of the data. Data disclosure should also be done in compliance with the applicable laws and regulations, as well as the data subjects' consent and expectations. Data disclosure should also be subject to audit and oversight, and avoid unnecessary or excessive disclosure of data.

Big Data Privacy

Big data privacy is the practice of properly managing large and complex data sets to minimize risk and protect sensitive data. Big data privacy is important for both data owners and data users, as it involves ethical, legal, and technical aspects of data collection, storage, analysis, and sharing. Some of the main

points to consider about big data privacy are:

- Big data privacy is a matter of customer trust. The more data you collect about users, the easier it gets to “connect the dots” and understand their behavior, preferences, and personal information. This can lead to privacy breaches, identity theft, discrimination, or manipulation if the data is mis-used or exposed. Therefore, data owners should respect the privacy rights and expectations of their customers, and data users should be transparent and accountable about how they use the data.
- Big data privacy is also a matter of compliance. There are many regulations and laws that govern the privacy of personal data and sensitive data, such as the General Data Protection Regulation (GDPR) in the European Union, the California Consumer Privacy Act (CCPA) in the United States, and the Personal Data Protection Act (PDPA) in Singapore. These regulations require data owners and data users to follow certain principles and obligations, such as obtaining consent, providing notice, ensuring data quality, implementing security measures, and honoring data subject rights.
- Big data privacy is a challenge for traditional data management processes. Because big data involves large volumes, high velocity, and diverse variety of data, many conventional data privacy methods cannot handle the scale and complexity required. For example, data anonymization techniques, such as masking or aggregation, may not be sufficient to prevent re-identification or inference attacks. Therefore, data owners and data users need to adopt new and innovative data privacy solutions, such as differential privacy, federated learning, or homomorphic encryption, that can preserve the utility and privacy of big data.
- Big data privacy is an ongoing responsibility. Data privacy is not a one-time event, but a continuous process that requires regular review and update. Data owners and data users should monitor the data lifecycle and the data environment, and adjust their data privacy policies and practices accordingly. For example, data owners should review their data collection, retention, and archiving activities, and delete or anonymize data that is no longer needed or relevant. Data users should review their data use, including use in testing, DevOps, and other data masking scenarios, and ensure that they comply with the data disclosure and sharing agreements.

Big Data Ethics

Big data ethics is a branch of ethics that deals with the moral and ethical issues related to the collection, analysis, and use of large and complex data sets, especially personal data. Big data ethics aims to create an ethical and moral code of conduct for data use that preserves the trust and dignity of data subjects and data users .

Some of the main topics and challenges in big data ethics are:

- **Privacy and consent:** How to protect the privacy and autonomy of data

subjects, especially when data is collected from multiple sources, shared across different platforms, and used for different purposes than originally intended or consented to ?

- **Transparency and accountability:** How to ensure that data users are transparent and accountable for their data practices, such as how they collect, store, process, and share data, and how they make decisions based on data analysis and algorithms ?
- **Fairness and justice:** How to prevent and mitigate the potential harms and biases that may arise from data analysis and algorithms, such as discrimination, exclusion, manipulation, and exploitation of data subjects or groups ?
- **Beneficence and non-maleficence:** How to balance the benefits and risks of data use, and ensure that data use does not cause harm or violate the rights and interests of data subjects or society at large ?
- **Responsibility and integrity:** How to foster a culture of ethical awareness and responsibility among data users, and ensure that data use is consistent with the values and principles of data ethics ?

Big data ethics is an emerging and interdisciplinary field that requires the collaboration and dialogue of various stakeholders, such as data scientists, data users, data subjects, regulators, policymakers, ethicists, and civil society . Big data ethics is also a dynamic and evolving field that needs to adapt to the changing technological and social contexts of data use .

Big Data Analytics

- Big data analytics is a form of advanced analytics that involves complex applications with elements such as predictive models, statistical algorithms and what-if analysis powered by analytics systems.
- Big data analytics refers to the methods, tools, and applications used to collect, process, and derive insights from varied, high-volume, high-velocity data sets.
- Big data analytics can help companies make better business decisions by discovering market trends, insights, and patterns from large and diverse data sources .
- Big data analytics can also enable faster and more efficient data processing, as well as support real-time analytics and streaming data.
- Big data analytics can be applied to various domains, such as e-commerce, healthcare, finance, education, social media, and smart cities.
- Big data analytics can face challenges such as data quality, security, privacy, scalability, and integration.

Challenges of conventional systems compared to Big Data

- Conventional systems are based on the relational data model, which requires data to be structured and normalized. Big Data can handle unstructured and semi-structured data, such as text, images, audio, video,

etc.

- Conventional systems are batch-oriented, which means they process data in fixed intervals, such as daily or weekly. Big Data can process data in real-time or near-real-time, which enables faster and more accurate insights.
- Conventional systems rely on expensive hardware, such as MPP (Massively Parallel Processing) systems, to achieve parallelism and scalability. Big Data can leverage commodity hardware, such as clusters of servers, to distribute the workload and scale horizontally.
- Conventional systems have predefined schemas and queries, which limit the flexibility and adaptability of data analysis. Big Data can use schema-on-read and dynamic queries, which allow for more exploration and discovery of data patterns and relationships.
- Conventional systems have difficulty handling the variety, velocity, and volume of Big Data, which can overwhelm the storage and processing capacity of traditional database systems. Big Data can use distributed file systems, such as Hadoop Distributed File System (HDFS), and parallel processing frameworks, such as MapReduce, to store and process large and complex data sets.

Intelligent Data Analysis in Big Data

- Intelligent Data Analysis (IDA) is one of the most important approaches in the field of data mining, which attracts great concerns from the researchers.
- IDA aims to extract useful knowledge and insights from large and complex datasets, using various techniques such as data preparation, data visualization, predictive modeling, and machine learning .
- Big Data refers to the massive volume, variety, velocity, and veracity of data that exceeds the capacity of traditional data processing systems.
- Big Data Analytics (BDA) is the application of advanced analytic techniques, including data mining, statistical analysis, and predictive modeling on big datasets as new business intelligence practice .
- BDA uses computational intelligence techniques to transform raw data into valuable information and actionable insights for decision making.
- AI and BDA work together to enhance the efficiency and effectiveness of data analysis, by automating and simplifying the complex and time-consuming tasks involved in IDA.
- AI and BDA also enable users to work with, manipulate, and surface actionable insights faster from large, complex datasets, by identifying patterns, relationships, anomalies, and trends in the data .
- AI and BDA can provide various benefits for various domains and applications, such as healthcare, education, finance, marketing, security, and social media, by improving the quality, accuracy, and reliability of data analysis and decision making .

Nature of data in Big Data

- Big data is a term that refers to the large, complex and diverse data sets that are generated from various sources and applications, such as social media, sensors, web logs, e-commerce, etc.
- Big data has three main characteristics, often known as the 3Vs: volume, variety and velocity.
- Volume refers to the amount of data that is collected and stored, which can range from terabytes to petabytes and beyond.
- Variety refers to the different types of data that are included in big data, such as structured, unstructured and semistructured data.
- Structured data is data that has a predefined format and schema, such as transactions, financial records, etc.
- Unstructured data is data that has no fixed format or structure, such as text, images, videos, audio, etc.
- Semistructured data is data that has some elements of structure, such as web server logs, streaming data, JSON, XML, etc.
- Velocity refers to the speed at which data is generated, processed and analyzed, which can be near real-time or batch-based.
- Big data also has other characteristics, such as veracity, value and variability.
- Veracity refers to the quality and reliability of the data, which can be affected by noise, inconsistency, incompleteness, etc.
- Value refers to the potential benefits and insights that can be derived from the data, which can vary depending on the use case and the analysis methods.
- Variability refers to the changes and fluctuations in the data, such as the format, meaning, context, etc.
- Big data requires new and advanced technologies and techniques to store, manage, process and analyze the data, such as distributed systems, cloud computing, parallel processing, machine learning, etc.
- Big data also poses new challenges and opportunities for data scientists, who need to have a combination of skills and knowledge, such as statistics, programming, domain expertise, communication, etc.

Analytic Processes and Tools for Big Data

- Big data refers to large, complex, and diverse datasets that are generated from various sources and require advanced techniques and technologies to store, process, and analyze.
- Analytic processes for big data involve the following steps:
 - Data acquisition: collecting and integrating data from multiple sources, such as sensors, web logs, social media, etc.
 - Data preprocessing: cleaning, transforming, and enriching data to make it suitable for analysis, such as removing noise, handling missing values, normalizing, etc.

- Data analysis: applying various methods and algorithms to extract insights, patterns, and knowledge from data, such as descriptive, predictive, and prescriptive analytics, machine learning, data mining, etc.
- Data visualization: presenting and communicating the results of data analysis using graphical and interactive tools, such as charts, dashboards, maps, etc.
- Analytic tools for big data are software applications or platforms that support one or more of the above steps and enable users to perform data analysis tasks efficiently and effectively. Some examples of analytic tools for big data are:
 - Hadoop: an open-source framework that allows distributed processing of large datasets across clusters of computers using simple programming models, such as MapReduce.
 - Spark: an open-source framework that provides fast and general-purpose data processing capabilities, such as batch, streaming, SQL, graph, and machine learning.
 - R: an open-source programming language and environment that offers a wide range of statistical and graphical techniques for data analysis and visualization.
 - Python: an open-source programming language that has a rich set of libraries and frameworks for data analysis and machine learning, such as NumPy, pandas, scikit-learn, TensorFlow, etc.
 - Tableau: a commercial software that enables users to create and share interactive and visual data analysis reports and dashboards.

Analysis vs Reporting in Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and tools to process, store, and analyze.
- Analysis and reporting are two common tasks that are performed on big data to extract insights and communicate them to the stakeholders.
- Analysis is the process of exploring, transforming, and modeling big data to discover patterns, trends, anomalies, and relationships that can support decision making and problem solving.
- Reporting is the process of presenting the results of analysis in a clear, concise, and visual way that can convey the main findings and recommendations to the audience.
- Analysis and reporting are different in terms of their goals, methods, and outputs.
- The goal of analysis is to answer specific questions or test hypotheses about the big data, such as “What are the factors that influence customer satisfaction?” or “How can we optimize the marketing campaign?”
- The goal of reporting is to summarize and communicate the key insights and actions derived from the analysis, such as “Customer satisfaction is

positively correlated with product quality and service speed.” or “We should target the young and urban segments with personalized offers.”

- The methods of analysis involve various techniques and tools that can handle the volume, variety, velocity, and veracity of big data, such as data mining, machine learning, statistics, and visualization.
- The methods of reporting involve various formats and platforms that can deliver the information effectively and efficiently, such as dashboards, charts, tables, and narratives.
- The outputs of analysis are often intermediate and detailed, such as data tables, graphs, models, and algorithms that can be further refined and validated.
- The outputs of reporting are often final and concise, such as reports, slides, infographics, and stories that can be easily understood and acted upon.

Modern data analytic tools for Big Data

- Big data refers to the large and complex datasets that are generated from various sources and require advanced techniques and technologies to process, store, and analyze.
- Data analytics is the process of extracting meaningful insights from big data using various methods, such as statistics, machine learning, data mining, visualization, etc.
- Modern data analytic tools are software applications or platforms that enable users to perform data analytics tasks on big data efficiently and effectively.
- Some examples of modern data analytic tools for big data are:
 - Apache Spark: A distributed computing framework that provides fast and scalable data processing, machine learning, streaming, and graph analytics capabilities.
 - Apache Hadoop: An open-source software framework that supports distributed storage and processing of large datasets using the MapReduce programming model and the Hadoop Distributed File System (HDFS).
 - Apache Kafka: A distributed streaming platform that allows users to publish and subscribe to streams of data, and process them in real-time or batch mode.
 - Apache Flink: A stream processing framework that provides high-throughput, low-latency, and stateful data processing, as well as support for batch processing and machine learning.
 - Apache Hive: A data warehouse software that facilitates data analysis and querying on large datasets stored in HDFS using a SQL-like language called HiveQL.
 - Apache Pig: A high-level data analysis platform that allows users to write complex data transformations and queries using a scripting

language called Pig Latin.

- Apache Cassandra: A distributed database system that provides high availability, scalability, and performance for handling large volumes of structured and unstructured data.
- Apache HBase: A distributed, column-oriented database that provides random, real-time access to big data.
- MongoDB: A document-oriented database that stores data in JSON-like format and supports dynamic schemas, indexing, aggregation, and replication.
- Elasticsearch: A distributed, RESTful search and analytics engine that provides full-text search, faceted navigation, and data visualization capabilities.
- Tableau: A data visualization tool that allows users to create interactive dashboards and reports from various data sources.
- Power BI: A business intelligence and analytics tool that enables users to connect to, transform, and visualize data from various sources.
- R: A programming language and environment that provides a wide range of statistical and graphical techniques for data analysis and visualization.
- Python: A general-purpose programming language that supports multiple paradigms, such as object-oriented, functional, and procedural, and has a rich set of libraries and frameworks for data analysis, such as NumPy, pandas, scikit-learn, TensorFlow, etc.

Unit 2 - Hadoop and Map Reduce

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and Hadoop MapReduce.
- HDFS is a distributed file system that provides high-throughput access to data stored on multiple nodes in a cluster. HDFS replicates data blocks across different nodes to ensure fault tolerance and reliability.
- MapReduce is a programming model that enables parallel processing of large data sets by dividing them into smaller chunks called splits. Each split is processed by a mapper function that transforms the input data into key-value pairs. The key-value pairs are then shuffled and sorted by a reducer function that aggregates the values for each key and produces the final output.
- Hadoop MapReduce follows a master-slave architecture, where a single node called the JobTracker coordinates the execution of multiple tasks across different nodes called TaskTrackers. The JobTracker assigns tasks to TaskTrackers based on their availability and data locality. The TaskTrackers report their progress and status to the JobTracker periodically.
- Hadoop MapReduce supports two types of tasks: map tasks and reduce

tasks. Map tasks run the mapper function on each split and generate intermediate key-value pairs. Reduce tasks run the reducer function on each group of key-value pairs with the same key and generate the final output. The number of map tasks is determined by the number of splits, while the number of reduce tasks is specified by the user.

- Hadoop MapReduce provides a Java API for writing custom mapper and reducer classes, as well as a set of built-in classes for common operations such as sorting, filtering, joining, etc. Hadoop MapReduce also supports other languages such as Python, Ruby, etc. through the Hadoop Streaming interface, which allows users to write mapper and reducer scripts using standard input and output streams.
- Hadoop MapReduce is suitable for batch processing of large and structured data sets that can be easily partitioned and parallelized. Hadoop MapReduce is not suitable for real-time or interactive processing of data that requires low latency or complex logic. Hadoop MapReduce is also not efficient for processing small or sparse data sets that do not benefit from parallelization or data locality.

Hadoop

- Hadoop is an open-source software framework for storing data and running applications on clusters of commodity hardware.
- Hadoop provides massive storage for any kind of data, enormous processing power and the ability to handle virtually limitless concurrent tasks or jobs.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage.
- Hadoop can efficiently store and process large datasets ranging in size from gigabytes to petabytes of data.
- Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation.
- Hadoop provides a software framework for distributed storage and processing of big data using the MapReduce programming model.
- Hadoop consists of four main components: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce and Hadoop YARN.
- Hadoop Common contains the libraries and utilities needed by other Hadoop modules.
- HDFS is a distributed file system that provides high-throughput access to application data.
- MapReduce is a programming model for processing large data sets with a parallel, distributed algorithm on a cluster.
- YARN is a resource-management platform responsible for managing compute resources in clusters and using them for scheduling of users' applications.

- Hadoop is used for various big data applications running under clustered systems, such as data analysis, data mining, data warehousing, machine learning, natural language processing, etc.

History of Hadoop

- Hadoop is an open-source framework for distributed storage and processing of large-scale data sets.
- Hadoop was created by Doug Cutting and Mike Cafarella in 2005, inspired by Google's MapReduce and Google File System papers.
- Hadoop was originally developed as a subproject of Apache Nutch, a web crawler and search engine.
- Hadoop was named after a toy elephant that belonged to Cutting's son.
- Hadoop became an Apache top-level project in 2008, and has since grown to include several related projects, such as HBase, Hive, Pig, Spark, and ZooKeeper.
- Hadoop is widely used by many organizations, such as Facebook, Yahoo, Netflix, Twitter, and LinkedIn, for various applications, such as data analysis, machine learning, web indexing, and recommendation systems.

Apache Hadoop

- Apache Hadoop is a collection of open-source software utilities that facilitates using a network of many computers to solve problems involving massive amounts of data and computation .
- Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models .
- Apache Hadoop is designed to scale up from single servers to thousands of machines, each offering local computation and storage.
- Apache Hadoop consists of four main modules: Hadoop Common, Hadoop Distributed File System (HDFS), Hadoop MapReduce, and Hadoop YARN .
- Hadoop Common contains the common utilities and libraries that support other Hadoop modules.
- Hadoop Distributed File System (HDFS) is a distributed file system that provides high-throughput access to application data and can store large amounts of data across multiple machines .
- Hadoop MapReduce is a programming model and software framework for writing applications that process large amounts of data in parallel on clusters of nodes using the map and reduce functions .
- Hadoop YARN is a resource management platform that manages the computing resources and scheduling of user applications in a Hadoop cluster .

Hadoop Distributed File System

- Hadoop Distributed File System (HDFS) is the primary data storage system used by Hadoop applications.
- HDFS is a distributed file system that provides high-throughput access to large data sets across scalable Hadoop clusters .
- HDFS employs a NameNode and DataNode architecture to implement the file system .
 - NameNode is the master node that manages the file system namespace and regulates access to files by clients.
 - DataNode is the slave node that stores the actual data in the form of blocks.
- HDFS splits files into large blocks (typically 64 MB or 128 MB) and distributes them across nodes in a cluster .
- HDFS also transfers packaged code into nodes to process the data in parallel using MapReduce, a programming model for large-scale data processing .
- HDFS is designed to handle hardware failures, data replication, load balancing, and data locality .
- HDFS has a web-based user interface that allows users to browse the file system, view file contents, and perform administrative tasks.

Components of Hadoop

Hadoop is a framework for distributed storage and processing of large-scale data. It consists of the following core components:

- **Hadoop Distributed File System (HDFS):** This is the storage layer of Hadoop that stores data in a distributed manner across multiple nodes in a cluster. HDFS provides high availability, fault tolerance, scalability, and data locality. HDFS splits the data into blocks and replicates them across different nodes for redundancy and faster access. HDFS also supports various file formats and compression techniques.
- **Hadoop MapReduce:** This is the processing layer of Hadoop that allows parallel execution of user-defined functions on the data stored in HDFS. MapReduce consists of two phases: map and reduce. The map phase applies a function to each block of data and produces intermediate key-value pairs. The reduce phase aggregates the intermediate values based on the keys and produces the final output. MapReduce can handle structured, semi-structured, and unstructured data. MapReduce also supports various programming languages and frameworks.
- **Hadoop YARN:** This is the resource management layer of Hadoop that allocates and manages the resources (such as CPU, memory, disk, and network) for the applications running on the cluster. YARN consists of two components: a resource manager and a node manager. The resource manager is the master node that oversees the cluster resources and schedules the applications. The node manager is the slave node that monitors and

reports the resource usage and status of the node. YARN also supports various types of applications, such as batch, interactive, streaming, and machine learning.

These are the core components of Hadoop that enable it to handle the challenges of Big Data, such as volume, velocity, variety, and value. Hadoop also has other components, such as Hadoop Common, Hadoop ZooKeeper, Hadoop Hive, Hadoop HBase, Hadoop Spark, and Hadoop Pig, that provide additional functionalities and features.

Data Format Co

- Data format co is a hypothetical company that provides data conversion and transformation services to various clients.
- Data format co specializes in converting data from one format to another, such as CSV, JSON, XML, SQL, etc.
- Data format co also offers data transformation services, such as data cleansing, data validation, data aggregation, data analysis, etc.
- Data format co uses various tools and techniques to perform data conversion and transformation, such as Python, Pandas, BeautifulSoup, SQL, etc.
- Data format co has a team of data engineers, data analysts, and data scientists who work on different projects and tasks.
- Data format co follows a standard workflow for each project, which includes the following steps:
 - Data collection: Data format co collects the data from the client or from other sources, such as web scraping, APIs, databases, etc.
 - Data conversion: Data format co converts the data from the original format to the desired format, using appropriate tools and libraries.
 - Data transformation: Data format co transforms the data according to the client's specifications, such as data cleansing, data validation, data aggregation, data analysis, etc.
 - Data delivery: Data format co delivers the converted and transformed data to the client, in the desired format and medium, such as files, databases, web pages, etc.
- Data format co charges the client based on the complexity and volume of the data, the time and resources required, and the quality and accuracy of the data.

Analyzing data with Hadoop

Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. It consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.

- HDFS is a distributed file system that stores data in blocks across multiple nodes in a cluster. It provides high availability, fault tolerance, scalability,

and parallelism.

- MapReduce is a programming model that allows users to write applications that process large amounts of data in parallel on multiple nodes. It consists of two phases: map and reduce.
 - Map phase: The input data is split into key-value pairs and assigned to different map tasks. Each map task applies a user-defined function to the key-value pairs and produces intermediate key-value pairs as output.
 - Reduce phase: The intermediate key-value pairs are shuffled and sorted by key and assigned to different reduce tasks. Each reduce task applies a user-defined function to the key-value pairs with the same key and produces final key-value pairs as output.

Some examples of data analysis tasks that can be performed with Hadoop are:

- Word count: Counting the frequency of words in a large text corpus.
- Inverted index: Building an index of words and their locations in a collection of documents.
- PageRank: Computing the importance of web pages based on the links between them.
- K-means clustering: Partitioning a set of data points into k groups based on their similarity.
- Recommendation system: Generating personalized recommendations for users based on their preferences and behavior.

Scaling out with Hadoop

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.
- HDFS is a distributed file system that provides high-throughput access to data stored on multiple nodes in a cluster. HDFS replicates data blocks across different nodes to ensure fault tolerance and availability.
- MapReduce is a programming model that enables parallel processing of data on HDFS. MapReduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input data block and produces intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.
- Hadoop enables scaling out by adding more nodes to a cluster without changing the application code. Hadoop automatically distributes the data and the computation across the nodes and handles failures and load balancing.
- Hadoop can scale out to thousands of nodes and handle petabytes of data. Hadoop is suitable for applications that require batch processing of large

and unstructured data, such as web log analysis, text mining, recommendation systems, etc.

Hadoop streaming

- Hadoop streaming is a utility that comes with the Hadoop distribution. It allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer .
- Hadoop streaming works by passing the input data to the mapper as the standard input and collecting the output data from the standard output. Similarly, the reducer receives the intermediate data from the standard input and writes the final output to the standard output .
- Hadoop streaming uses the Hadoop jar command to launch the streaming jobs. The command takes various options to specify the input and output directories, the mapper and reducer scripts, the input and output formats, the partitioner, the combiner, and other parameters .
- Hadoop streaming supports various languages such as Python, Ruby, Perl, Bash, and R for writing the mapper and reducer scripts. The scripts should follow the Hadoop streaming protocol, which is based on the key-value pairs separated by tabs .
- Hadoop streaming is a powerful feature that enables users to leverage the scalability and reliability of Hadoop for processing large-scale data with any programming language of their choice.

Hadoop pipes

- Hadoop pipes is the name of the C++ interface to Hadoop MapReduce .
- Hadoop pipes allows users to write map and reduce functions in C++ and run them on a Hadoop cluster.
- Hadoop pipes uses sockets as the channel over which the tasktracker communicates with the process running the C++ map or reduce function . JNI is not used.
- Hadoop pipes requires users to compile the C++ code and link it with the pipes library, which provides the implementation of the Hadoop API in C++.
- Hadoop pipes also requires users to implement a factory class that creates map and reduce objects on demand.
- Hadoop pipes can be run on any platform that supports sockets and C++, such as Linux, Windows, or IBM PowerLinux.
- Hadoop pipes can handle large data records by using the RecordReader and RecordWriter interfaces, which allow users to define how to split and serialize the data.

Hadoop Ecosystem

- The Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop.

- Apache Hadoop is a software library that allows for the distributed processing of large data sets across clusters of computers using simple programming models.
- Hadoop is designed to scale up from a single computer to thousands of clustered computers, with each machine offering local computation and storage .
- Hadoop enables multiple types of analytic workloads to run on the same data, at the same time, at massive scale on industry-standard hardware.
- The Hadoop Ecosystem consists of the following components:
 - HDFS: Hadoop Distributed File System, a distributed and fault-tolerant file system that stores data across multiple nodes.
 - YARN: Yet Another Resource Negotiator, a resource management layer that allocates and schedules tasks on the cluster.
 - MapReduce: A programming model and framework for parallel data processing using key-value pairs.
 - Spark: An in-memory data processing engine that supports batch, streaming, SQL, machine learning, and graph analytics.
 - PIG, HIVE: Query based processing of data services that provide a high-level abstraction over MapReduce.
 - HBase: A NoSQL database that provides random access and consistent updates to large amounts of structured and semi-structured data.
 - Sqoop, Flume, Kafka: Data ingestion tools that transfer data from various sources to HDFS or HBase.
 - Oozie, Zookeeper: Coordination and workflow management tools that orchestrate and monitor the execution of Hadoop jobs.
 - Mahout, MLlib: Machine learning libraries that provide scalable and distributed algorithms for data mining and analysis.
 - Ambari, Hue: Web-based user interfaces that provide administration and visualization of the Hadoop cluster and its components.

Map Reduce

- MapReduce is a programming model introduced by Google for processing and generating large data sets on clusters of computers .
- MapReduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input record and produces a set of intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.
- MapReduce is designed to handle large-scale data analysis in a distributed and fault-tolerant manner. It can run on commodity hardware and scale up to thousands of nodes .
- MapReduce is often used with Apache Hadoop, a data analytics engine widely used for Big Data. Hadoop provides a distributed file system (HDFS) and a resource management framework (YARN) for MapReduce applications .

- MapReduce can be used for various applications, such as web indexing, data mining, machine learning, log analysis, text processing, and image processing .

Map Reduce framework and basics

- MapReduce is a programming model or pattern within the Hadoop framework that is used to access big data stored in the Hadoop File System (HDFS) .
- MapReduce consists of two distinct tasks – Map and Reduce. The map function takes input pairs, processes, and produces another set of intermediate pairs as output. The reduce function takes the intermediate pairs, groups them by key, and performs some aggregation or computation on the values .
- The MapReduce framework consists of a single master ResourceManager, one worker NodeManager per cluster-node, and MRAppMaster per application. The ResourceManager allocates resources to the applications and monitors their progress. The NodeManager runs the map and reduce tasks on the cluster nodes. The MRAppMaster coordinates the execution of the application and communicates with the ResourceManager and the NodeManager .
- MapReduce is used for various applications such as log analysis, full-text indexing, PageRank calculation, reverse web-link graph, machine learning, and data mining .
- MapReduce is a software framework for processing large data sets in a distributed fashion over several machines. The core idea behind MapReduce is mapping your data set into a collection of <key, value> pairs, and then reducing over all pairs with the same key .

How MapReduce works

MapReduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.

The basic steps of MapReduce are:

- **Map:** A user-defined function that takes an input key-value pair and produces a set of intermediate key-value pairs. The input data is split into smaller blocks and assigned to different map tasks that run in parallel on different nodes in the cluster .
- **Shuffle:** The framework collects the intermediate key-value pairs from the map tasks and redistributes them to the reduce tasks based on the intermediate keys. This ensures that all the values associated with the same key are sent to the same reduce task .
- **Reduce:** A user-defined function that takes an intermediate key and a set of values for that key, and merges those values into a smaller set of

values. The reduce tasks process the shuffled data and produce the final output .

- **Combine and Partition:** Optional steps that can optimize the performance of MapReduce. The combine function can perform a local aggregation of the intermediate values to reduce the amount of data shuffled. The partition function can control how the intermediate keys are distributed among the reduce tasks.

Some of the features and uses of MapReduce are:

- It can handle large-scale, distributed, and fault-tolerant data processing on commodity hardware .
- It can support various types of data, such as structured, semi-structured, or unstructured .
- It can be used for various applications, such as web indexing, data mining, machine learning, log analysis, and more .
- It can be implemented in different languages, such as Java, Python, C++, and more .
- It can be integrated with other frameworks, such as Hadoop, Spark, and more .

Developing a Map Reduce application

- MapReduce is a framework for processing parallelizable problems across large datasets using a large number of computers (nodes), collectively referred to as a cluster or a grid.
- Writing a program in MapReduce follows a certain pattern. You start by writing your map and reduce functions, ideally with unit tests to make sure they do what you expect .
- The map function takes a key-value pair as input and produces a list of intermediate key-value pairs as output. The reduce function takes an intermediate key and a list of values associated with that key as input and produces a list of final key-value pairs as output .
- You can write your map and reduce functions in any language that supports reading from standard input and writing to standard output, such as Java, Python, Ruby, etc. You can use the Hadoop Streaming API to run your MapReduce program on a Hadoop cluster .
- You can also write your map and reduce functions in Java using the Hadoop MapReduce API, which provides more control and flexibility over the execution of your program. You can use the Hadoop Java API to access the configuration, input, output, and intermediate data of your MapReduce program .
- You need to configure your development environment to run your MapReduce program locally or on a cluster. You need to install Hadoop and its dependencies, such as Java, on your machine or on the cluster nodes. You also need to set up the Hadoop configuration files, such as `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, etc., to specify the parameters of your

Hadoop cluster, such as the location of the namenode, the number of map and reduce tasks, the memory and disk space allocated to each task, etc .

- You can use the `GenericOptionsParser`, `Tool`, and `ToolRunner` classes to parse the command-line arguments of your MapReduce program, such as the input and output paths, the number of reducers, the custom options, etc. You can also use the `Configuration` class to access the configuration XML and can be combined (if a var is repeated, last is used). You can use the `Job` class to create and submit a MapReduce job to the cluster .
- You can use the JUnit framework to write unit tests for your map and reduce functions. You can use the `MRUnit` library to mock the input and output of your map and reduce functions and to verify the expected results. You can also use the `MiniDFSCluster` and `MiniMRCluster` classes to create a local Hadoop cluster for testing your MapReduce program .
- You can run your MapReduce program locally or on a cluster using the Hadoop command-line interface. You can use the `hadoop jar` command to run a Java MapReduce program, or the `hadoop streaming` command to run a non-Java MapReduce program. You can use the `hadoop fs` command to interact with the Hadoop Distributed File System (HDFS), where you can store your input and output data .
- You can use the MapReduce Web UI to monitor the progress and status of your MapReduce job. You can access the Web UI at <http://:/>, where you can see the list of running and completed jobs, the details of each job, such as the map and reduce tasks, the counters, the logs, the errors, etc .
- You can use the Hadoop logs to debug and troubleshoot your MapReduce program. You can find the logs in the `$HADOOP_HOME/logs` directory on each node of the cluster, or in the Web UI. You can use the `log4j.properties` file to configure the logging level and format of your MapReduce program .
- You can tune your MapReduce job to improve its performance and efficiency. You can use the counters to measure the statistics and metrics of your MapReduce job, such as the number of input and output records, the number of bytes read and written, the number of map and reduce tasks, the CPU and memory usage, etc. You can use the combiner function to reduce the amount of data transferred between the map and reduce tasks. You can

Unit Tests with MR Unit

- MR Unit is a Java library that allows you to write unit tests for Hadoop MapReduce jobs.
- MR Unit is based on JUnit and Mockito frameworks and provides classes and methods to simulate the execution of mappers, reducers, and map-reduce chains.
- MR Unit helps you to develop and debug your MapReduce code without the need of a Hadoop cluster or a large amount of data.

- MR Unit supports testing different aspects of MapReduce jobs, such as input and output formats, counters, configuration, partitioning, sorting, and combiners.
- MR Unit also allows you to mock external dependencies, such as distributed cache files, HBase tables, or Hive queries, using Mockito or other mocking frameworks.
- To use MR Unit, you need to add the mrunit jar file and its dependencies to your project's classpath. You can download the latest version of mrunit from <https://mvnrepository.com/artifact/org.apache.mrunit/mrunit>.
- To write a unit test with MR Unit, you need to create a driver object for the component you want to test, such as a MapperDriver, a ReducerDriver, or a MapReduceDriver.
- You can use the driver object to set up the input and output key-value pairs, the configuration, the counters, and the mocks for your test case.
- You can use the driver object to run the test case and verify the output, the counters, and the mocks using JUnit assertions or matchers.
- You can also use the driver object to debug your code by setting break-points in your mapper or reducer classes and stepping through the execution.
- Here is an example of a unit test for a mapper class that counts the number of words in a text file using MR Unit:

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mrunit.mapreduce.MapDriver;
import org.junit.Before;
import org.junit.Test;

public class WordCountMapperTest {

    private MapDriver<LongWritable, Text, Text, IntWritable> mapDriver;

    @Before
    public void setUp() {
        // create a mapper instance
        WordCountMapper mapper = new WordCountMapper();
        // create a driver for the mapper
        mapDriver = new MapDriver<>(mapper);
    }

    @Test
    public void testMapper() throws Exception {
        // set up the input key-value pair
        mapDriver.withInput(new LongWritable(0), new Text("Hello World"));
        // set up the expected output key-value pairs
    }
}
```

```

mapDriver.withOutput(new Text("Hello"), new IntWritable(1));
mapDriver.withOutput(new Text("World"), new IntWritable(1));
// run the test and verify the output
mapDriver.runTest();
}
}

```

Test data and local tests in map reduce

- Test data is the input data that is used to test the functionality and performance of a map reduce program. Test data can be generated artificially or obtained from real sources, depending on the requirements and objectives of the testing.
- Local tests are the tests that are performed on a single machine, without using a distributed cluster. Local tests are useful for debugging and validating the logic of the map and reduce functions, as well as the data flow and format between them.
- Some of the ways to perform local tests in map reduce are:
 - Using hadoop streaming, which allows writing map and reduce scripts in any language that can read from standard input and write to standard output. For example, if the map and reduce scripts are written in Python, they can be tested locally like this:

```
cat *.csv | map.py | sort -k1,1 | reducer.py
```

This command simulates the map reduce process by piping the input data to the map script, sorting the output by key, and piping it to the reduce script.

- Using MRUnit, which is a testing framework that lets you test and debug map reduce jobs in isolation without spinning up a hadoop cluster. MRUnit provides mock objects and drivers for testing the map and reduce functions, as well as the combiners, partitioners, and counters. For example, to test a mapper class using MRUnit, one can write a test case like this:

```

public class WordCountMapperTest extends TestCase {

    public void testMapper() {
        Mapper mapper = new WordCountMapper();
        MapperDriver driver = new MapperDriver(mapper);

        driver.withInput(new LongWritable(1), new Text("cat cat dog"))
            .withOutput(new Text("cat"), new IntWritable(1))
            .withOutput(new Text("cat"), new IntWritable(1))
            .withOutput(new Text("dog"), new IntWritable(1))
    }
}

```

```

        .runTest();
    }
}

```

This test case verifies that the mapper produces the expected key-value pairs for a given input.

- Using JUnit, which is a general-purpose testing framework for Java programs. JUnit can be used to test the map and reduce classes, as well as the main driver class, by creating test cases and assertions. For example, to test a reducer class using JUnit, one can write a test case like this:

```

public class WordCountReducerTest {

    @Test
    public void testReducer() throws IOException {
        Reducer reducer = new WordCountReducer();
        Reducer.Context context = mock(Reducer.Context.class);

        List<IntWritable> values = new ArrayList<IntWritable>();
        values.add(new IntWritable(1));
        values.add(new IntWritable(1));
        values.add(new IntWritable(1));

        reducer.reduce(new Text("cat"), values, context);

        verify(context).write(new Text("cat"), new IntWritable(3));
    }
}

```

This test case mocks the reducer context and verifies that the reducer produces the expected output for a given key and list of values.

Anatomy of a Map Reduce job run

- A Map Reduce job is a unit of work that consists of a map function and a reduce function, applied to a set of input data.
- A Map Reduce job run is the process of executing a Map Reduce job on a distributed system, such as Hadoop.
- A Map Reduce job run involves the following steps:
 - The job client submits the job to the job tracker, which is the master node that coordinates the job execution.
 - The job tracker assigns a unique ID to the job and splits the input data into fixed-size chunks called input splits. Each input split is assigned to a map task, which is a unit of work for the map function.

- The job tracker locates the data nodes that store the input splits and assigns the map tasks to the task trackers, which are the worker nodes that run the map and reduce tasks.
- The task trackers launch the map tasks and run the map function on each input split. The map function processes the input key-value pairs and emits intermediate key-value pairs as the output.
- The intermediate key-value pairs are partitioned by a partitioner function, which determines which reduce task will receive them. The default partitioner uses a hash function on the intermediate keys.
- The intermediate key-value pairs are sorted by the intermediate keys and grouped by the same keys. The sorted and grouped intermediate key-value pairs are called shuffle output.
- The shuffle output is transferred to the reduce tasks, which are assigned by the job tracker to the task trackers. The reduce tasks run the reduce function on each group of intermediate key-value pairs with the same key. The reduce function combines the values and emits the final output key-value pairs.
- The final output key-value pairs are written to the output files in the output directory specified by the job client. The output files are stored in the distributed file system, such as HDFS.
- The job tracker monitors the progress and status of the map and reduce tasks. If a task fails or times out, the job tracker reschedules it to another task tracker. The job tracker also handles the speculative execution of tasks, which is a technique to improve the performance by running backup tasks on idle nodes.
- The job tracker notifies the job client when the job is completed or failed. The job client can also query the job tracker for the job status and statistics.

Failures in MapReduce

- MapReduce is a programming model and framework for processing large-scale data sets in parallel and distributed environments.
- MapReduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase applies another user-defined function to all the values associated with the same intermediate key and produces a set of output key-value pairs.
- MapReduce can handle failures of individual machines or tasks by re-executing them on other available machines or tasks. However, failures can still affect the performance and correctness of MapReduce applications in various ways.
- Some of the common types of failures in MapReduce are:
 - **Worker failures:** A worker is a machine or a process that executes

map or reduce tasks. A worker can fail due to hardware faults, network errors, software bugs, or resource exhaustion. Worker failures can cause the loss of intermediate or output data, or delay the completion of the MapReduce job.

- **Master failures:** A master is a machine or a process that coordinates the execution of MapReduce jobs. It assigns tasks to workers, monitors their progress, and handles failures. A master can fail due to similar reasons as workers. Master failures can cause the loss of the entire MapReduce job, or require restarting the job from scratch.
- **Task failures:** A task is a unit of work assigned to a worker by the master. A task can fail due to incorrect or invalid input data, user-defined function errors, or worker failures. Task failures can cause the loss of intermediate or output data, or require re-executing the task on another worker.
- **Data failures:** A data failure occurs when the input, intermediate, or output data of a MapReduce job is corrupted, missing, or inaccessible. Data failures can be caused by disk failures, network failures, or human errors. Data failures can cause the failure of tasks or workers that depend on the data, or require re-generating or re-fetching the data from other sources.

Job scheduling in MapReduce

- Job scheduling is the process of assigning tasks to workers in a distributed system, such as MapReduce, to optimize the performance and resource utilization of the system.
- MapReduce is a programming model and framework for processing large-scale data sets in parallel using multiple machines, or nodes, in a cluster.
- A MapReduce job consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase applies another user-defined function to all the intermediate values associated with the same intermediate key and produces a set of output key-value pairs.
- Job scheduling in MapReduce involves deciding how to partition the input data, how to assign map and reduce tasks to nodes, how to balance the load among nodes, and how to handle failures and stragglers.
- There are different job scheduling algorithms and policies for MapReduce, such as FIFO, Fair, Capacity, and Delay. Each algorithm has its own advantages and disadvantages, depending on the characteristics of the jobs and the cluster.
- FIFO (First-In-First-Out) is the simplest job scheduling algorithm, which assigns tasks to nodes in the order of their arrival. FIFO is easy to implement and understand, but it can cause starvation and poor resource utilization if there are long-running jobs or heterogeneous jobs in the queue.
- Fair is a job scheduling algorithm that aims to allocate resources fairly among jobs, such that each job gets an equal share of the cluster over

time. Fair can improve the response time and throughput of the system, but it can also introduce overhead and complexity in managing the fair shares and the preemption of tasks.

- Capacity is a job scheduling algorithm that allows multiple queues with different capacities and priorities to be defined for different types of jobs. Capacity can improve the resource utilization and the quality of service of the system, but it can also require careful tuning and configuration of the queue parameters and the admission control policies.
- Delay is a job scheduling algorithm that tries to balance the trade-off between fairness and data locality, which is the degree to which tasks are assigned to nodes that have the input data locally. Delay can improve the performance and the network bandwidth consumption of the system, but it can also introduce latency and unpredictability in the task execution.

Shuffle and Sort in Map Reduce

- Map Reduce is a programming model for processing large-scale data sets in parallel and distributed manner.
- Map Reduce consists of two phases: map and reduce. In the map phase, each input record is transformed into a key-value pair by a user-defined function. In the reduce phase, all the values associated with the same key are aggregated by another user-defined function.
- Shuffle and Sort is an intermediate phase between map and reduce. It is performed by the Map Reduce framework and is not visible to the user.
- Shuffle and Sort has two main functions:
 - To transfer the map outputs to the reducers as inputs.
 - To sort the map outputs by key before feeding them to the reducers.
- Shuffle and Sort consists of the following steps:
 - Partitioning: The map outputs are partitioned by a hash function based on the key. Each partition corresponds to a reducer and is stored in a local disk of the mapper node.
 - Spilling: The map outputs are periodically written to the local disk as intermediate files. This is done to avoid running out of memory. The intermediate files are sorted by key and optionally compressed.
 - Merging: The intermediate files are merged into a single sorted file per partition. This is done by a priority queue that maintains the smallest key from each file.
 - Copying: The sorted files are copied from the mapper nodes to the reducer nodes over the network. This is done by a background thread that fetches the files from the mappers as they become available.
 - Sorting: The sorted files are merged again by the reducer node into a single sorted input. This is done by the same priority queue technique as before.
- Shuffle and Sort is a crucial phase in Map Reduce as it affects the performance and scalability of the job. Some of the factors that influence the shuffle and sort are:

- The number and size of the map outputs and the intermediate files.
- The number and location of the reducers and the network bandwidth.
- The partitioning function and the key distribution.
- The sorting algorithm and the comparator function.
- The compression codec and the serialization format.

Task execution in MapReduce

- MapReduce is a programming model designed to process large amount of data in parallel by dividing the job into several independent local tasks.
- A MapReduce job consists of two types of tasks: map tasks and reduce tasks.
- Map tasks take input data and apply a user-defined function to generate intermediate key-value pairs.
- Reduce tasks take intermediate key-value pairs and apply another user-defined function to aggregate them and produce the final output.
- The execution of tasks is controlled by two types of entities: a JobTracker and multiple TaskTrackers.
- The JobTracker acts like a master, responsible for scheduling, monitoring and coordinating the tasks. It also handles the failures and retries of tasks.
- The TaskTrackers act like slaves, each of them performing the tasks assigned by the JobTracker and reporting the progress and status to it.
- The JobTracker and the TaskTrackers communicate through a heartbeat mechanism, where the TaskTrackers periodically send a message to the JobTracker indicating their availability and health.
- The framework sorts the outputs of the map tasks, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a file-system, such as HDFS.
- The framework takes care of distributing the data, balancing the load, and optimizing the performance of the tasks.

Map Reduce types in map reduce

MapReduce is a programming model for processing large-scale data sets in parallel and distributed manner. It consists of two main functions: map and reduce.

- Map: The map function takes an input key-value pair and produces a list of intermediate key-value pairs. The input and output types of the map function can be different. For example, the map function can take a line of text as input and output a list of words and their frequencies as intermediate key-value pairs.
- Reduce: The reduce function takes an intermediate key and a list of values associated with that key and produces a list of output key-value pairs. The output types of the reduce function can be different from the input types. For example, the reduce function can take a word and a list of frequencies as input and output the word and its total frequency as output key-value pair.

MapReduce types are the data types used for the input and output of the map and reduce functions. They are defined as follows:

- Map input key type (K1): The type of the key for the input key-value pair of the map function.
- Map input value type (V1): The type of the value for the input key-value pair of the map function.
- Map output key type (K2): The type of the key for the intermediate key-value pair produced by the map function.
- Map output value type (V2): The type of the value for the intermediate key-value pair produced by the map function.
- Reduce input key type (K2): The type of the key for the input key-value pair of the reduce function. It is the same as the map output key type.
- Reduce input value type (V2): The type of the value for the input key-value pair of the reduce function. It is the same as the map output value type.
- Reduce output key type (K3): The type of the key for the output key-value pair produced by the reduce function.
- Reduce output value type (V3): The type of the value for the output key-value pair produced by the reduce function.

MapReduce types are important for specifying the input and output formats of the MapReduce job. They also determine how the data is serialized, deserialized, partitioned, sorted, and grouped by the MapReduce framework. Different MapReduce types can be implemented using different classes in Java, such as Writable, WritableComparable, Text, IntWritable, LongWritable, etc.

Input Formats in Map Reduce

- Input formats are the classes that define how the input data for a MapReduce job is split, read, and processed.
- Input formats are responsible for creating input splits, which are the logical units of work that each mapper operates on.
- Input formats also provide a record reader, which is an object that reads the input split and converts it into key-value pairs that are suitable for the mapper.
- There are different types of input formats in MapReduce, each suited for different kinds of input data. Some of the common input formats are:
 - FileInputFormat: It is the base class for all file-based input formats. It handles common tasks such as splitting files into input splits, validating input paths, and filtering hidden files.
 - TextInputFormat: It is the default input format. It reads each line of a text file as a record, and splits the line by a delimiter (default is tab) into a key and a value. The key is the byte offset of the line, and the value is the line content.

- `KeyValueTextInputFormat`: It is similar to `TextInputFormat`, but it splits the line by the first occurrence of the delimiter, instead of the last. The key is the part before the delimiter, and the value is the part after the delimiter.
- `SequenceFileInputFormat`: It is an input format that reads sequence files, which are binary files that store key-value pairs in a compressed and serialized format. Sequence files are suitable for storing large amounts of data that can be efficiently processed by MapReduce.
- `SequenceFileAsTextInputFormat`: It is a variant of `SequenceFileInputFormat` that converts the binary key-value pairs into text key-value pairs, by calling the `toString()` method on the keys and values. This is useful for debugging or testing purposes.
- `SequenceFileAsBinaryInputFormat`: It is another variant of `SequenceFileInputFormat` that preserves the binary key-value pairs as byte arrays, without any conversion. This is useful for processing binary data that cannot be converted to text.
- `NLineInputFormat`: It is an input format that splits the input file into input splits based on the number of lines, instead of the file size. Each input split contains N lines, where N is specified by the user. This is useful for processing files that have a logical structure based on the number of lines, such as XML documents or log files.
- `DBInputFormat`: It is an input format that reads data from a relational database, using JDBC. It allows the user to specify a SQL query to select the data to be processed by MapReduce. The query results are converted into key-value pairs, where the key is the primary key of the table, and the value is a `DBWritable` object that contains the rest of the columns.

Output Formats in Map Reduce

Output formats in Map Reduce are the classes that define how the output files of a Map Reduce job are stored in a file system. They also provide the `RecordWriter` implementation to write the output records of the mappers and reducers. The output format can be specified by the `FileOutputFormat.setOutputPath()` method in the job configuration. Some of the common output formats in Map Reduce are:

- `TextOutputFormat`: This is the default output format that writes plain text files. Each record is a line of text that consists of the key and the value separated by a tab character. The keys and values are converted to strings using the `toString()` method.
- `SequenceFileOutputFormat`: This output format writes sequence files, which are binary files that store serialized key-value pairs. Sequence files are efficient and compact, and can be compressed and split. They are suitable for storing intermediate data between Map Reduce jobs or for storing large amounts of data that need fast random access.

- `SequenceFileAsBinaryOutputFormat`: This is a variant of `SequenceFileOutputFormat` that writes the keys and values as binary data instead of using serialization. This can be useful for performance reasons or for handling complex data types that are not easily serialized.
- `MapFileOutputFormat`: This output format writes map files, which are a special type of sequence files that support random access by key. Map files consist of two sequence files: a data file that stores the key-value pairs and an index file that stores the keys and their offsets in the data file. Map files are useful for creating indexes or lookup tables that can be queried by key.
- `MultipleOutputs`: This is a utility class that allows writing to multiple output files from a single Map Reduce job. It can be used to write different types of output files based on the keys or values, or to write to different directories or file systems. `MultipleOutputs` can be used with any output format, and it provides methods to create and write to named outputs.
- `LazyOutputFormat`: This is a wrapper output format that prevents the creation of empty output files. It can be used with any output format, and it delays the creation of the output file until the first record is written to it. This can be useful for reducing the number of output files when the number of reducers is large or when the output is filtered or sparse.
- `DBOutputFormat`: This output format writes the output records to a relational database table using JDBC. It can be used to export data from Hadoop to a database, or to perform aggregation or join operations with database tables. `DBOutputFormat` requires specifying the database connection parameters, the table name, and the column names in the job configuration.

Map Reduce features

- Map Reduce is a programming model and an associated implementation for processing and generating large data sets with a parallel, distributed algorithm on a cluster.
- Map Reduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key-value pair and produces a set of intermediate key-value pairs. The reduce phase aggregates all the intermediate values associated with the same intermediate key and produces the final output.
- Map Reduce is designed to handle large-scale data processing on commodity hardware. It can scale up from a single machine to thousands of machines, each offering local computation and storage.
- Map Reduce is fault-tolerant and resilient to failures. It automatically handles the failures of machines, processes, or network partitions. It also provides mechanisms for monitoring and debugging the execution of the tasks.
- Map Reduce is expressive and flexible. It can support a wide range of applications, such as data mining, machine learning, text processing, web

crawling, log analysis, and more. It can also be extended with user-defined functions, combiners, partitioners, and custom input and output formats.

Real-world Map Reduce

- MapReduce is a programming model and an associated implementation for processing and generating large data sets in parallel and distributed manner.
- MapReduce consists of two phases: map and reduce. The map phase applies a user-defined function to each input key/value pair and produces a set of intermediate key/value pairs. The reduce phase merges all intermediate values associated with the same intermediate key using another user-defined function.
- MapReduce is designed to handle large-scale data processing on clusters of commodity hardware, and provides fault tolerance, load balancing, data locality, scalability and simplicity.
- MapReduce can be used for various applications, such as word count, inverted index, web link analysis, machine learning, data mining, etc.
- One real-world example of MapReduce is how Twitter manages its tweets. Twitter receives around 500 million tweets per day, which is nearly 3000 tweets per second. To analyze the tweets, Twitter uses MapReduce to perform tasks such as filtering, aggregating, counting, sorting, etc.
- The following illustration shows how Twitter uses MapReduce to count the number of tweets per user:

MapReduce example

- In the map phase, each tweet is mapped to a key/value pair, where the key is the user name and the value is 1. The map output is then shuffled and sorted by the key, and sent to the reduce phase.
- In the reduce phase, each reducer receives a list of values for each key, and sums up the values to get the total number of tweets per user. The reduce output is then stored in a file system or a database.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

- HDFS is a distributed file system that handles large data sets running on commodity hardware.
- HDFS is one of the major components of Apache Hadoop, the others being MapReduce and YARN.
- HDFS provides high-performance access to data across highly scalable Hadoop clusters.
- HDFS employs a NameNode and DataNode architecture to implement a distributed file system.
- The NameNode is the master node that manages the file system namespace and regulates access to files by clients.

- The DataNodes are the slave nodes that store the actual data in blocks and perform read and write operations as instructed by the NameNode.
- HDFS is designed to be fault-tolerant, reliable, and efficient.
- HDFS supports a large number of concurrent clients, a large number of files, and a large file size.
- HDFS follows a write-once-read-many model, where files are appended to but not modified.
- HDFS provides a command-line interface and a Java API for interacting with the file system.
- HDFS also supports a web-based browser for viewing the file system status and contents.
- Hadoop Environment is the set of software and hardware components that are required to run Hadoop applications.
- Hadoop Environment includes the Hadoop core components (HDFS, MapReduce, and YARN), the Hadoop ecosystem components (such as Hive, Pig, Spark, etc.), and the underlying infrastructure (such as network, storage, memory, CPU, etc.).
- Hadoop Environment can be deployed on a single node, a cluster of nodes, or a cloud platform.
- Hadoop Environment requires proper configuration, tuning, and monitoring to ensure optimal performance and availability.

HDFS

HDFS stands for Hadoop Distributed File System. It is a distributed file system that handles large data sets running on commodity hardware. It is one of the major components of Apache Hadoop, the others being MapReduce and YARN.

Some of the main features and benefits of HDFS are:

- It is fault-tolerant and can recover from hardware failures and network partitions .
- It is scalable and can store and process petabytes of data across thousands of nodes .
- It is optimized for high-throughput and low-latency data access .
- It supports multiple data formats and types, such as structured, semi-structured, and unstructured data .
- It provides a simple and consistent interface for applications to interact with the data .

Some of the main components and concepts of HDFS are:

- NameNode: The master node that manages the namespace and the meta-data of the file system.
- DataNode: The worker node that stores and serves the data blocks of the files.

- **Block:** The smallest unit of data in HDFS, typically 64 MB or 128 MB in size.
- **Replication:** The mechanism of storing multiple copies of the same block on different DataNodes for fault-tolerance and load-balancing.
- **Rack:** A group of nodes that are physically close to each other and share a network switch.
- **Client:** The application or user that accesses the data in HDFS through the Hadoop API.

Design of HDFS

HDFS stands for Hadoop Distributed File System. It is a file system that is designed to store very large files across multiple machines in a cluster. HDFS has the following characteristics and features:

- HDFS is based on the Google File System (GFS) .
- HDFS was originally built as infrastructure for the Apache Nutch web search engine project .
- HDFS is a member of the Hadoop Ecosystem, which includes other components such as MapReduce, YARN, Hive, Pig, HBase, etc.
- HDFS is highly fault-tolerant and can handle hardware failures, network partitions, and data corruption .
- HDFS is optimized for streaming data access patterns, where files are read and written in large sequential blocks .
- HDFS relaxes some POSIX requirements to enable high throughput and scalability .
- HDFS supports replication of data blocks across multiple nodes to ensure data availability and durability .
- HDFS has a master-slave architecture, where a single NameNode manages the namespace and metadata of the file system, and multiple DataNodes store and serve the data blocks .
- HDFS provides a Java API and a command-line interface for clients to interact with the file system .
- HDFS also provides a web interface and a REST API for monitoring and administration .

: <https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs>
<https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

HDFS concepts

- HDFS stands for Hadoop Distributed File System. It is a distributed file system that stores large-scale data across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where one node acts as the NameNode (master) and the rest of the nodes act as DataNodes (slaves).

- The NameNode is responsible for managing the namespace, metadata, and access control of the files and directories in HDFS. It also coordinates the replication and placement of data blocks among the DataNodes.
- The DataNodes are responsible for storing the actual data blocks of the files in HDFS. They also perform read and write operations on the data blocks as instructed by the NameNode.
- HDFS splits a file into fixed-size blocks (typically 128 MB) and distributes them across the DataNodes for storage. Each block is replicated a number of times (default is 3) for fault tolerance and availability.
- HDFS provides a high-level abstraction of the files and directories, hiding the details of the block storage and replication from the users. Users can access the files and directories in HDFS using a standard file system interface or a web browser.
- HDFS is designed to handle large-scale data (petabytes or more) with high throughput and low latency. It is optimized for batch processing and sequential access of data, rather than random access or interactive queries.
- HDFS is also designed to be resilient to failures and self-healing. It can detect and recover from node failures, network failures, disk failures, and data corruption. It can also balance the load and utilization of the cluster by moving data blocks across the DataNodes.

Benefits of HDFS

- HDFS is a distributed file system that can store large amounts of data across multiple nodes in a cluster.
- HDFS provides high availability and fault tolerance by replicating data blocks on different nodes and automatically recovering from node failures.
- HDFS supports high throughput and scalability by allowing parallel processing and data streaming of large files.
- HDFS is designed for batch processing of structured and unstructured data, such as text, images, videos, etc.
- HDFS is compatible with various frameworks and tools for data analysis, such as MapReduce, Spark, Hive, Pig, etc.
- HDFS is open source and can be customized and extended according to the user's needs.

Challenges of HDFS

HDFS is a distributed file system that is designed to store and process large amounts of data across multiple nodes in a cluster. However, HDFS also has some limitations and challenges that need to be considered before using it. Some of the major challenges of HDFS are:

- **Issues with small files:** HDFS is not suitable for storing and accessing small files, as each file occupies a block of fixed size (typically 64 MB or 128 MB) and has a metadata entry in the NameNode, which is the master node that manages the file system namespace. This can lead to inefficient

use of disk space and memory, as well as increased network traffic and load on the NameNode. HDFS also lacks the ability to support random reading and writing of small files, as it is optimized for sequential access of large files .

- **Slow processing speed:** HDFS relies on MapReduce, a programming model that divides a large task into smaller subtasks that are executed in parallel by the worker nodes in the cluster. However, MapReduce has some drawbacks, such as high latency, high disk I/O, and lack of support for iterative and interactive processing. MapReduce also requires the data to be transferred between the nodes for each stage of the computation, which can be costly and time-consuming .
- **Support for batch processing only:** HDFS is designed for batch processing of large and static data sets, where the data is ingested once and processed periodically. It is not suitable for streaming data, where the data is continuously generated and processed in real time. HDFS also does not support transactions, concurrency, and updates, which are essential for some applications that require data consistency and integrity .
- **No real-time processing:** HDFS does not provide any mechanism for real-time processing of data, such as querying, filtering, aggregating, or analyzing the data as soon as it arrives. HDFS also does not support indexing, caching, or compression of data, which can improve the performance and efficiency of data processing. HDFS users have to rely on other frameworks or tools, such as Apache HBase, Apache Spark, or Apache Kafka, to enable real-time processing of data on top of HDFS .
- **Iterative processing:** HDFS is not well suited for iterative processing, where the same data set is processed multiple times with different parameters or algorithms. This is because HDFS does not allow the data to be reused or shared between different MapReduce jobs, and each iteration requires a new read and write of the data from and to the disk. This can result in high overhead and low performance. HDFS users have to use other frameworks or tools, such as Apache Spark, Apache Flink, or Apache Giraph, to enable iterative processing of data on top of HDFS .
- **Latency:** HDFS has a high latency, which is the time taken to access or process the data, due to several factors, such as the network communication between the nodes, the disk I/O, the replication of data blocks, the scheduling of MapReduce tasks, and the fault tolerance mechanisms. HDFS is not suitable for applications that require low latency, such as online gaming, web search, or social media .
- **No ease of use:** HDFS has a steep learning curve, as it requires the users to have a good understanding of the underlying architecture, configuration, and programming model of HDFS and MapReduce. HDFS also has a complex and cumbersome API, which makes it difficult to write, debug, and maintain the code. HDFS users have to use other frameworks or tools, such as Apache Pig, Apache Hive, or Apache Sqoop, to simplify the data ingestion, processing, and analysis on top of HDFS .

- **Security issue:** HDFS has a weak security model, as it does not provide any encryption, authentication, or authorization of data or users. HDFS relies on the underlying operating system or network for security, which can be vulnerable to attacks or breaches. HDFS also does not support data privacy or compliance, which are important for some applications that deal with sensitive or personal data. HDFS users have to use other frameworks or tools, such as Apache Ranger, Apache Knox, or Apache Kerberos, to enhance the security of data and users on top of HDFS .

File sizes in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS is designed to handle files that are typically in the range of gigabytes to terabytes in size.
- HDFS splits large files into fixed-size blocks, which are stored on different nodes in the cluster. The default block size is 128 MB, but it can be configured by the user.
- HDFS maintains multiple replicas of each block for fault tolerance and load balancing. The default replication factor is 3, but it can also be configured by the user.
- HDFS supports appending data to existing files, but not random writes or updates. This is because HDFS is optimized for sequential access and batch processing, rather than interactive or real-time applications.
- HDFS provides a command-line interface and a web interface for users to interact with the file system. Users can create, delete, copy, move, rename, and list files and directories in HDFS. Users can also check the status, location, and size of files and blocks in HDFS.

Block sizes in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS divides a file into one or more blocks, which are fixed-sized chunks of data that are stored on different nodes in the cluster.
- The default block size in HDFS is 64 MB for Hadoop 1.0 and 128 MB for Hadoop 2.0, but it can be configured manually by changing the `dfs.block.size` property in `hdfs-site.xml` .
- The advantage of using large block sizes in HDFS is that it reduces the number of disk seeks and network transfers, which improves the performance and throughput of data processing .
- The disadvantage of using large block sizes in HDFS is that it may cause internal fragmentation and waste disk space, especially if the file size is smaller than the block size .
- HDFS supports variable length blocks, which means that the last block of

a file can be smaller than the configured block size, and users can start a new block without filling out the last block.

- HDFS also supports replication, which means that each block is copied to multiple nodes in the cluster to ensure fault tolerance and availability. The replication factor is the number of copies of each block, and it can be configured per file by changing the `dfs.replication` property in `hdfs-site.xml`.

Block abstraction in HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant file system for storing large amounts of data across multiple nodes.
- HDFS exposes a file system namespace and allows user data to be stored in files. Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes.
- HDFS block size is usually 64MB-128MB and unlike other filesystems, a file smaller than the block size does not occupy the complete block size's worth of memory. The block size is kept so large so that less time is made doing disk seeks as compared to the data transfer rate.
- In HDFS the abstraction is made over the blocks of a file rather than a single file which simplifies the storage subsystem. Since the size of the blocks is fixed it is easy to manage and calculate how many blocks can be stored on a single disk.
- HDFS also maintains metadata about the blocks, such as their locations, sizes, checksums, and replication factors. The metadata is stored in the NameNode, which executes file system namespace operations like opening, closing, and renaming files and directories.

Data replication in HDFS

- Data replication is the process of copying data from one HDFS service to another, or to and from other storage systems such as Amazon S3 or Microsoft ADLS.
- Data replication is used for fault tolerance, backup, disaster recovery, and data availability.
- HDFS stores each file as a sequence of blocks, and each block is replicated across multiple DataNodes according to a replication factor.
- The default replication factor is 3, which means that each block is stored on 3 different DataNodes.
- The replication factor can be configured per file, per directory, or globally.
- The NameNode is responsible for managing the replication of blocks, and it uses a replication target choosing algorithm to select the best DataNodes for each block.

- The algorithm considers factors such as rack awareness, network bandwidth, disk space, load balancing, and data locality .
- The client writes data to the first DataNode in the list, and the first DataNode forwards the data to the second DataNode, and so on, until the replication factor is met .
- The NameNode periodically receives block reports and heartbeat messages from the DataNodes, and it detects any missing or corrupted blocks .
- The NameNode initiates the replication of missing or corrupted blocks to maintain the desired replication factor .
- The NameNode also balances the distribution of blocks across the cluster to avoid hotspots and ensure even utilization of resources .

How does HDFS store

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant file system for storing large amounts of data in a distributed manner.
- HDFS splits a file into one or more blocks, each of which has a fixed size (default 128 MB, but configurable) and a unique identifier.
- HDFS stores these blocks on different DataNodes, which are the worker nodes in the Hadoop cluster. Each DataNode can store multiple blocks from different files.
- HDFS also maintains a file system namespace, which is a hierarchical structure of directories and files. The namespace is managed by the NameNode, which is the master node in the Hadoop cluster.
- The NameNode keeps track of the metadata of the files and blocks, such as the file name, size, permissions, locations of the blocks, etc. The NameNode does not store the actual data of the blocks, only the metadata.
- The NameNode communicates with the DataNodes to perform file system operations, such as creating, deleting, reading, and writing files. The NameNode also monitors the health and status of the DataNodes, and balances the load and replication of the blocks across the cluster.
- HDFS provides high availability, reliability, and scalability by replicating each block on multiple DataNodes (default 3, but configurable). This ensures that the data is not lost or corrupted due to node failures, network issues, or disk errors.
- HDFS also supports fault tolerance and recovery mechanisms, such as heartbeat messages, block reports, checksums, snapshots, and safemode. These features help to detect and handle failures, and restore the system to a consistent state.

Read Operations in HDFS

- HDFS is a distributed file system that stores large amounts of data across multiple nodes in a cluster.
- To read a file from HDFS, a client needs to interact with the NameN-

ode, which stores the metadata about the file, such as its location, size, replication factor, and block IDs.

- The client requests the NameNode for the block locations of the file, and the NameNode returns a list of DataNodes that have the blocks of the file.
- The client then contacts one of the DataNodes for each block and reads the data from it. The client can read the data in parallel from multiple DataNodes to improve the performance.
- The client can also perform checksum verification to ensure the data integrity. The checksums are stored in separate files in HDFS, and the client can compare them with the data blocks it reads.
- The client can also handle failures and errors during the read operation, such as network issues, DataNode failures, or corrupted blocks. The client can retry reading from another DataNode or report the problem to the NameNode.

Write operations in HDFS

- HDFS stands for Hadoop Distributed File System, which is a distributed and scalable file system that stores large amounts of data across multiple nodes in a cluster.
- To write data in HDFS, the client follows these steps:
 - The client contacts the NameNode and requests permission to create a new file in a given directory. The NameNode checks if the file already exists and if the client has the write access to the directory. If yes, the NameNode grants the permission and returns a list of DataNodes that can store the file blocks.
 - The client splits the data into fixed-size blocks (default size is 128 MB) and writes them to a local buffer. The client also computes the checksum for each block and appends it to the block.
 - The client creates a pipeline of DataNodes to write the first block. The pipeline size is determined by the replication factor of the file (default is 3). The client sends the block and its checksum to the first DataNode in the pipeline.
 - The first DataNode stores the block and its checksum in its local disk and forwards them to the second DataNode in the pipeline. The second DataNode does the same and forwards them to the third DataNode. The process continues until the last DataNode in the pipeline receives the block and its checksum.
 - The last DataNode sends an acknowledgement (ACK) to the previous DataNode, which sends an ACK to its previous DataNode, and so on until the first DataNode sends an ACK to the client. The client then verifies that the block has been successfully written to

all the DataNodes in the pipeline.

- The client repeats the steps 3 to 5 for the remaining blocks of the file. The client can choose different DataNodes for each block to achieve load balancing and fault tolerance.
- The client notifies the NameNode that the file write operation is complete. The NameNode updates its metadata and makes the file available for read.
- The advantages of write operations in HDFS are:
 - High throughput: The data is written in parallel to multiple DataNodes, which increases the write speed and reduces the network congestion.
 - High reliability: The data is replicated to multiple DataNodes, which ensures the data availability and durability in case of node failures.
 - High scalability: The data is distributed across multiple nodes, which allows the system to handle large volumes of data and to add more nodes as needed.
- The disadvantages of write operations in HDFS are:
 - High latency: The data is written to multiple DataNodes, which involves multiple network hops and disk I/O operations, which increase the write latency.
 - Write-once-read-many: The data is immutable once written to HDFS, which means the client cannot modify or append to the existing file. The client has to create a new file or overwrite the existing file to update the data.
- No random access: The data is stored in fixed-size blocks, which means the client cannot access a specific byte or record within a block. The client has to read the entire block to access the data.

Java interfaces to HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for large-scale data processing applications.
- HDFS provides several interfaces for accessing and manipulating data stored in its filesystem, such as command-line interface, web interface, and Java interface.
- The Java interface is the most commonly used interface for Hadoop filesystem interactions, as it exposes the Hadoop FileSystem class, which is the abstract base class for all Hadoop filesystem implementations.
- The FileSystem class provides methods for creating, opening, reading, writing, deleting, renaming, and listing files and directories in a Hadoop filesystem.
- A file in a Hadoop filesystem is represented by a Hadoop Path object, which encapsulates the URI of the file. A Path object can be created from

a string, such as `Path path = new Path("/path/to/file.ext");`

- To access a Hadoop filesystem, a Java application needs to create a `FileSystem` object, which can be obtained from a `Configuration` object, such as `FileSystem fileSystem = FileSystem.get(conf);`
- The `Configuration` object contains the settings and properties for the Hadoop cluster, such as the default filesystem URI, the replication factor, the block size, etc. The `Configuration` object can be created from a default or a custom configuration file, such as `Configuration conf = new Configuration();` or `Configuration conf = new Configuration(new Path("/path/to/config.xml"));`
- To read data from a file in a Hadoop filesystem, a Java application needs to create an `FSDDataInputStream` object, which is a subclass of `java.io.InputStream`, such as `FSDDataInputStream in = fileSystem.open(path);`
- The `FSDDataInputStream` object supports random access to the file, as well as various methods for reading bytes, primitives, and objects from the stream, such as `in.read()`, `in.readInt()`, `in.readUTF()`, etc.
- To write data to a file in a Hadoop filesystem, a Java application needs to create an `FSDDataOutputStream` object, which is a subclass of `java.io.OutputStream`, such as `FSDDataOutputStream out = fileSystem.create(path);`
- The `FSDDataOutputStream` object supports various methods for writing bytes, primitives, and objects to the stream, such as `out.write()`, `out.writeInt()`, `out.writeUTF()`, etc.
- To query the filesystem, a Java application can use the `FileSystem` class methods, such as `fileSystem.exists(path)`, `fileSystem.getFileStatus(path)`, `fileSystem.listStatus(path)`, etc.
- To close the filesystem, a Java application needs to call the `fileSystem.close()` method, which releases the resources associated with the `FileSystem` object.

Command Line Interface to HDFS

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for big data processing.
- Command Line Interface (CLI) is one of the simplest ways to interact with HDFS. CLI has support for filesystem operations like reading, writing, creating, moving, deleting, and listing files and directories.
- To use CLI, we need to have Hadoop installed and configured on our system. We can run the following command to check the Hadoop version and the HDFS URI:

```
hadoop version
```

```
hdfs getconf -confKey fs.defaultFS
```

- The HDFS commands are prefixed with `hdfs dfs` or `hadoop fs`. For example, to list the files and directories in the root directory of HDFS, we

can run:

```
hdfs dfs -ls /
```

- We can also use the `-help` option to get detailed help on every command. For example, to get help on the `-ls` command, we can run:

```
hdfs dfs -help ls
```

- Some of the common HDFS commands are:
 - `hdfs dfs -mkdir`: to create a directory in HDFS.
 - `hdfs dfs -put`: to copy a file from the local file system to HDFS.
 - `hdfs dfs -get`: to copy a file from HDFS to the local file system.
 - `hdfs dfs -cat`: to display the contents of a file in HDFS.
 - `hdfs dfs -rm`: to delete a file or a directory in HDFS.
 - `hdfs dfs -mv`: to move or rename a file or a directory in HDFS.
 - `hdfs dfs -cp`: to copy a file or a directory within HDFS.
 - `hdfs dfs -du`: to display the disk usage of a file or a directory in HDFS.
 - `hdfs dfs -df`: to display the available space in HDFS.
 - `hdfs dfs -chmod`: to change the permissions of a file or a directory in HDFS.
 - `hdfs dfs -chown`: to change the owner and group of a file or a directory in HDFS.
- For more information on the HDFS commands, please refer to the official documentation.

Hadoop file system interfaces

- Hadoop file system interfaces are the Java abstract classes and interfaces that define the client API to interact with various file systems in Hadoop.
- The main interface is `org.apache.hadoop.fs.FileSystem`, which represents a generic file system that can be local, distributed, or cloud-based.
- `FileSystem` provides methods to create, delete, rename, read, write, and list files and directories, as well as to get file status and configuration information.
- `FileSystem` also supports the concept of URI schemes, which allow users to specify the file system implementation and the location of the data using a uniform syntax.
- For example, `hdfs://namenode:port/path` is a URI scheme for HDFS, while `s3a://bucket/path` is a URI scheme for Amazon S3.
- Hadoop provides several concrete implementations of `FileSystem`, such as `HDFS`, `LocalFileSystem`, `RawLocalFileSystem`, `FTPFileSystem`, `S3AFileSystem`, `AzureBlobFileSystem`, and so on.
- Each implementation has its own configuration properties, performance characteristics, and limitations.

- Users can also extend `FileSystem` or its subclasses to create custom file system implementations for their specific needs.

Data flow in HDFS

- HDFS is a distributed file system that stores large files across multiple nodes in a cluster.
- HDFS follows a master-slave architecture, where a single `NameNode` manages the metadata of the file system, and multiple `DataNodes` store the actual data blocks of files.
- A file in HDFS is split into fixed-size blocks (typically 128 MB or 256 MB) and replicated across multiple `DataNodes` for fault tolerance.
- The default replication factor is 3, which means each block has 3 copies on different `DataNodes`.
- The `NameNode` maintains the mapping of files to blocks and blocks to `DataNodes`, as well as the replication factor and the health status of `DataNodes`.
- The `NameNode` does not store the data blocks itself, nor does it directly communicate with clients who want to read or write files.
- The clients interact with the `NameNode` to get the metadata information, such as the location of blocks, the size of files, the permissions of files, etc.
- The clients then communicate with the `DataNodes` to read or write the data blocks of files.
- The data flow in HDFS can be summarized as follows:
 - **Write operation:**
 - * The client requests the `NameNode` to create a new file in the file system namespace.
 - * The `NameNode` checks if the file already exists, and if the client has the permission to write the file.
 - * If the file does not exist and the client has the permission, the `NameNode` grants a lease to the client to create the file.
 - * The client requests the `NameNode` for a list of `DataNodes` that can host the replicas of the first block of the file.
 - * The `NameNode` returns a list of `DataNodes`, sorted by their network distance from the client.
 - * The client writes the data to the first `DataNode` in the list, which acts as the primary `DataNode` for the block.
 - * The primary `DataNode` forwards the data to the next `DataNode` in the list, which in turn forwards it to the next `DataNode`, and so on, forming a pipeline of `DataNodes`.

- * The DataNodes send acknowledgments to the previous DataNodes in the pipeline, and the primary DataNode sends the final acknowledgment to the client.
 - * The client requests the NameNode for a new list of DataNodes for the next block of the file, and repeats the process until all the blocks of the file are written.
 - * The client tells the NameNode that the file is complete, and the NameNode commits the file to the file system namespace.
- **Read operation:**
- * The client requests the NameNode for the list of DataNodes that host the replicas of the blocks of the file.
 - * The NameNode returns the list of DataNodes for each block, sorted by their network distance from the client.
 - * The client reads the data from the closest DataNode for each block, and if the read fails, it tries the next DataNode in the list, and so on, until the read is successful or the list is exhausted.
 - * The client verifies the checksum of the data to detect any corruption or errors.
 - * The client reads all the blocks of the file in sequence, and closes the file when done.

Data Ingest with Flume and Sqoop in HDFS

- Data ingestion is the process of collecting, transferring, and loading data from various sources into a data store, such as HDFS, for analysis and processing.
- Data ingestion can be done in different ways, depending on the type, source, and volume of data, as well as the frequency and speed of data transfer.
- Flume and Sqoop are two popular tools in the Hadoop ecosystem that are used for data ingestion from different sources into HDFS.
- Flume is a distributed, reliable, and scalable service that can collect and transport large amounts of streaming data, such as log files, events, and messages, from various sources to HDFS, HBase, or other destinations.
- Sqoop is a command-line tool that can perform bulk import and export of data between relational databases, such as MySQL, Oracle, or SQL Server, and HDFS, Hive, or HBase.

Some key differences between Flume and Sqoop are:

- Flume is designed for streaming data, while Sqoop is designed for structured data.
- Flume can handle multiple sources and destinations, while Sqoop can only handle one source and one destination at a time.
- Flume can perform data filtering, transformation, and aggregation, while Sqoop can only perform data conversion and compression.
- Flume can handle data ingestion in real-time or near real-time, while

Sqoop can only handle batch data ingestion.

Hadoop archives in HDFS

- Hadoop archives (HAR) are a way of compressing large numbers of small files in HDFS to reduce the metadata overhead and improve the performance of the file system.
- HAR files are similar to ZIP or TAR files, but they are not compressed. They are created by using the `hadoop archive` command, which takes a list of files or directories as input and produces a single HAR file as output.
- HAR files can be accessed transparently by using the `har://` scheme in the file path. For example, `har:///user/hadoop/test.har/file1.txt` refers to the file `file1.txt` inside the HAR file `test.har` in the HDFS directory `/user/hadoop`.
- HAR files can be used to store rarely accessed files or historical data that do not need to be modified. They can also be used as input or output for MapReduce jobs, as long as the job does not require splitting the input files.
- HAR files have some limitations and trade-offs. They cannot be modified or appended once created. They also introduce some overhead in reading and writing, as the HAR file has to be scanned to locate the desired file. They also do not support compression or encryption.

Hadoop I/O

- Hadoop I/O is the set of data types and I/O classes that are common to Hadoop MapReduce and HDFS.
- Hadoop I/O supports serialization and compression of data, as well as file-based and network-based data access.
- Serialization is the process of converting an object into a sequence of bytes that can be stored or transmitted. Hadoop I/O provides several serialization frameworks, such as Writable, Avro, and Protocol Buffers, that are optimized for performance and interoperability.
- Compression is the process of reducing the size of data by applying a compression algorithm. Hadoop I/O supports various compression codecs, such as Gzip, Snappy, and Bzip2, that can be applied to different types of data, such as text, binary, or splittable.
- File-based data access refers to reading and writing data from files stored on HDFS or other file systems. Hadoop I/O provides several file formats, such as SequenceFile, MapFile, and Parquet, that are suitable for different use cases, such as storing key-value pairs, sorted data, or columnar data.
- Network-based data access refers to reading and writing data from sockets or RPC services. Hadoop I/O provides several network protocols, such as Hadoop RPC, HTTP, and Thrift, that are used for communication between Hadoop components, such as NameNode, DataNode, JobTracker,

and TaskTracker.

Compression in Hadoop io

- Compression is a technique to reduce the size of data files by applying some algorithms that encode the data more efficiently.
- Compression can save disk space, network bandwidth, and processing time in Hadoop, especially when dealing with large volumes of data.
- Hadoop supports various compression codecs, such as DEFLATE, gzip, bzip2, LZO, LZ4, and Snappy. Each codec has different characteristics in terms of compression ratio, speed, and splittability.
- Splittability means that a compressed file can be split into smaller chunks and processed in parallel by multiple map tasks. This is important for Hadoop's scalability and performance.
- Only bzip2 is splittable among the standard codecs, but some custom codecs like LZO can be made splittable with additional indexing .
- Hadoop provides a CodecFactory class that can detect the compression format of an input file based on its extension and return the appropriate CompressionCodec object .
- Compression can be applied at different stages of the data flow in Hadoop, such as input, output, intermediate, and shuffle. Each stage has its own benefits and trade-offs, and the choice of compression codec and level depends on the use case and the data characteristics .

Serialization in Hadoop IO

- Serialization is the process of converting an object into a stream of bytes that can be stored, transmitted, or reconstructed later.
- Hadoop IO provides a framework for defining and implementing serialization formats for different types of data, such as text, binary, or complex objects.
- Hadoop IO also provides some built-in serialization formats, such as Writable, Avro, and SequenceFile, that are optimized for performance, compression, and interoperability.
- Serialization in Hadoop IO is important for several reasons:
 - It enables data to be stored in a compact and efficient way, reducing disk space and network bandwidth usage.
 - It allows data to be processed by different applications or languages, as long as they support the same serialization format.
 - It facilitates the distribution and parallelization of data across multiple nodes in a cluster, enabling scalable and fault-tolerant computation.

Avro and file based data structures in Hadoop io

- Avro is a data serialization system that allows data to be stored and exchanged in a compact and efficient binary format.

- Avro supports schema evolution, which means that the schema of the data can change over time without breaking compatibility with existing data or applications.
- Avro also provides a rich set of data types, such as records, enums, arrays, maps, unions, and logical types, that can represent complex and nested data structures.
- Avro data can be stored in files or transmitted over the network using Avro's RPC (remote procedure call) framework.
- Avro files have a self-describing format that includes the schema of the data and a sync marker that allows random access and splitting of the file.
- Avro files can be compressed using various codecs, such as Snappy, Deflate, or Bzip2, to reduce the size of the data and improve the performance of reading and writing.
- Avro files can be processed by Hadoop MapReduce, Spark, Hive, Pig, and other frameworks that support the Hadoop io API.
- Avro files can also be integrated with other data formats, such as Parquet, ORC, JSON, and CSV, using tools like Avro Tools, Sqoop, Flume, and Kafka Connect.

Hadoop Environment

- Hadoop is an open source software framework that allows for the distributed storage and processing of large data sets across clusters of computers using simple programming models .
- Hadoop is based on the MapReduce programming model, which enables parallel processing of large data sets by dividing them into smaller chunks and assigning them to different nodes in the cluster.
- Hadoop has two main components: Hadoop Distributed File System (HDFS) and Hadoop MapReduce.
 - HDFS is a distributed file system that provides high availability, fault tolerance, scalability, and data locality for large data sets.
 - Hadoop MapReduce is a software framework that supports the execution of user-defined Map and Reduce functions on the data stored in HDFS.
- Hadoop also has a rich ecosystem of tools and applications that extend its functionality and provide additional features, such as data ingestion, data integration, data analysis, data visualization, machine learning, etc .
 - Some of the popular tools and applications in the Hadoop ecosystem are: Apache Hive, Apache Pig, Apache Spark, Apache HBase, Apache Sqoop, Apache Flume, Apache Kafka, Apache Oozie, Apache ZooKeeper, etc .
- Hadoop requires the Java Runtime Environment (JRE) 1.6 or higher to run. It also requires that Secure Shell (SSH) be set up between nodes in the cluster for the standard startup and shutdown scripts.
- Hadoop can run on various platforms, such as Linux, Windows, Mac OS

X, etc. It can also run on cloud platforms, such as Google Cloud, Amazon Web Services, Microsoft Azure, etc.

Setting up a Hadoop cluster in Hadoop Environment

A Hadoop cluster is a collection of machines that run the Hadoop software and store and process large amounts of data using the Hadoop Distributed File System (HDFS) and the MapReduce framework. To set up a Hadoop cluster in a Hadoop environment, the following steps are required:

- Install Java on all the machines in the cluster, as Hadoop is written in Java and requires it to run.
- Download the latest stable version of Hadoop from the official website and extract it to a desired location on all the machines in the cluster.
- Configure the Hadoop environment variables, such as `HADOOP_HOME`, `HADOOP_CONF_DIR`, `JAVA_HOME`, etc., on all the machines in the cluster.
- Configure the Hadoop configuration files, such as `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, `yarn-site.xml`, etc., on all the machines in the cluster. These files specify the parameters for the Hadoop components, such as the name node, the data nodes, the resource manager, the node managers, etc.
- Set up passwordless SSH access between the machines in the cluster, as Hadoop uses SSH to communicate and execute commands on the remote machines.
- Format the HDFS file system on the name node machine, which is the master node of the cluster that manages the metadata of the HDFS files and directories.
- Start the Hadoop daemons on all the machines in the cluster, such as the name node, the data nodes, the resource manager, the node managers, etc.
- Verify the status of the Hadoop cluster by using the web interface or the command line tools, such as `hdfs dfsadmin -report`, `yarn node -list`, etc.

These are the basic steps to set up a Hadoop cluster in a Hadoop environment. Depending on the specific requirements and the size of the cluster, some additional steps or configurations may be needed. For more details, please refer to the official Hadoop documentation.

Cluster Specification in Hadoop Environment

- A Hadoop cluster is a special type of computational cluster designed specifically for storing and analyzing huge amounts of unstructured data in a distributed computing environment .
- A Hadoop cluster consists of a collection of computers, known as nodes, that are networked together to perform parallel computations on big data sets.

- A Hadoop cluster is often referred to as a shared-nothing system because the only thing that is shared between the nodes is the network itself.
- A Hadoop cluster typically has two types of nodes: master nodes and worker nodes.
- Master nodes are responsible for coordinating the distribution of data and tasks among the worker nodes, as well as maintaining the overall health and status of the cluster.
- Worker nodes are responsible for storing and processing the data assigned to them by the master nodes, as well as reporting their progress and status to the master nodes.
- The most common master nodes in a Hadoop cluster are the NameNode and the JobTracker, while the most common worker nodes are the DataNodes and the TaskTrackers.
- The NameNode is the master node that manages the distributed file system (HDFS) and maintains the metadata of the files and directories stored in the cluster.
- The DataNodes are the worker nodes that store the actual data blocks of the files in the cluster and serve read and write requests from the clients.
- The JobTracker is the master node that manages the execution of MapReduce jobs and assigns tasks to the TaskTrackers.
- The TaskTrackers are the worker nodes that run the tasks of the MapReduce jobs and report their status to the JobTracker.
- To configure the Hadoop cluster, you will need to configure the environment in which the Hadoop daemons execute as well as the configuration parameters for the Hadoop daemons .
- The environment configuration includes setting up the SSH access, the JAVA_HOME variable, and the HADOOP_CONF_DIR variable for the Hadoop daemons.
- The configuration parameters include setting up the core-site.xml, hdfs-site.xml, mapred-site.xml, and yarn-site.xml files for the Hadoop daemons.
- The core-site.xml file contains the basic settings for the Hadoop cluster, such as the default file system URI, the default block size, and the default replication factor.
- The hdfs-site.xml file contains the settings for the HDFS, such as the location of the NameNode and DataNode directories, the checkpoint interval, and the block scanner interval.
- The mapred-site.xml file contains the settings for the MapReduce framework, such as the location of the JobTracker and TaskTracker directories, the number of map and reduce slots, and the memory limits for the tasks.
- The yarn-site.xml file contains the settings for the YARN framework, which is the resource management layer of Hadoop, such as the location of the ResourceManager and NodeManager directories, the resource allocation policies, and the scheduler options.

Cluster setup and installation in Hadoop Environment

- A Hadoop cluster is a collection of machines that run the Hadoop software and store the data in a distributed manner.
- A Hadoop cluster can be classified into two types: single-node cluster and multi-node cluster.
- A single-node cluster is a cluster that consists of only one machine, which acts as both the master and the worker node. It is useful for testing and development purposes, but not for production use.
- A multi-node cluster is a cluster that consists of more than one machine, which are divided into master nodes and worker nodes. The master nodes are responsible for managing the cluster, such as coordinating the tasks, monitoring the status, and maintaining the metadata. The worker nodes are responsible for storing the data and executing the tasks.
- To set up and install a Hadoop cluster, the following steps are required:
 1. Install Java on all the machines, as Hadoop is written in Java and requires Java Runtime Environment (JRE) to run.
 2. Download and extract the Hadoop binary package from the official website or a mirror site on all the machines.
 3. Configure the Hadoop environment variables, such as HADOOP_HOME, HADOOP_CONF_DIR, JAVA_HOME, etc., on all the machines.
 4. Configure the Hadoop configuration files, such as core-site.xml, hdfs-site.xml, mapred-site.xml, yarn-site.xml, etc., on all the machines. The configuration files specify the properties and parameters of the Hadoop components, such as the cluster name, the namenode address, the replication factor, the memory allocation, the scheduler, etc.
 5. Set up the SSH connection between the machines, so that the master node can communicate with the worker nodes without password authentication.
 6. Format the Hadoop Distributed File System (HDFS) on the master node, which initializes the namenode and creates the directory structure for the data blocks.
 7. Start the Hadoop daemons on all the machines, such as the namenode, the datanode, the resource manager, the node manager, etc., using the start-all.sh or start-dfs.sh and start-yarn.sh scripts.
 8. Verify the status of the Hadoop cluster, such as the number of live nodes, the available disk space, the running applications, etc., using the web interface or the command-line tools.
- To stop the Hadoop cluster, use the stop-all.sh or stop-dfs.sh and stop-yarn.sh scripts on the master node, which will stop the Hadoop daemons on all the machines.

Hadoop configuration in Hadoop Environment

Hadoop configuration is the process of setting the parameters and properties of

the Hadoop components and services to optimize their performance and functionality in a Hadoop cluster. Hadoop configuration can be done at different levels, such as:

- **Hadoop environment:** This refers to the environment variables and scripts that control the behavior and options of the Hadoop daemons, such as NameNode, DataNode, ResourceManager, NodeManager, etc. These variables and scripts are usually located in the `etc/hadoop` directory of the Hadoop installation and can be customized according to the cluster specifications. For example, the `etc/hadoop/hadoop-env.sh` script can be used to set the `JAVA_HOME`, `HADOOP_CONF_DIR`, `HADOOP_LOG_DIR`, `HADOOP_HEAPSIZE`, and other variables for the Hadoop daemons.
- **Hadoop daemons:** These are the processes that run on the nodes of the Hadoop cluster and provide the core functionality of Hadoop, such as distributed storage (HDFS) and distributed computation (YARN). Each daemon has its own configuration file that defines the parameters and properties for its operation, such as the port number, the directory paths, the memory allocation, the replication factor, the scheduler policy, etc. These configuration files are usually in XML format and are also located in the `etc/hadoop` directory of the Hadoop installation. For example, the `etc/hadoop/core-site.xml` file can be used to set the default file system URI, the `etc/hadoop/hdfs-site.xml` file can be used to set the block size and the number of replicas for HDFS, the `etc/hadoop/yarn-site.xml` file can be used to set the resource manager address and the node manager resource allocation for YARN, etc .
- **Hadoop applications:** These are the programs that run on top of the Hadoop framework and use the Hadoop services and APIs to perform various tasks, such as data processing, analysis, machine learning, etc. Each application can have its own configuration file that specifies the parameters and properties for its execution, such as the input and output paths, the mapper and reducer classes, the combiner and partitioner functions, the compression and serialization formats, etc. These configuration files can also be in XML format and can be passed as arguments to the application jar file or the Hadoop command line interface. For example, the `etc/hadoop/mapred-site.xml` file can be used to set the mapreduce framework name and the job history server address for MapReduce, the `etc/hadoop/oozie-site.xml` file can be used to set the oozie base URL and the workflow engine class for Oozie, etc .

Hadoop configuration is an important and essential step in setting up and managing a Hadoop cluster, as it can affect the performance, reliability, security, and scalability of the Hadoop components and services. Therefore, it is recommended to follow the best practices and guidelines for Hadoop configuration, such as:

- Use consistent and descriptive names for the configuration parameters and properties.

- Use appropriate values and units for the configuration parameters and properties, and avoid using default or hard-coded values.
- Use comments and documentation to explain the purpose and effect of the configuration parameters and properties.
- Use environment variables and placeholders to avoid repeating the same values in multiple configuration files.
- Use configuration management tools and frameworks, such as Ansible, Puppet, Chef, etc., to automate and simplify the configuration process and ensure consistency and accuracy across the cluster nodes.
- Test and validate the configuration changes before applying them to the production cluster, and monitor and troubleshoot the configuration issues if they arise.

Security in Hadoop in Hadoop Environment

- Hadoop is a distributed system that stores and processes large amounts of data across multiple nodes in a cluster.
- Hadoop security is the process of protecting the data and the cluster from unauthorized access, modification, or deletion by malicious users or applications.
- Hadoop security can be divided into four main aspects: authentication, authorization, encryption, and auditing.
- Authentication is the process of verifying the identity of a user or an application that tries to access the cluster or the data. Hadoop supports various authentication mechanisms, such as Kerberos, LDAP, or OAuth.
- Authorization is the process of granting or denying permissions to a user or an application to perform certain actions on the cluster or the data. Hadoop supports various authorization mechanisms, such as HDFS permissions, ACLs, Ranger, or Sentry.
- Encryption is the process of transforming the data into an unreadable form to prevent unauthorized access or disclosure. Hadoop supports various encryption mechanisms, such as HDFS encryption, KMS, or TDE.
- Auditing is the process of recording and monitoring the activities and events that occur on the cluster or the data. Hadoop supports various auditing mechanisms, such as HDFS audit logs, Ranger audit, or Sentry audit.

Administering Hadoop in Hadoop Environment

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers.
- Hadoop consists of several components, such as Hadoop Distributed File System (HDFS), MapReduce, YARN, HBase, Hive, Pig, Spark, etc.
- Administering Hadoop involves installing, configuring, monitoring, troubleshooting, and securing the Hadoop cluster and its components.

- Some of the tasks involved in administering Hadoop are:
 - Setting up the Hadoop cluster, which includes choosing the hardware, network, and software specifications, and installing the Hadoop distribution and its components on the nodes.
 - Configuring the Hadoop cluster, which includes setting the parameters for HDFS, MapReduce, YARN, and other components, and creating and managing the users, groups, and permissions.
 - Monitoring the Hadoop cluster, which includes collecting and analyzing the metrics, logs, and alerts from the Hadoop components, and using tools such as Ambari, Ganglia, Nagios, etc. to visualize and manage the cluster health and performance.
 - Troubleshooting the Hadoop cluster, which includes identifying and resolving the issues related to the Hadoop components, such as data loss, corruption, replication, availability, etc., and using tools such as fsck, balancer, distcp, etc. to repair and optimize the HDFS.
 - Securing the Hadoop cluster, which includes implementing the authentication, authorization, encryption, and auditing mechanisms for the Hadoop components, and using tools such as Kerberos, Ranger, Knox, etc. to protect the data and access to the cluster.

Hello, I am Sydney, your AI assistant. I can help you with any topic you want to learn or discuss. You have asked me to write the content in markdown format on the topic of HDFS monitoring and maintenance in Hadoop environment. Here is what I have written:

HDFS monitoring & maintenance in Hadoop Environment

- HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant file system that stores large amounts of data across multiple nodes in a cluster.
- HDFS monitoring and maintenance are essential tasks for ensuring the reliability, availability, and performance of the HDFS cluster and the data stored in it.
- HDFS monitoring involves collecting and analyzing various metrics and logs from the HDFS components, such as the NameNode, the DataNodes, the JournalNodes, and the clients.
- HDFS maintenance involves performing various operations and tasks on the HDFS cluster, such as adding or removing nodes, balancing the data distribution, repairing corrupted blocks, upgrading the software, and backing up or restoring the data.
- Some of the common tools and techniques for HDFS monitoring and maintenance are:
 - The HDFS web interface, which provides a graphical view of the

cluster status, the file system namespace, the data blocks, and the health of the nodes.

- The HDFS shell commands, which allow the user to interact with the HDFS cluster and perform various operations, such as creating, deleting, copying, or moving files and directories, checking the disk usage, listing the files, and changing the permissions or ownership of the files.
- The HDFS admin commands, which allow the user to perform administrative tasks on the HDFS cluster, such as formatting the NameNode, starting or stopping the services, setting the quotas, refreshing the nodes, or running the balancer or the fsck tool.
- The HDFS JMX interface, which exposes various Java Management Extensions (JMX) beans that provide information about the HDFS components, such as the configuration, the metrics, the statistics, and the status.
- The HDFS audit logs, which record the user actions and the outcomes of the operations performed on the HDFS cluster, such as the file or directory creation, deletion, modification, or access.
- The HDFS metrics system, which collects and reports various performance and usage metrics from the HDFS components, such as the number of files, blocks, bytes, transactions, RPC calls, or errors.
- The HDFS alerts and notifications, which notify the user or the administrator about any critical or abnormal events or conditions in the HDFS cluster, such as the node failure, the disk space shortage, the block corruption, or the service outage.
- The HDFS backup and restore, which allow the user or the administrator to create and restore snapshots or backups of the HDFS data or metadata, in case of any data loss or corruption.

Hadoop benchmarks in Hadoop Environment

- Hadoop benchmarks are tools or programs that measure the performance of Hadoop clusters and applications in various aspects, such as throughput, latency, scalability, reliability, etc.
- Hadoop benchmarks can be classified into two categories: micro-benchmarks and macro-benchmarks.
- Micro-benchmarks test the low-level components of Hadoop, such as HDFS, MapReduce, YARN, etc. They are useful for debugging, tuning, and optimizing the system configuration and parameters. Examples of micro-benchmarks are TestDFSIO, TeraSort, PiEstimator, etc.
- Macro-benchmarks test the end-to-end performance of Hadoop applications and workloads, such as data analytics, machine learning, graph processing, etc. They are useful for evaluating the functionality, usability, and efficiency of Hadoop solutions. Examples of macro-benchmarks are HiBench, BigBench, BigDataBench, etc.
- Hadoop benchmarks can be run in different modes, such as local mode,

pseudo-distributed mode, and fully distributed mode, depending on the number and type of nodes involved in the cluster. Local mode runs the benchmark on a single node, pseudo-distributed mode runs the benchmark on a single node with multiple processes, and fully distributed mode runs the benchmark on a multi-node cluster.

- Hadoop benchmarks can be run using different tools, such as Hadoop command-line interface, Hadoop Streaming, Hadoop Pipes, etc. Hadoop command-line interface is the simplest way to run Hadoop benchmarks, as it only requires specifying the input and output paths, the number of mappers and reducers, and the class name of the benchmark. Hadoop Streaming allows running Hadoop benchmarks written in any programming language, such as Python, Ruby, etc., by using standard input and output streams. Hadoop Pipes allows running Hadoop benchmarks written in C++ by using a C++ API for Hadoop.
- Hadoop benchmarks can be evaluated using different metrics, such as execution time, throughput, speedup, scalability, efficiency, etc. Execution time is the total time taken to run the benchmark, including the data preparation, job submission, job execution, and data cleanup phases. Throughput is the amount of data processed per unit time by the benchmark. Speedup is the ratio of the execution time of the benchmark on a single node to the execution time of the benchmark on a multi-node cluster. Scalability is the ability of the benchmark to handle increasing amounts of data and nodes without degrading the performance. Efficiency is the ratio of the speedup to the number of nodes in the cluster.

Hadoop in the cloud in Hadoop Environment

- Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers.
- Hadoop in the cloud is a way of deploying and managing Hadoop clusters on cloud platforms, such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), etc.
- Hadoop in the cloud offers several benefits, such as:
 - Scalability: Cloud platforms allow users to scale up or down the resources (such as CPU, memory, storage, etc.) according to the data volume and processing needs.
 - Cost-effectiveness: Cloud platforms charge users based on the usage of resources, rather than fixed upfront costs. Users can also leverage the pay-as-you-go model to avoid paying for idle resources.
 - Flexibility: Cloud platforms provide various options for configuring and customizing the Hadoop clusters, such as choosing the operating system, the Hadoop distribution, the cluster size, the security settings, etc.
 - Availability: Cloud platforms ensure high availability and reliability of the Hadoop clusters by providing backup and recovery mechanisms, load balancing, fault tolerance, etc.

- Hadoop in the cloud also poses some challenges, such as:
 - Data transfer: Moving large amounts of data to and from the cloud can incur significant network costs and latency issues. Users need to optimize the data transfer methods and formats, such as compression, encryption, partitioning, etc.
 - Data security: Storing and processing sensitive data on the cloud can raise privacy and compliance concerns. Users need to ensure the data is protected by encryption, authentication, authorization, auditing, etc.
 - Data governance: Managing and monitoring the data quality, lineage, provenance, and usage on the cloud can be complex and challenging. Users need to establish and enforce the data governance policies and standards, such as metadata management, data cataloging, data quality assessment, etc.

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

- Hadoop Ecosystem is a platform or a suite which provides various services to solve the big data problems. It includes Apache projects and various commercial tools and solutions.
- Hadoop Ecosystem consists of four major elements: HDFS, MapReduce, YARN, and Hadoop Common.
- HDFS is the distributed file system that stores the data in a cluster of nodes. It provides high availability, fault tolerance, scalability, and reliability.
- MapReduce is the programming model that processes the data in parallel using key-value pairs. It consists of two phases: map and reduce. Map phase applies a function to each input record and generates intermediate key-value pairs. Reduce phase aggregates the intermediate values associated with the same key and produces the final output.
- YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop. It is responsible for allocating and scheduling the resources (CPU, memory, disk, network) for the applications running on the cluster. It also monitors and manages the workloads.
- Hadoop Common is the set of common utilities and libraries that support the other Hadoop modules. It provides the basic functionality such as configuration, logging, security, and I/O operations.
- NoSQL databases are non-relational databases that store and retrieve data in different ways than the traditional relational databases. They are designed to handle large volumes of unstructured, semi-structured, or structured data with high performance, scalability, and availability.

- NoSQL databases can be classified into four types: key-value, document, column, and graph. Each type has its own advantages and disadvantages depending on the use case and data model.
- MongoDB is a popular document-based NoSQL database that stores data as JSON-like documents. It supports dynamic schema, indexing, aggregation, replication, sharding, and transactions.
- MongoDB provides various operations to create, read, update, and delete (CRUD) documents, as well as query and analyze the data using filters, projections, sorts, limits, skips, and aggregations.
- MongoDB also supports capped collections, which are fixed-size collections that automatically remove the oldest documents when they reach the maximum size. Capped collections are useful for storing log data, time series data, or other data that needs to be accessed in insertion order.
- Spark is a fast and general-purpose framework for large-scale data processing. It provides an in-memory computation engine that can run up to 100 times faster than MapReduce for certain workloads.
- Spark supports various programming languages such as Scala, Python, Java, and R. It also provides a rich set of libraries for SQL, streaming, machine learning, graph processing, and natural language processing.
- Spark consists of four main components: Spark Core, Spark SQL, Spark Streaming, and Spark MLlib. Spark Core is the base layer that provides the basic functionality such as distributed task scheduling, memory management, fault recovery, and data storage. Spark SQL is the module that supports structured and semi-structured data processing using SQL or DataFrames. Spark Streaming is the module that enables real-time data processing using micro-batches or discretized streams. Spark MLlib is the module that offers scalable and easy-to-use machine learning algorithms and utilities.
- Scala is a multi-paradigm programming language that combines the features of object-oriented and functional programming. It runs on the Java Virtual Machine (JVM) and interoperates seamlessly with Java code.
- Scala is designed to be concise, expressive, and scalable. It supports various features such as pattern matching, higher-order functions, lazy evaluation, immutability, traits, case classes, and implicits.
- Scala is also the native language of Spark, which means that it can leverage the full power and performance of the Spark framework. Scala also offers a REPL (Read-Eval-Print Loop) that allows interactive and iterative development and testing of Spark applications.

Hadoop Eco System and YARN

- Hadoop is an open source framework that allows distributed processing of large-scale data using clusters of commodity hardware.
- Hadoop ecosystem refers to the various components and tools that work together to support Hadoop functionality, such as data ingestion, storage, processing, analysis, and visualization.
- YARN (Yet Another Resource Negotiator) is one of the core components of Hadoop ecosystem, introduced in Hadoop 2.0 to overcome the limitations of Hadoop 1.0 MapReduce.
- YARN is responsible for managing and allocating resources across the cluster, as well as scheduling and monitoring the execution of applications.
- YARN architecture consists of two main components: ResourceManager (RM) and ApplicationMaster (AM).
 - ResourceManager is the global authority that arbitrates resources among all the applications in the system. It has two sub-components: Scheduler and ApplicationsManager.
 - ApplicationMaster is the per-application framework that negotiates resources from the ResourceManager and works with the NodeManagers to execute and monitor the tasks.
- YARN enables the Hadoop ecosystem to support a variety of applications and frameworks, such as MapReduce, Spark, Hive, Pig, HBase, etc., by providing a common platform for resource management and isolation.
- YARN also improves the performance, scalability, and flexibility of the Hadoop ecosystem, by allowing multiple applications to run concurrently on the same cluster, and by supporting dynamic resource allocation and containerization.

Hadoop ecosystem components

The Hadoop ecosystem is a collection of software components and tools that enable the processing and analysis of large-scale data sets on a distributed computing platform. The Hadoop ecosystem consists of four main layers: data storage, data processing, data access, and data management.

- Data storage: This layer is responsible for storing the raw data in a distributed and fault-tolerant manner. The core component of this layer is the Hadoop Distributed File System (HDFS), which splits the data into blocks and distributes them across multiple nodes in the cluster. HDFS also provides replication and recovery mechanisms to ensure data availability and reliability. Other components of this layer include HBase, a column-oriented database that runs on top of HDFS, and Kudu, a storage system that supports both analytical and transactional workloads.
- Data processing: This layer is responsible for executing various types of computations on the data stored in HDFS or other sources. The core component of this layer is the Hadoop MapReduce framework, which allows the parallel processing of large data sets using a map and reduce

paradigm. MapReduce divides the data into key-value pairs and applies a user-defined function to each pair. The results are then shuffled and aggregated by another user-defined function. Other components of this layer include YARN, a resource manager that allocates and schedules the tasks across the cluster, and Spark, a fast and general-purpose engine for large-scale data processing that supports batch, streaming, SQL, machine learning, and graph analytics .

- **Data access:** This layer is responsible for providing various interfaces and tools to access and query the data stored and processed in the Hadoop ecosystem. The core component of this layer is Hive, a data warehouse system that allows the analysis of large data sets using a SQL-like language called HiveQL. Hive converts the queries into MapReduce or Spark jobs and executes them on the cluster. Other components of this layer include Pig, a high-level scripting language for data analysis, Sqoop, a tool for transferring data between Hadoop and relational databases, and Flume, a tool for collecting and aggregating streaming data from various sources .
- **Data management:** This layer is responsible for providing various services and utilities to manage and monitor the Hadoop ecosystem. The core component of this layer is Zookeeper, a distributed coordination service that maintains the configuration and status information of the cluster. Zookeeper also provides synchronization, naming, and group services for the other components. Other components of this layer include Oozie, a workflow scheduler that orchestrates the execution of multiple jobs, and Ambari, a web-based tool that provides administration and monitoring capabilities for the Hadoop cluster .

The Hadoop ecosystem is an evolving and expanding platform that incorporates new and emerging technologies to address the challenges and opportunities of big data. The Hadoop ecosystem components work together to provide a scalable, reliable, and flexible solution for data-intensive applications.

Schedulers in Hadoop Ecosystem

- Schedulers are algorithms that are used to schedule tasks in a Hadoop cluster when multiple clients submit requests for data processing.
- Schedulers help in ensuring optimal utilization of resources and access to unused capacity in the cluster.
- Schedulers also help in managing the priority, fairness, and resource allocation of different jobs and queues.
- There are mainly four types of schedulers in Hadoop ecosystem:
 - **FIFO (First In First Out) Scheduler:** This is the default scheduler in Hadoop. It schedules jobs in the order of their submission. It does not consider the size, complexity, or priority of the jobs. It is simple and easy to use, but it can cause starvation and low resource

utilization for large or complex jobs that are submitted later.

- **Capacity Scheduler:** This scheduler divides the cluster resources into multiple queues, each with a predefined capacity and a maximum limit. It allocates resources to jobs based on the queue capacity and the job priority. It also supports hierarchical queues, preemption, and resource sharing. It is suitable for multi-tenant environments where different users or groups have different resource requirements and priorities.
- **Fair Scheduler:** This scheduler assigns resources to jobs such that each job gets a fair share of the cluster resources over time. It also supports multiple queues, each with a minimum and a maximum share of resources. It dynamically adjusts the resource allocation based on the demand and the availability of resources. It also supports preemption, weight, and pool-based scheduling. It is suitable for environments where fairness and flexibility are important.
- **YARN Scheduler:** This scheduler is part of the YARN framework, which is the resource management layer of Hadoop 2. It consists of two components: the Resource Manager and the Node Manager. The Resource Manager is responsible for allocating resources to applications across the cluster. The Node Manager is responsible for managing the resources and executing the tasks on each node. The YARN scheduler supports pluggable policies, such as FIFO, Capacity, and Fair schedulers. It also supports dynamic resource allocation, node labels, and reservation systems. It is suitable for environments where scalability and compatibility are important.

Fair and Capacity in Hadoop Ecosystem

- Hadoop is a batch processing ecosystem that can handle large-scale data analysis using distributed computing.
- Hadoop has a resource management layer called YARN (Yet Another Resource Negotiator) that allocates resources to different applications running on the cluster.
- Hadoop also has a scheduling layer that decides the order and priority of the applications that request resources from YARN.
- There are mainly three types of schedulers in Hadoop: FIFO, Capacity, and Fair.
- FIFO (First In First Out) Scheduler is the simplest and default scheduler that assigns resources to applications based on their submission time. It does not consider the priority, size, or resource requirements of the applications. It can cause resource starvation and low cluster utilization if there are long-running or high-demand applications in the queue.
- Capacity Scheduler is a more advanced scheduler that allows multiple queues with different capacities and priorities to be created. Each queue can have sub-queues and can be assigned a minimum and maximum capacity of resources. The Capacity Scheduler ensures that each queue gets

its guaranteed capacity and can use the free resources from other queues if available. It also supports preemption, which means that it can reclaim resources from low-priority applications if high-priority applications need them .

- Fair Scheduler is another advanced scheduler that aims to provide fair and equal share of resources to all applications over time. It does not require predefined queues or capacities, but dynamically creates pools for each user or group and balances the resources among them. It also supports preemption, weight, and priority to adjust the resource allocation according to the needs of the applications .
- Both Capacity and Fair Schedulers are suitable for large and heterogeneous Hadoop clusters that run multiple types of applications with different resource demands and priorities. They can improve the cluster utilization, throughput, and performance of the applications. However, they have different trade-offs and advantages. The Capacity Scheduler is more suitable for multi-tenant environments that need strict isolation and resource guarantees for each queue. The Fair Scheduler is more suitable for shared environments that need flexible and dynamic resource allocation for each application.

Hadoop 2.0 New Features - NameNode high availability

- NameNode is the master node in HDFS that maintains the filesystem tree and the metadata of all the files and directories.
- In Hadoop 1.x, NameNode was a single point of failure (SPOF) in an HDFS cluster. If the NameNode failed or became unavailable, the entire cluster would be inaccessible until the NameNode was restored or replaced.
- Hadoop 2.0 overcomes this SPOF problem by providing support for multiple NameNodes. It introduces Hadoop 2.0 High Availability feature that brings in an extra NameNode (Passive Standby NameNode) to the Hadoop Architecture which is configured for automatic failover .
- The Active NameNode is the one that serves the client requests and performs the normal NameNode operations. The Standby NameNode is the one that keeps its state synchronized with the Active NameNode and takes over the role of Active NameNode in case of a failure.
- The synchronization between the Active and Standby NameNodes is achieved by using a shared storage directory called JournalNode, where both NameNodes write their edit logs. The Standby NameNode reads the edit logs from the JournalNode and applies them to its own namespace.
- The failover process is triggered by a component called Failover Controller, which monitors the health of both NameNodes and initiates a failover when it detects that the Active NameNode is down or unresponsive. The Failover Controller can be either a separate daemon or a part of the DataNode daemon.
- The Hadoop 2.0 High Availability feature enables the HDFS cluster to run with redundant NameNodes in an Active/Passive configuration with

a hot standby. This eliminates the NameNode as a potential single point of failure and improves the availability and reliability of the HDFS cluster.

HDFS Federation in Hadoop Ecosystem

- HDFS Federation is a feature introduced in Hadoop 2 that enhances the existing HDFS architecture by adding multiple NameNode/namespaces support to HDFS.
- A NameNode is a master node that manages the metadata of the files and directories stored in HDFS. A namespace is a logical grouping of files and directories under a common root directory.
- In the previous HDFS architecture, there was only one NameNode/namespace for the entire cluster, which posed some limitations such as:
 - Single point of failure: If the NameNode fails, the entire cluster becomes inaccessible.
 - Scalability bottleneck: The NameNode has to handle all the metadata operations and keep them in memory, which limits the number of files and blocks that can be stored in HDFS.
 - Performance bottleneck: The NameNode has to communicate with all the DataNodes (slave nodes that store the actual data blocks) in the cluster, which increases the network traffic and latency.
- HDFS Federation overcomes these limitations by allowing the use of more than one NameNode/namespace in the same cluster. Each NameNode manages a subset of the files and directories in HDFS, called a namespace volume. Each namespace volume has its own block pool, which is a collection of blocks that belong to the files in that namespace. The DataNodes store the blocks from multiple block pools and report them to the respective NameNodes .
- HDFS Federation provides the following benefits:
 - Isolation: Each NameNode operates independently of each other, which reduces the impact of failures and improves the availability of the cluster.
 - Scalability: The cluster can store more files and blocks by adding more NameNodes/namespaces, which also distributes the metadata load and memory usage among them.
 - Performance: The NameNodes can handle the metadata operations in parallel, which reduces the network congestion and latency. The DataNodes can also serve the data requests from multiple NameNodes, which increases the throughput .
- HDFS Federation architecture also opens up the possibility for future innovations, such as supporting different file system implementations, policies, and features for different namespaces.

MRv2 in Hadoop ecosystem

- MRv2 stands for **MapReduce version 2**, which is an application framework that runs within YARN (Yet Another Resource Negotiator) on Hadoop 2 .
- YARN is a **cluster resource management** layer that separates the resource allocation and scheduling tasks from the data processing logic of MapReduce .
- MRv2 provides **backward compatibility** with the org.apache.hadoop.mapred APIs of Hadoop 1, which means that the compiled binaries of MapReduce applications can run on MRv2 without any modification.
- MRv2 also provides **improved performance** and **scalability** of MapReduce applications, as well as **fault tolerance** and **security** features.
- MRv2 enables **multi-tenancy** support and **diverse workloads** on Hadoop 2, as YARN can allocate resources to other application engines besides MapReduce, such as Spark, Storm, Tez, etc. .

YARN

- Yarn is a long continuous length of interlocked fibres, used in sewing, crocheting, knitting, weaving, embroidery, ropemaking, and the production of textiles.
- Thread is a type of yarn intended for sewing by hand or machine.
- Yarn is selected for different textiles based on the characteristics of the yarn fibres, such as warmth (wool), light weight (cotton or rayon), durability (nylon is added to sock yarn, for example), or softness (cashmere, alpaca).
- Yarn is composed of twisted strands of fiber, which are known as plies when grouped together.
- The thickness of the yarn is determined by the number of plies, the weight of the yarn, and the gauge of the knitting or crochet pattern.
- Yarn can be dyed in various colors, either before or after being spun into yarn.
- Yarn can also be blended with different types of fibres to create different effects, such as wool and silk, cotton and acrylic, or bamboo and nylon.
- Yarn can be classified into natural yarns (made from animal, plant, or mineral sources) and synthetic yarns (made from man-made materials).
- Some examples of natural yarns are wool, cotton, linen, silk, alpaca, cashmere, and hemp.
- Some examples of synthetic yarns are acrylic, polyester, nylon, rayon, and spandex.
- Yarn can also be categorized by the way it is spun, such as worsted, woolen, or novelty yarns.
- Worsted yarns are spun from long and smooth fibres that are combed and aligned, resulting in a strong and smooth yarn that is suitable for garments and accessories.
- Woolen yarns are spun from short and fuzzy fibres that are carded and not aligned, resulting in a soft and fluffy yarn that is suitable for warm

and cozy items.

- Novelty yarns are spun from fibres that have been manipulated to create special effects, such as boucle, eyelash, chenille, or ribbon yarns.
- Yarn can also be used as a package manager that doubles down as project manager for software development.
- Yarn can help developers to work on one-shot projects or large monorepos, as a hobbyist or an enterprise user.
- Yarn can also help developers to split their project into sub-components kept within a single repository, called workspaces.
- Yarn can also help developers to ensure the stability, security, and performance of their code, by using checksums, lockfiles, and zero-installs.
- Yarn can also help developers to customize their workflows, by using plugins, scripts, and hooks.
- Yarn can also help developers to access a large ecosystem of packages, by using the npm registry or other sources.

Running MRv1 in YARN

- MRv1 is the first version of MapReduce, a programming model for processing large-scale data sets in parallel.
- YARN is the second version of MapReduce, also known as MRv2, which separates the resource management and scheduling functions from the data processing component.
- YARN allows running MRv1 applications on a YARN cluster, as well as other types of applications such as Spark, Hive, and Pig.
- To run MRv1 applications on YARN, the following steps are required:
 - Configure the YARN cluster with the appropriate settings for MRv1, such as the map and reduce memory allocation, the number of map and reduce slots per node, and the job history server address.
 - Use the **yarn** command in the Hadoop-YARN bin folder to submit the MRv1 application, instead of the **hadoop** command in the Hadoop-MapReduce bin folder.
 - Specify the **mapred.job.tracker** property as **yarn** in the MRv1 application configuration file, to indicate that the application should run on YARN.
 - Use the ResourceManager web interface to monitor the MRv1 application status, progress, and logs on the YARN cluster.
- Running MRv1 applications on YARN has some advantages, such as:
 - Improved resource utilization and scalability, as YARN can dynamically allocate resources to different applications based on their needs and priorities.
 - Increased compatibility and flexibility, as YARN can support multiple versions and frameworks of MapReduce, as well as other types of applications.
 - Reduced operational overhead, as YARN simplifies the cluster management and administration tasks.

NoSQL Databases

- NoSQL databases are databases that do not use the relational model or SQL (Structured Query Language) to store and manipulate data.
- NoSQL databases are designed to handle large volumes of unstructured, semi-structured, or structured data that may change rapidly or frequently.
- NoSQL databases offer flexible schemas, high scalability, high performance, and easy distribution across multiple nodes or servers.
- NoSQL databases can be classified into four main types based on their data model: document, key-value, wide-column, and graph.
- Document databases store data as JSON-like documents, where each document has a unique key and can contain nested fields and arrays. Document databases are suitable for applications that need to store complex and dynamic data, such as e-commerce, content management, or social media. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.
- Key-value databases store data as pairs of keys and values, where each key is unique and can be used to retrieve the associated value. Key-value databases are suitable for applications that need to store simple and fast data, such as caching, session management, or user preferences. Examples of key-value databases are Redis, Memcached, and Couchbase.
- Wide-column databases store data as tables of rows and columns, where each row has a unique key and each column can have different attributes and values. Wide-column databases are suitable for applications that need to store large and sparse data, such as analytics, recommendation systems, or Internet of Things. Examples of wide-column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, where each node represents an entity and each edge represents a relationship between entities. Graph databases are suitable for applications that need to store and query complex and interconnected data, such as social networks, fraud detection, or knowledge graphs. Examples of graph databases are Neo4j, OrientDB, and ArangoDB.

Introduction to NoSQL databases

- NoSQL databases are databases that do not use the SQL language or the relational model for data storage and retrieval.
- NoSQL stands for “not only SQL” or “non-relational” to emphasize the differences from traditional relational databases.
- NoSQL databases are designed to handle large volumes of unstructured, semi-structured, or structured data that may change rapidly or frequently.
- NoSQL databases offer flexible schemas, horizontal scalability, high availability, and high performance for various types of data and applications.
- NoSQL databases can be classified into four main types based on their data model: document, key-value, wide-column, and graph.

- Document databases store data as JSON-like documents, which can have different fields and structures. They allow querying and indexing based on the document content. Examples of document databases are MongoDB, CouchDB, and Elasticsearch.
- Key-value databases store data as pairs of keys and values, which can be any type of data. They allow fast access and update of data based on the key. Examples of key-value databases are Redis, DynamoDB, and Couchbase.
- Wide-column databases store data as rows and columns, but unlike relational databases, the columns can vary for each row. They allow efficient querying and analysis of large amounts of structured or semi-structured data. Examples of wide-column databases are Cassandra, HBase, and Bigtable.
- Graph databases store data as nodes and edges, which represent entities and relationships. They allow complex queries and operations based on the graph structure and properties. Examples of graph databases are Neo4j, OrientDB, and ArangoDB.

MongoDB

- MongoDB is an open source, nonrelational database management system (DBMS) that uses flexible documents instead of tables and rows to process and store various forms of data .
- MongoDB is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use.
- MongoDB is a document-oriented database, which means that data is stored as documents, and documents are grouped in collections. The document model is a lot more natural for developers to work with because documents are self-contained and can be treated as objects.
- MongoDB supports ad-hoc queries, secondary indexing, and real-time aggregations to provide powerful ways to access and analyze your data .
- MongoDB is licensed under the Server Side Public License (SSPL) which is deemed non-free by several distributions.

Introduction to MongoDB

MongoDB is a popular and widely used database system that offers the following features and characteristics:

- It is an **open source, nonrelational** database management system (DBMS) that uses **flexible documents** instead of tables and rows to process and store various forms of data .
- It is a **distributed database** at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use.
- It is a **document-oriented database**, which means that data is stored as documents, and documents are grouped in collections. The document

model is a lot more natural for developers to work with because documents are self-contained and can be treated as objects.

- It uses **JSON-like documents** with optional schemas. JSON (JavaScript Object Notation) is a human-readable and widely used data format that can store various types of data, such as strings, numbers, arrays, and objects.
- It supports **ad-hoc queries**, **secondary indexing**, and **real-time aggregations** to provide powerful ways to access and analyze your data.
- It is **free to use** for personal and commercial purposes, but it is licensed under the **Server Side Public License (SSPL)** which is deemed non-free by several distributions .

MongoDB is designed to help developers and organizations in any industry to remove barriers to innovation and build applications faster and easier. It is suitable for various use cases, such as web applications, mobile applications, IoT, analytics, content management, and more.

Data Types in MongoDB

MongoDB is a document-oriented database that stores data in BSON format, which is a binary-encoded version of JSON. BSON supports various data types, some of which are specific to MongoDB. The following are some of the common data types in MongoDB:

- **String:** This is the most commonly used data type to store text data. Strings in MongoDB must be UTF-8 valid.
- **Integer:** This is a data type that is used to store numerical values, such as integers in other programming languages. MongoDB supports 32-bit or 64-bit integers, depending on the server.
- **Boolean:** This is a data type that is used to store a logical value, either true or false.
- **Double:** This is a data type that is used to store floating-point numbers, such as decimals or fractions.
- **Date:** This is a data type that is used to store the date and time as a UNIX timestamp, which is the number of milliseconds since January 1, 1970. MongoDB provides various methods to manipulate and format dates.
- **ObjectId:** This is a data type that is used to store a unique identifier for each document in a collection. ObjectId is a 12-byte value that consists of a 4-byte timestamp, a 5-byte random value, and a 3-byte incrementing counter. MongoDB automatically generates an ObjectId for each document if not specified.
- **Array:** This is a data type that is used to store a list of values, such as strings, numbers, or other documents. Arrays can be nested and can have different data types in the same array.
- **Object:** This is a data type that is used to store a document, which is a set of key-value pairs. Objects can be nested and can have different data

types in the same object.

- **JavaScript:** This is a data type that is used to store a JavaScript function or expression. MongoDB can execute JavaScript code in certain contexts, such as the `$where` operator or the `mapReduce` function.
- **JavaScript with scope:** This is a data type that is used to store a JavaScript function or expression along with a scope object that defines the variables and values available to the function.
- **Null:** This is a data type that is used to store a null value, which represents the absence of a value.
- **Binary:** This is a data type that is used to store binary data, such as images, audio, or video. Binary data is stored as a subtype and a byte array.
- **Regular expression:** This is a data type that is used to store a regular expression, which is a pattern that can be used to match or search for strings. MongoDB uses the Perl Compatible Regular Expression (PCRE) library to perform regular expression operations.
- **Symbol:** This is a data type that is used to store a symbol, which is similar to a string but intended for languages that use symbols, such as Ruby.
- **NumberLong:** This is a data type that is used to store a 64-bit integer explicitly. This is useful for languages that do not support 64-bit integers, such as JavaScript. MongoDB provides the `NumberLong()` constructor to create a `NumberLong` value.
- **NumberInt:** This is a data type that is used to store a 32-bit integer explicitly. This is useful for languages that do not support 32-bit integers, such as Java. MongoDB provides the `NumberInt()` constructor to create a `NumberInt` value.
- **NumberDecimal:** This is a data type that is used to store a 128-bit decimal number explicitly. This is useful for applications that require high precision arithmetic, such as financial or scientific calculations. MongoDB provides the `NumberDecimal()` constructor to create a `NumberDecimal` value.

Creating documents in MongoDB

- MongoDB is a NoSQL database that stores data as documents in collections.
- A document is a JSON-like object that can have different fields and values.
- To create documents in MongoDB, you can use the following methods:
 - `insertOne()`: inserts a single document into a collection. If the collection does not exist, it will be created automatically. The syntax is: `db.collection.insertOne(document)`.
 - `insertMany()`: inserts an array of documents into a collection. If the collection does not exist, it will be created automatically. The syntax is: `db.collection.insertMany(documents)`.
 - `insert()`: inserts one or more documents into a collection. If the

collection does not exist, it will be created automatically. The syntax is: `db.collection.insert(documents)`.

- Each document must have a unique `_id` field that acts as a primary key. If the document does not specify the `_id` field, MongoDB will generate an `ObjectId` for it automatically.
- You can use a MongoDB Playground to create documents using the MongoDB CRUD Operators. A MongoDB Playground is a VS Code extension that allows you to run MongoDB commands and scripts in a code editor.
- You can also use the `create` command to create collections and indexes explicitly. The syntax is: `db.createCollection(name, options)`. The options parameter can specify various collection properties, such as validation rules, storage engine, etc. The syntax for creating indexes is: `db.collection.createIndex(keys, options)`. The keys parameter specifies the fields and directions for the index. The options parameter can specify various index properties, such as uniqueness, name, etc.

Updating documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To update one or more documents in a collection, MongoDB provides the following methods:
 - `db.collection.updateOne()`: Updates a single document that matches the filter condition.
 - `db.collection.updateMany()`: Updates all documents that match the filter condition.
 - `db.collection.replaceOne()`: Replaces a single document that matches the filter condition with a new document.
- The update methods take the following parameters:
 - **filter**: A query document that specifies the criteria for selecting the documents to update.
 - **update**: An update document that specifies the modifications to apply to the matching documents. The update document can use update operators, such as `$set`, `$inc`, `$push`, etc., to modify the fields of the documents.
 - **options**: An optional document that specifies additional settings, such as `upsert`, `multi`, `writeConcern`, etc.
- The update methods return a `WriteResult` object that contains information about the operation, such as the number of matched and modified documents, the `_id` of the upserted document, etc.
- To update a document in MongoDB, follow these steps:
 1. Connect to the MongoDB server and access the database and collection that contain the document to update.
 2. Use one of the update methods with the appropriate parameters to perform the update operation.
 3. Check the `WriteResult` object to verify the outcome of the operation.

- Here is an example of updating a document in MongoDB using the `db.collection.updateOne()` method:

```
// Connect to the MongoDB server and access the test database and the products collection
> use test
switched to db test
> db.products.find()
{ "_id" : ObjectId("60f9a9b9f8a3a3b3a3b3a3b3"), "name" : "Laptop", "price" : 500, "stock" : 50 }
{ "_id" : ObjectId("60f9a9c0f8a3a3b3a3b3a3b4"), "name" : "Mouse", "price" : 10, "stock" : 50 }

// Update the price of the laptop to 550
> db.products.updateOne({name: "Laptop"}, {$set: {price: 550}})
{ "acknowledged" : true, "matchedCount" : 1, "modifiedCount" : 1 }

// Verify the update
> db.products.find()
{ "_id" : ObjectId("60f9a9b9f8a3a3b3a3b3a3b3"), "name" : "Laptop", "price" : 550, "stock" : 50 }
{ "_id" : ObjectId("60f9a9c0f8a3a3b3a3b3a3b4"), "name" : "Mouse", "price" : 10, "stock" : 50 }
```

Deleting documents in MongoDB

- MongoDB is a document-oriented database that stores data in collections of JSON-like documents.
- To delete documents from a collection, MongoDB provides various methods and commands that can be used in the mongo shell or in a driver.
- The following are some of the ways to delete documents in MongoDB:
 - The `db.collection.remove()` method: This method takes a query filter as a parameter and deletes all the documents that match the filter. If no filter is specified, it deletes all the documents in the collection. Optionally, it can take a second parameter that specifies whether to delete just one document or not. For example, `db.users.remove({name: "Alice"})` deletes all the documents in the `users` collection where the `name` field is “Alice”. `db.users.remove({name: "Alice"}, true)` deletes only one document that matches the filter.
 - The `delete` command: This command can also be used to delete documents from a collection. Internally, the `remove` method also uses the `delete` command. The `delete` command needs to be run with the `db.runCommand()` method and an object as a parameter. The object must have the `delete` field set to the name of the collection and the `deletes` field set to an array of objects that specify the query filters and the limits for each deletion. For example, `db.runCommand({delete: "users", deletes: [{q: {name: "Alice"}, limit: 0}, {q: {age: {$gt: 30}}, limit: 1}]})` deletes all the documents in the `users` collection where the

`name` field is “Alice” and one document where the `age` field is greater than 30.

- The `db.collection.deleteOne()` method: This method deletes at most one document that matches a given filter. It returns an object that contains the number of deleted documents and a write concern information. For example, `db.users.deleteOne({name: "Alice"})` deletes one document in the `users` collection where the `name` field is “Alice” and returns an object like `{deletedCount: 1, ok: 1}`.
- The `db.collection.deleteMany()` method: This method deletes all the documents that match a given filter. It returns an object that contains the number of deleted documents and a write concern information. For example, `db.users.deleteMany({age: {$gt: 30}})` deletes all the documents in the `users` collection where the `age` field is greater than 30 and returns an object like `{deletedCount: 5, ok: 1}`.

Querying documents in MongoDB

- MongoDB is a document-oriented database that stores data in JSON-like format.
- To query documents in MongoDB, one can use the `find()` method on a collection, which returns a cursor object that can be iterated over to access the matching documents.
- The `find()` method can take two optional parameters: a query filter and a projection.
- A query filter is a document that specifies the criteria for selecting the documents. It can use various operators, such as `$eq`, `$gt`, `$in`, `$and`, `$or`, etc., to compare the values of the document fields with the specified values.
- A projection is a document that specifies which fields to include or exclude from the returned documents. It can use the value `1` to include a field, or `0` to exclude it. By default, the `_id` field is always included, unless explicitly excluded.
- For example, the following query finds all the documents in the `users` collection where the `age` field is greater than 25, and returns only the `name` and `email` fields:

```
db.users.find({age: {$gt: 25}}, {name: 1, email: 1, _id: 0})
```

- To sort the results of a query, one can use the `sort()` method on the cursor object, which takes a document that specifies the sort order for each field. The value `1` means ascending order, and `-1` means descending order.
- For example, the following query sorts the results by the `name` field in ascending order, and then by the `age` field in descending order:

```
db.users.find().sort({name: 1, age: -1})
```

- To limit the number of results returned by a query, one can use the `limit()` method on the cursor object, which takes a positive integer as the argument.
- For example, the following query returns only the first 10 documents from the `users` collection:

```
db.users.find().limit(10)
```

- To skip some results from the beginning of a query, one can use the `skip()` method on the cursor object, which takes a positive integer as the argument.
- For example, the following query skips the first 5 documents and returns the next 10 documents from the `users` collection:

```
db.users.find().skip(5).limit(10)
```

- To count the number of documents that match a query, one can use the `count()` method on the cursor object, which returns an integer.
- For example, the following query counts the number of documents in the `users` collection where the `gender` field is "female":

```
db.users.find({gender: "female"}).count()
```

- To update one or more documents that match a query, one can use the `updateOne()`, `updateMany()`, or `replaceOne()` methods on a collection, which take a query filter and an update document as the arguments.
- The update document can use various operators, such as `$set`, `$inc`, `$push`, `$pull`, etc., to modify the values of the document fields.
- The `updateOne()` method updates the first document that matches the query filter, while the `updateMany()` method updates all the documents that match the query filter.
- The `replaceOne()` method replaces the first document that matches the query filter with the update document.
- For example, the following query updates the `email` field of the first document in the `users` collection where the `name` field is "Alice":

```
db.users.updateOne({name: "Alice"}, {$set: {email: "alice@example.com"}})
```

- To delete one or more documents that match a query, one can use the `deleteOne()` or `deleteMany()` methods on a collection, which take a query filter as the argument.
- The `deleteOne()` method deletes the first document that matches the query filter, while the `deleteMany()` method deletes all the documents that match the query filter.
- For example, the following query deletes all the documents in the `users` collection where the `age` field is less than 18:

```
db.users.deleteMany({age: {$lt: 18}})
```

Indexing in MongoDB

- Indexing is a technique that improves the efficiency of query processing in MongoDB by storing some information related to the documents in a special data structure .
- Indexes are ordered by the value of the field or fields specified in the index .
- MongoDB supports various types of indexes, such as single field, compound, multikey, text, geospatial, hashed, and wildcard .
- To create an index, MongoDB provides a method called `createIndex()` that takes a document as a parameter, where the keys are the fields to index and the values are the index types .
- To drop an index, MongoDB provides a method called `dropIndex()` that takes a document as a parameter, where the keys are the fields to drop and the values are the index types.
- Indexes can improve the performance of queries by reducing the number of documents that need to be scanned, but they also have some drawbacks, such as increasing the storage space and the time required to insert or update documents.
- Therefore, it is important to choose the appropriate indexes for the queries and the data, and to monitor the index usage and statistics.

Aggregation in MongoDB

- Aggregation is the process of selecting data from a collection in MongoDB and performing various operations on the selected data to produce computed results .
- Aggregation operations are expressions that can be used to reduce and summarize data in MongoDB.
- Aggregation operations can be performed using the **aggregation pipeline**, the **map-reduce function**, or the **single purpose aggregation methods**.
- The aggregation pipeline is a framework that allows you to create a sequence of stages, each of which performs an operation on the input documents and outputs the modified documents to the next stage .
- The aggregation pipeline supports various stages, such as `$match`, `$group`, `$sort`, `$project`, `$unwind`, `$lookup`, and more .
- The aggregation pipeline can be used for various purposes, such as filtering, grouping, transforming, joining, sorting, and aggregating data .
- The aggregation pipeline can be created using the `aggregate()` method, which accepts one or more stage names as arguments.
- The map-reduce function is a way of performing aggregation by applying a map function to each document and then reducing the results by a key.
- The map-reduce function can be used for complex aggregation tasks that cannot be easily expressed by the aggregation pipeline.
- The map-reduce function can be created using the `mapReduce()` method,

which accepts a map function, a reduce function, and an output collection as arguments.

- The single purpose aggregation methods are simple methods that perform specific aggregation tasks, such as counting, finding the minimum or maximum value, or calculating the average.
- The single purpose aggregation methods can be used for simple aggregation tasks that do not require multiple stages or complex logic.
- The single purpose aggregation methods include `count()`, `distinct()`, `group()`, and more.

Capped Collections in MongoDB

- Capped collections are fixed-size collections that support high-throughput operations that insert and retrieve documents based on insertion order .
- Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection .
- You must create capped collections explicitly using the `db.createCollection()` method, which is a mongosh helper for the create command .
- When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection .
- You can also specify the maximum number of documents that the capped collection can store, but this is optional.
- Capped collections have the following characteristics and limitations :
 - Capped collections guarantee preservation of the insertion order. As a result, queries do not need an index to return documents in insertion order. Without this indexing overhead, capped collections can support higher insertion throughput.
 - Capped collections automatically remove the oldest documents in the collection without requiring scripts or explicit remove operations.
 - Capped collections cannot be sharded. However, you can create a sharded cluster that contains replica sets, where each replica set contains one or more capped collections.
 - Capped collections cannot use the `$out` aggregation pipeline stage.
 - You cannot remove documents from a capped collection. To remove all documents from a capped collection, use the `drop()` method.
 - You cannot delete a document and then insert a new document to replace the deleted document. However, you can use the `update()` method with the `overwrite` option to replace the contents of an existing document in a capped collection.
 - You cannot update a document in a capped collection if the update operation causes the document to grow beyond its original size. However, you can update a document in a capped collection if the update operation does not cause the document to grow beyond its original size.

- You can create indexes on a capped collection, but you cannot create a text index.
- You can create a capped collection that is also a time series collection. See Time Series Collections for more information.
- Capped collections are useful for storing log information, high volume of data, and cache information .

Spark

Spark is a word that has two distinct meanings. As a noun, it means a small fiery particle thrown off from a fire, alight in ashes, or produced by striking together two hard surfaces such as stone or metal. Or as a verb, it means emit sparks of fire or electricity.

Some points to remember about spark are:

- A spark can also be a small flash of light produced by a sudden disruptive electrical discharge through the air.
- A spark can also be an electrical discharge serving to ignite the explosive mixture in an internal combustion engine.
- A spark can also be a trace of a specified quality or intense feeling, such as anger, interest, or excitement.
- A spark can also be a sense of liveliness and excitement, or a lively young fellow.
- A spark can also be used to describe the act of providing the stimulus for something, such as a fire, a protest, or a romance.
- A spark can also be used to describe the act of engaging in courtship.

Some examples of using spark in sentences are:

- A log fire was sending sparks onto the rug.
- Angry sparks were flashing in her eyes.
- There was a spark of light.
- A tiny spark of anger flared within her.
- There was a spark between them at their first meeting.
- The ignition sparks as soon as the gas is turned on.
- The explosion sparked a fire.
- The severity of the plan sparked off street protests.
- He went a sparking among the rosy country girls.

Installing spark

Spark is an open-source distributed computing framework that can process large-scale data sets using in-memory caching and parallel processing. Spark can run on various platforms, such as Hadoop, Mesos, Kubernetes, standalone, or in the cloud. To install Spark, you need to follow these steps:

- Download the latest version of Spark from the official website: <https://spark.apache.org/downloads.html>. Choose a package type

- Extract the downloaded file to a location of your choice, such as `/opt/spark`.
- Set the environment variables `SPARK_HOME` and `PATH` to point to the Spark installation directory and its `bin` subdirectory, respectively. For example, in Linux, you can add these lines to your `.bashrc` file:

- Verify that Spark is installed correctly by running the `spark-shell` command, which will launch an interactive Scala shell with Spark. You should see a welcome message and a prompt like this:

```

  /---/
 / \  /---\  /---\  /---\  /---\
 \  \  \---\  \---\  \---\  \---\
 /---/  .---\  ,---/  /---\  /---\
 /---/

```

version 3.2.0

```
scala>
```

- ```
spark-submit --class org.apache.spark.examples.SparkPi \
--master spark://master:7077 \
--executor-memory 1G \
--total-executor-cores 4 \
$SPARK_HOME/examples/jars/spark-examples_2.12-3.2.0.jar 1000
```

- ## Spark Applications

- 89

- The driver process runs the main function of the program, creates a Spark session, and coordinates the execution of tasks across the executors.
- The Spark session is the entry point to the Spark API, and it allows the driver to access the cluster manager, which allocates resources for the application.
- The executors are processes that run on the worker nodes, and they are responsible for executing the tasks assigned by the driver, reading and writing data, and caching intermediate results in memory or disk.
- A task is a unit of work that applies a transformation or an action to a partition of a distributed dataset, such as a Resilient Distributed Dataset (RDD) or a DataFrame.
- A partition is a logical chunk of a distributed dataset that is stored on a single node and processed by a single task.
- Spark applications can be written in various languages, such as Scala, Python, Java, R, or C#, and they can use various components of the Spark framework, such as Spark SQL, Spark Streaming, MLlib, GraphX, or SparkR.
- Spark applications can run on various cluster managers, such as Apache Hadoop YARN, Apache Mesos, Kubernetes, or Spark's own standalone cluster manager.
- Spark applications can also run locally on a single machine for testing or development purposes.

## Jobs in Spark

Spark is a distributed computing framework that allows processing large-scale data in parallel using clusters of machines. Spark can run on various platforms such as Hadoop, Kubernetes, Mesos, or standalone. Spark supports multiple programming languages such as Scala, Python, Java, and R. Spark also provides various libraries for data analysis, machine learning, graph processing, and streaming.

There are different types of jobs in Spark, depending on the role and the skill set required. Some of the common jobs in Spark are:

- **Spark Engineer:** A Spark engineer is responsible for developing, testing, and deploying Spark applications using various languages and frameworks. A Spark engineer should have a strong background in programming, data structures, algorithms, and distributed systems. A Spark engineer should also be familiar with Spark core concepts such as RDDs, DataFrames, transformations, actions, jobs, stages, and tasks. A Spark engineer should also be able to use Spark libraries such as Spark SQL, Spark MLlib, Spark GraphX, and Spark Streaming. A Spark engineer should also have experience with cloud platforms such as AWS, Azure, or GCP, and tools such as Docker, Kubernetes, or Mesos. A Spark engineer should also be able to optimize Spark performance, troubleshoot issues, and debug errors. A Spark engineer job can be found in various domains such as e-commerce,

finance, healthcare, social media, gaming, etc.

- **Spark Program Lead:** A Spark program lead is responsible for managing and overseeing Spark projects and teams. A Spark program lead should have a strong background in project management, leadership, communication, and collaboration. A Spark program lead should also have a good understanding of Spark architecture, design, and best practices. A Spark program lead should also be able to define the scope, objectives, deliverables, and timelines of Spark projects. A Spark program lead should also be able to coordinate with stakeholders, clients, and vendors. A Spark program lead should also be able to monitor, evaluate, and report on the progress and performance of Spark projects and teams. A Spark program lead job can be found in various domains such as education, non-profit, government, etc.
- **Spark Delivery Driver:** A Spark delivery driver is responsible for delivering goods and services to customers using a vehicle. A Spark delivery driver should have a valid driver's license, a clean driving record, and a reliable vehicle. A Spark delivery driver should also have good customer service, communication, and navigation skills. A Spark delivery driver should also be able to follow instructions, handle payments, and report any issues. A Spark delivery driver job can be found in various domains such as food, retail, logistics, etc.
- **Spark Electrical Project Manager:** A Spark electrical project manager is responsible for planning, executing, and managing electrical projects using Spark Power. Spark Power is a company that provides electrical solutions such as power generation, distribution, transmission, and storage. A Spark electrical project manager should have a strong background in electrical engineering, construction, and management. A Spark electrical project manager should also have a good understanding of Spark Power's products, services, and standards. A Spark electrical project manager should also be able to estimate costs, schedule resources, manage risks, and ensure quality and safety of electrical projects. A Spark electrical project manager job can be found in various domains such as renewable energy, industrial, commercial, etc.

### Stages and Tasks in Spark

- Spark is a distributed computing framework that allows parallel processing of large-scale data using a cluster of nodes.
- Spark divides a computation into smaller units called **jobs**, **stages**, and **tasks**.
- A **job** is a parallel computation of tasks that is triggered by an action operation, such as **save**, **count**, or **collect**.
- A **stage** is a set of parallel tasks that operate on a subset of the data. The tasks within a stage are executed on different executor nodes in the

cluster, and the data is partitioned into smaller chunks called **partitions**, which are processed by the tasks in parallel.

- A **task** is a single operation, such as **map**, **filter**, or **reduce**, applied to a single partition.
- Spark creates a **directed acyclic graph (DAG)** to represent the logical execution plan of a job. The DAG consists of one or more stages, and each stage consists of one or more tasks.
- Spark divides a job into stages based on **shuffle boundaries**, which are operations that require data to be redistributed across the cluster, such as **groupBy**, **join**, or **sortBy**.
- There are two types of stages in Spark: **ShuffleMapStage** and **ResultStage**.
- A **ShuffleMapStage** is an intermediate stage that prepares data for subsequent stages by performing a shuffle operation. The output of this stage is stored in memory or disk as **shuffle files**, which are used as input for the next stage.
- A **ResultStage** is a final stage that performs an action operation on the data and returns the result to the driver node or writes it to an external storage system.
- Spark optimizes the execution of a job by using **pipelining**, which is the process of combining multiple tasks within a stage into a single task, and **caching**, which is the process of storing intermediate data in memory or disk for faster access.

## Resilient Distributed Databases in Spark

- Resilient Distributed Databases (RDDs) are the fundamental data structure of Spark .
- RDDs are immutable distributed collections of objects that can be operated on in parallel .
- Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster .
- RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .
- RDDs are reliable and memory-efficient when it comes to parallel processing.
- RDDs are fault-tolerant, meaning they can recover from failures and errors by using a lineage graph that records how the dataset was constructed .
- RDDs offer two types of operations: transformations and actions.
  - Transformations create a new RDD from an existing one, such as **map**, **filter**, **join**, etc.
  - Actions return a value to the driver program or write data to an external storage system, such as **count**, **collect**, **save**, etc.
- RDDs can be created from various sources, such as parallelizing an existing collection, reading from a file system, or applying a transformation to an existing RDD.

- RDDs can be cached or persisted in memory or disk for faster access in future operations .
- RDDs can be controlled by two parameters: persistence level and partitioning scheme.
  - Persistence level determines how and where the RDD is stored, such as memory only, memory and disk, disk only, etc.
  - Partitioning scheme determines how the RDD is split into partitions, such as hash partitioning, range partitioning, custom partitioning, etc.

### **Anatomy of a Spark job run**

- A Spark job is a parallel computation that runs on a cluster of nodes and consists of one or more stages.
- A stage is a set of tasks that perform the same computation on different partitions of the input data.
- A task is a unit of work that runs on a single executor and processes a single partition of the input data.
- An executor is a process that runs on a worker node and executes tasks assigned by the driver.
- A driver is the process that coordinates the execution of a Spark job and runs the user-defined main function.
- A Spark job run follows these steps:
  1. The user submits a Spark application to the cluster manager, which allocates resources and launches the driver and executors.
  2. The driver converts the user-defined logic into a logical plan, which is then optimized and translated into a physical plan of stages and tasks.
  3. The driver sends the tasks to the executors and monitors their progress and status.
  4. The executors run the tasks and communicate with each other to exchange data if needed.
  5. The executors send the results and metrics back to the driver, which collects and aggregates them.
  6. The driver terminates the executors and releases the resources when the job is finished.

### **Spark on YARN**

- Spark is a distributed computing framework that can run on various cluster managers, such as YARN, Mesos, Kubernetes, or standalone mode .
- YARN (Yet Another Resource Negotiator) is a cluster manager that is part of the Apache Hadoop project. It is responsible for allocating resources

and scheduling tasks on a Hadoop cluster .

- Running Spark on YARN allows users to leverage the benefits of both Spark and YARN, such as scalability, fault tolerance, security, and resource management .
- To run Spark on YARN, users need to have a binary distribution of Spark that is built with YARN support. Binary distributions can be downloaded from the downloads page of the project website .
- There are two deploy modes that can be used to launch Spark applications on YARN: cluster mode and client mode .
  - In cluster mode, the Spark driver runs inside an application master process that is managed by YARN on the cluster, and the client can go away after initiating the application .
  - In client mode, the Spark driver runs on the client machine that submits the application, and the application master is only used for requesting resources from YARN .
- To run Spark on YARN, users need to configure some Spark and YARN properties, such as `spark.yarn.archive`, `spark.yarn.jars`, `spark.yarn.queue`, `spark.yarn.am.memory`, etc .
- Users can use `spark-submit`, `spark-shell`, or `pyspark` commands to launch Spark applications on YARN, with the `-master` option set to `yarn` and the `-deploy-mode` option set to either `cluster` or `client` .
- Users can monitor and manage their Spark applications on YARN using the YARN web UI, the Spark web UI, or the Spark history server .

## SCALA

Scala is a general-purpose programming language that supports both object-oriented and functional programming paradigms. It is designed to be concise, expressive, and interoperable with Java. Scala runs on the Java Virtual Machine (JVM) and can use any Java library. Some of the main features of Scala are:

- It is a **strongly typed** language, which means that every value has a type that is checked at compile time. This helps to prevent type errors and improve performance.
- It is a **high-level** language, which means that it abstracts away many low-level details and provides powerful constructs to manipulate data and control flow. This helps to write concise and readable code.
- It is a **multi-paradigm** language, which means that it supports different programming styles and techniques. Scala allows programmers to use object-oriented, functional, imperative, and concurrent programming in the same program. This helps to choose the best approach for each problem.
- It is a **scalable** language, which means that it can handle complex and large-scale applications as well as simple and small ones. Scala supports modularity, concurrency, parallelism, and distributed computing. This helps to build high-performance and reliable systems.

## Introduction to Scala

Scala is a general-purpose programming language that supports both object-oriented and functional programming paradigms. It is designed to be concise, expressive, and interoperable with Java. Scala runs on the Java Virtual Machine (JVM) and can use any Java library. Scala also has a JavaScript compiler that allows Scala code to run in web browsers.

Some of the main features of Scala are:

- **Strong static typing:** Scala enforces type safety at compile time, which helps to avoid many runtime errors and bugs. Scala also supports type inference, which reduces the need for explicit type annotations.
- **Unified type system:** Scala treats everything as an object, including primitive types, functions, and classes. Scala also supports generic types, abstract types, and type aliases for more flexibility and readability.
- **Multiple inheritance:** Scala allows a class to inherit from multiple traits, which are similar to interfaces in Java but can also contain concrete methods and fields. Traits can be mixed in at the class definition or at the object creation.
- **Pattern matching:** Scala provides a powerful and concise way of handling multiple cases with a single expression. Pattern matching can be used to decompose complex data structures, implement algebraic data types, and handle exceptions.
- **Higher-order functions:** Scala supports passing functions as arguments to other functions, returning functions from functions, and storing functions in variables. Scala also provides anonymous functions, currying, and partial application for more concise and expressive code.
- **Immutability:** Scala encourages the use of immutable data structures and values, which are easier to reason about and avoid side effects. Scala also provides mutable versions of some collections for performance reasons, but they are clearly distinguished from the immutable ones.
- **Lazy evaluation:** Scala supports lazy evaluation, which means that an expression is only evaluated when it is needed. This can improve performance and avoid unnecessary computations. Scala also provides lazy `val`, which is a value that is only initialized once when it is first accessed.
- **Case classes:** Scala provides a special kind of class called a case class, which is useful for modeling immutable data. Case classes automatically provide methods for equality, hashing, copying, and string representation. They also support pattern matching and can be used as algebraic data types.
- **Singleton objects:** Scala provides a way of defining a single instance of a class using the `object` keyword. Singleton objects can be used to implement static methods and fields, enumerations, and companion objects. Companion objects are singleton objects that have the same name as a class and can access its private members.
- **Interoperability with Java:** Scala can seamlessly interoperate with

Java code and libraries, as both languages run on the JVM. Scala can call Java methods, implement Java interfaces, extend Java classes, and use Java annotations. Java can also call Scala methods, implement Scala traits, extend Scala classes, and use Scala generics.

## Classes and Objects in Scala

- Classes in Scala are blueprints for creating objects. They can contain methods, values, variables, types, objects, traits, and classes which are collectively called members .
- Objects in Scala are single instances of their own definitions. They can be used to hold static methods or values, or to implement singleton patterns.
- To define a class in Scala, use the keyword `class` followed by an identifier and an optional list of constructor parameters . For example:

```
// A class named Point with two parameters x and y
class Point(val x: Int, val y: Int)
```

- To create an object of a class, use the keyword `new` followed by the class name and the arguments for the constructor parameters . For example:

```
// An object of class Point with x = 3 and y = 4
val p = new Point(3, 4)
```

- To define an object in Scala, use the keyword `object` followed by an identifier. An object can extend a class or a trait, or be standalone. For example:

```
// An object named Hello that extends the App trait
object Hello extends App {
 println("Hello, world!")
}
```

- An object can also have the same name as a class, in which case it is called a companion object. A companion object can access the private members of the class, and vice versa. For example:

```
// A class named Complex with two parameters real and imaginary
class Complex(val real: Double, val imaginary: Double)

// A companion object named Complex with a factory method
object Complex {
 // A method that creates a Complex object from polar coordinates
 def fromPolar(magnitude: Double, angle: Double) = {
 new Complex(magnitude * math.cos(angle), magnitude * math.sin(angle))
 }
}
```

- A companion object can also define an `apply` method, which can be used to create objects of the class without using the `new` keyword. For example:



```

// A class named Person with two parameters firstName and lastName
class Person(val firstName: String, val lastName: String)

// A companion object named Person with an apply method
object Person {
 // A method that creates a Person object from a full name
 def apply(fullName: String) = {
 val nameParts = fullName.split(" ")
 new Person(nameParts(0), nameParts(1))
 }
}

// Creating a Person object using the apply method
val p = Person("John Doe")

```

## Basic types and operators in Scala

- Scala has a rich set of built-in types, such as `Int`, `Double`, `Boolean`, `String`, etc.
- Scala also supports user-defined types, such as classes, traits, and objects.
- Scala has a unified type system, which means that every value is an object and every operation is a method call.
- Scala has two main categories of types: value types and reference types.
- Value types are stored directly on the stack or embedded in objects, and they include `Byte`, `Short`, `Int`, `Long`, `Char`, `Float`, `Double`, `Boolean`, and `Unit`.
- Reference types are stored on the heap and they include `AnyRef`, `String`, `Array`, `List`, and user-defined types.
- Scala has a special type called `Any`, which is the supertype of all types, and a special type called `Nothing`, which is the subtype of all types.
- Scala has a special type called `Null`, which is the subtype of all reference types, and a special value called `null`, which is the only instance of `Null`.
- Scala has a special type called `AnyVal`, which is the supertype of all value types, and a special type called `Unit`, which is the subtype of `AnyVal` and has a single value `()`.
- Scala has a special type called `Function`, which is the supertype of all function types, such as `Int => String` or `(Int, String) => Boolean`.
- Scala has a rich set of operators, such as `+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`, `!`, `&`, `|`, `^`, `<<`, `>>`, `>>>`, `+=`, `-=`, `*=`, `/=`, `%=`, `++`, `--`, `::`, `:+`, `+::`, `++:`, `::+`, `apply`, `update`, etc.
- Scala operators are actually methods, which means that `a + b` is equivalent to `a.+(b)`, and `a(b)` is equivalent to `a.apply(b)`.
- Scala operators can be overloaded by defining methods with the same name and signature as the operator.
- Scala operators can be infix, prefix, or postfix, depending on their position and arity.

- Scala operators have precedence and associativity rules, which determine the order of evaluation and grouping of operands.
- Scala operators can be used to create custom DSLs (domain-specific languages), such as `a + b * c` or `a :: b :: c`.

### Built-in control structures in Scala

- Scala has only a handful of built-in control structures: **if**, **while**, **for**, **try**, **match**, and **function calls**.
- The reason Scala has so few is that it has included **function literals** since its inception. A function literal is a function that is not defined by a name, but by its parameters and body, such as `(a:Int, b:Int) => a + b`.
- Scala's control structures are closer to the **functional style** than the imperative style. This means that they are expressions that return a value, rather than statements that perform side effects.
- Some examples of Scala's control structures are:

- **if/else**: This is a conditional expression that evaluates a condition and returns one value if it is true, and another value if it is false. For example:

```
val x = 10
val y = if (x > 0) "positive" else "negative"
// y is "positive"
```

- **while**: This is a loop that executes a block of code repeatedly as long as a condition is true. For example:

```
var i = 0
while (i < 10) {
 println(i)
 i += 1
}
// prints 0 to 9
```

- **for**: This is a loop that iterates over a collection or a range of values, and optionally applies a filter or a transformation. For example:

```
for (i <- 1 to 10 if i % 2 == 0) {
 println(i)
}
// prints 2, 4, 6, 8, 10
```

- **try/catch/finally**: This is a way to handle exceptions that may occur during the execution of a block of code. The try block contains the code that may throw an exception, the catch block contains the

code that handles the exception, and the finally block contains the code that is always executed regardless of the exception. For example:

```
try {
 val n = 10 / 0 // throws ArithmeticException
} catch {
 case e: ArithmeticException => println("Cannot divide by zero")
} finally {
 println("This is always executed")
}
// prints "Cannot divide by zero" and "This is always executed"
```

- **match:** This is a way to pattern match on a value and execute different code depending on the case. The match expression has a value to match on, and a series of cases that specify a pattern and a corresponding value or action. For example:

```
val x = 10
val y = x match {
 case 0 => "zero"
 case 1 => "one"
 case _ => "other" // default case
}
// y is "other"
```

- **function calls:** This is a way to invoke a function with some arguments and get a result. A function can be defined by a name, or by a function literal. For example:

```
def add(a: Int, b: Int): Int = a + b // named function
val mul = (a: Int, b: Int) => a * b // function literal
val x = add(2, 3) // function call
val y = mul(2, 3) // function call
// x is 5, y is 6
```

## Functions and Closures in Scala

- A function is a block of code that takes some input, performs some computation, and returns some output.
- A function can be defined using the `def` keyword, followed by the function name, parameters, return type, and body.
- A function can also be defined as an anonymous function, which is a function without a name, using the `=>` syntax.
- A function can be assigned to a variable, passed as an argument to another function, or returned from a function.
- A closure is a special type of function that can access variables that are defined outside its scope.

- A closure captures the values of the external variables at the time of its creation, and can use them in its body.
- A closure can be useful for creating higher-order functions, such as map, filter, reduce, etc.
- A closure can also be used to create a function factory, which is a function that returns another function based on some parameters.

Here are some examples of functions and closures in Scala:

```
// A function that takes two integers and returns their sum
def add(x: Int, y: Int): Int = {
 x + y
}

// An anonymous function that takes two integers and returns their product
val multiply = (x: Int, y: Int) => x * y

// A function that takes a function and a list of integers and applies the function to each
def map(f: Int => Int, xs: List[Int]): List[Int] = {
 xs match {
 case Nil => Nil
 case head :: tail => f(head) :: map(f, tail)
 }
}

// A closure that takes an integer and returns a function that adds that integer to another
def adder(x: Int): Int => Int = {
 (y: Int) => x + y
}

// Using the functions and closures
val a = add(3, 4) // a = 7
val b = multiply(5, 6) // b = 30
val c = map(add(1, _), List(1, 2, 3)) // c = List(2, 3, 4)
val d = adder(10) // d is a function that adds 10 to its argument
val e = d(5) // e = 15
```

## Inheritance in Scala

- Inheritance is a mechanism that allows a class to inherit the features and behavior of another class.
- The class that inherits is called the **subclass** or the **derived class**.
- The class that is inherited from is called the **superclass** or the **base class**.
- In Scala, a subclass can inherit from a superclass using the **extends** keyword.
- For example, `class Dog extends Animal` means that the class `Dog` is a subclass of the class `Animal`.

- A subclass can access the public and protected members of its superclass, but not the private members.
- A subclass can also override the methods of its superclass using the **override** keyword.
- For example, `override def speak(): Unit = println("Woof")` means that the subclass `Dog` overrides the method `speak` of its superclass `Animal`.
- A subclass can also call the methods of its superclass using the **super** keyword.
- For example, `super.speak()` means that the subclass `Dog` calls the method `speak` of its superclass `Animal`.
- In Scala, a class can inherit from only one superclass, but it can implement multiple **traits**.
- Traits are like interfaces in Java, they define abstract methods and fields that can be mixed in with classes.
- A class can implement a trait using the **with** keyword.
- For example, `class Dog extends Animal with Friendly` means that the class `Dog` inherits from the class `Animal` and implements the trait `Friendly`.
- A class can implement multiple traits using multiple **with** keywords.
- For example, `class Dog extends Animal with Friendly with Furry` means that the class `Dog` inherits from the class `Animal` and implements the traits `Friendly` and `Furry`.
- A trait can also extend another trait or a class using the **extends** keyword.
- For example, `trait Furry extends Animal` means that the trait `Furry` inherits from the class `Animal`.
- A trait can also override the methods of its superclass or another trait using the **override** keyword.
- For example, `override def speak(): Unit = println("Meow")` means that the trait `Furry` overrides the method `speak` of its superclass `Animal`.
- A trait can also call the methods of its superclass or another trait using the **super** keyword.
- For example, `super.speak()` means that the trait `Furry` calls the method `speak` of its superclass `Animal`.
- In Scala, multiple inheritance is achieved by using traits, which can be mixed in with classes at runtime.
- This allows for more flexibility and modularity than single inheritance.

## Hadoop Eco System Frameworks , Pig , Hive and HBase

- Hadoop is an open-source framework that allows distributed processing of large-scale data sets across clusters of computers using simple programming models.
- Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce. HDFS is a distributed file system that provides high-throughput access to data, while MapReduce is a programming model that enables parallel processing of data on HDFS.

- Hadoop also includes several additional modules that provide additional functionality, such as Hive, Pig, and HBase .
- Hive is a data warehouse infrastructure that provides data summarization and ad-hoc querying using a SQL-like query language called Hive Query Language (HQL). Hive allows users to access and analyze data stored in HDFS using a familiar SQL syntax.
- Pig is a high-level data-flow language and execution framework for parallel computation. Pig allows users to write complex data transformations and analysis using a simple scripting language called Pig Latin. Pig translates Pig Latin scripts into MapReduce jobs and runs them on Hadoop.
- HBase is a distributed column-oriented database that supports structured data storage for large tables. HBase is built on top of HDFS and provides fast random access to data. HBase is suitable for applications that require real-time read/write access to big data.

### **Hadoop Eco System Frameworks**

- Hadoop is an open-source software framework that allows for the distributed processing of large data sets across clusters of commodity hardware .
- Hadoop is composed of four core components: Hadoop Distributed File System (HDFS), MapReduce, YARN, and Hadoop Common .
- HDFS is a distributed file system that provides high-throughput access to data and can store any kind of data .
- MapReduce is a programming model that enables parallel processing of large data sets using key-value pairs .
- YARN is a resource management layer that allocates and schedules computing resources for different applications running on Hadoop .
- Hadoop Common is a set of utilities and libraries that support the other Hadoop components .
- Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Hadoop. Some of the popular ones are:
  - Apache Pig: a high-level scripting language that simplifies data analysis and transformation on Hadoop .
  - Apache Hive: a data warehouse system that provides SQL-like query language and schema-on-read for structured and semi-structured data on Hadoop .
  - Apache HBase: a column-oriented database that provides low-latency random access to large-scale data on Hadoop .
  - Apache Spark: a fast and general-purpose engine for large-scale data processing, supporting batch, streaming, SQL, machine learning, and graph analytics on Hadoop .
  - Apache Kafka: a distributed messaging system that enables high-throughput and low-latency data ingestion and integration on Hadoop .

- Apache Flume: a service that collects, aggregates, and moves large amounts of log data from various sources to HDFS .
- Apache Sqoop: a tool that transfers data between Hadoop and relational databases .
- Apache Oozie: a workflow scheduler that coordinates and executes Hadoop jobs .
- Apache ZooKeeper: a service that provides coordination, configuration, and synchronization for distributed applications on Hadoop .
- Apache Mahout: a library that provides scalable machine learning algorithms on Hadoop .

### **Applications of Big Data using Pig**

- Apache Pig is a platform or tool that provides a high-level language called Pig Latin for processing large datasets on top of Hadoop and MapReduce.
- Pig Latin is a scripting language that allows users to write complex data transformations and analysis using simple commands.
- Pig can handle both structured and unstructured data, and can perform various operations such as filtering, grouping, joining, sorting, aggregating, and more.
- Some of the applications of big data using Pig are:
  - Exploring large datasets: Pig can be used to quickly and easily explore large datasets by writing Pig scripts that perform various tasks such as loading, filtering, sampling, and summarizing data .
  - Ad-hoc queries: Pig can also support ad-hoc queries across large datasets by allowing users to write Pig scripts that execute on the fly and return the results in a tabular format .
  - Prototyping of algorithms: Pig can be used to prototype and test large-scale data processing algorithms by writing Pig scripts that implement the logic and run them on a subset of data before deploying them on the full dataset .
  - Time-sensitive data loads: Pig can process time-sensitive data loads such as web logs, click streams, sensor data, and social media data by writing Pig scripts that perform the necessary transformations and analysis on the data in a timely manner.
  - Analytical insights: Pig can provide analytical insights on large datasets by writing Pig scripts that perform various functions such as data cleansing, data integration, data enrichment, data mining, and data visualization.
  - User-defined functions: Pig can also invoke user-defined functions written in other languages such as Java, Python, Ruby, and Groovy, and embed them in Pig scripts to perform custom tasks on the data.

### **Applications of Big Data using Hive**

- Hive is a data warehouse software that facilitates reading, writing, and managing large datasets residing in distributed storage using SQL-like queries.
- Hive is mainly used for big data analytics, as it can process and analyze structured and semi-structured data from various sources, such as web logs, social media, sensor data, etc .
- Some of the common applications of big data using Hive are:
  - Running analytics reports on transaction behavior, activity, volume, and more .
  - Tracking fraudulent activity and generating reports on this activity .
  - Creating dashboards based on the data .
  - Auditing purposes and a store for historical data .
  - Feeding data for machine learning and building intelligence around it .
  - Processing and analyzing trade data of up to 90 billion events using SQL.
- Hive includes many features that make it a useful tool for big data analysis, such as:
  - Support for partitioning, indexing, and user-defined functions (UDFs).
  - A number of optimization techniques to improve query performance, such as predicate pushdown, column pruning, and query parallelization.
  - Compatibility with various data formats, such as JSON, CSV, ORC, Parquet, etc.
  - Integration with other big data frameworks, such as Hadoop, Spark, Pig, etc.
  - Scalability and reliability, as it can handle petabytes of data on distributed clusters .

### **Applications on Big Data using HBase**

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS) .
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases .
- HBase is well suited for real-time data processing or random read/write access to large volumes of data .
- HBase works well with Hive, a query engine for batch processing of big data, to enable fault-tolerant big data applications .
- HBase hosts very large tables on top of clusters of commodity hardware .
- HBase is modeled after Google's Bigtable, which acts upon Google File System .



- Some of the applications of HBase are:
  - In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area .
  - In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights .
  - In sports, HBase is used to store match details and the history of each match .
  - In social media, HBase is used to store user profiles, messages, comments, likes, and other interactions .
  - In finance, HBase is used to store transaction records, stock prices, market trends, and other financial data .

## **Pig**

- Pigs are mammals that belong to the order Artiodactyla, which means they have an even number of toes on each foot.
- Pigs are also known as hogs or swine. Male pigs of any age are called boars; female pigs are called sows.
- Pigs are found and raised all over the world, and provide valuable products to humans, including pork, lard, leather, glue, fertilizer, and a variety of medicines.
- Pigs have the intelligence of a human toddler and are ranked as the fifth most intelligent animal in the world. They can learn their names, come when they are called, and play video games better than some primates.
- Pigs have a keen sense of smell and can detect odors up to 7 miles away and 20 feet underground. They use their snouts for digging, rooting, and communicating.
- Pigs do not have many sweat glands, so they cool themselves by wallowing in mud or water. This also helps them to protect their skin from parasites and sunburn.
- Pigs are social animals that form complex hierarchies and bonds with each other. They enjoy playing, cuddling, and grooming each other.
- Pigs are omnivorous and will eat almost anything, including plants, insects, worms, eggs, carrion, and feces. They can also be trained to eat a specific diet.
- Pigs have four toes on each foot, but only use the middle two for walking. These are called trotters.
- Pigs can live up to 20 years in captivity, but usually have a much shorter lifespan in the wild or on farms. The average lifespan of a domestic pig is 6 to 10 years.

## **Pig - Introduction to PIG**

- Pig is a high-level scripting language that allows users to write complex data analysis programs for large-scale data sets.

- Pig is built on top of Hadoop, an open-source framework for distributed storage and processing of big data.
- Pig programs are translated into a series of MapReduce jobs that run on the Hadoop cluster.
- Pig provides a simple and expressive syntax that is similar to SQL, but also supports user-defined functions, complex data types, and nested data structures.
- Pig can handle both structured and unstructured data, and can perform various operations such as filtering, grouping, joining, sorting, aggregating, and transforming data.
- Pig can also integrate with other languages and tools, such as Python, Java, R, and Hive, to extend its functionality and interoperability.
- Pig is designed to be scalable, flexible, and extensible, and to support parallel execution and optimization of data flows.

### Execution Modes of Pig

Pig is a high-level data processing language that runs on top of Hadoop. Pig can execute its programs in different modes depending on the environment and the data source. Pig has two types of execution modes: local mode and MapReduce mode. Pig also has three mechanisms of execution: interactive mode, batch mode, and embedded mode. Here are some points to understand the execution modes and mechanisms of Pig:

- Local mode: In this mode, Pig runs on a single machine using the local file system. No Hadoop installation is required. This mode is useful for testing and debugging purposes. To run Pig in local mode, use the `-x local` option in the command line or the `pig -x local` command in the Grunt shell .
- MapReduce mode: In this mode, Pig runs on a Hadoop cluster using the Hadoop Distributed File System (HDFS). This mode is suitable for large-scale data processing. To run Pig in MapReduce mode, use the `-x mapreduce` option in the command line or the `pig -x mapreduce` command in the Grunt shell .
- Interactive mode: In this mode, Pig is executed in the Grunt shell, which is an interactive command-line interface. In this shell, you can enter Pig Latin statements and see the results immediately. You can also load and run Pig scripts from the shell. To invoke the Grunt shell, use the `pig` command without any options .
- Batch mode: In this mode, Pig runs a script file that contains Pig Latin commands. The script file has a `.pig` extension and can be executed from the command line or the Grunt shell. This mode is useful for running predefined workflows and pipelines. To run a Pig script in batch mode, use the `-f` option followed by the script name in the command line or the `exec` command followed by the script name in the Grunt shell .
- Embedded mode: In this mode, Pig can be embedded in a Java program

as a user-defined function (UDF). This mode allows you to extend the functionality of Pig and use custom logic and data types. To run Pig in embedded mode, you need to use the PigServer class and the Java API .

### Comparison of Pig with Databases

- Pig is a high-level data-flow language and execution framework for parallel computation on large-scale data sets, while databases are systems that store, organize, and manipulate structured or semi-structured data using a query language, such as SQL.
- Pig can process unstructured or complex data formats, such as JSON, XML, or images, while databases usually require a predefined schema or data model to store and query data.
- Pig can run on top of Hadoop, a distributed file system and a platform for MapReduce programming, which enables parallel processing and scalability, while databases may run on a single server or a cluster of servers, depending on the type and architecture of the database.
- Pig can be used to construct data flows, run scheduled jobs, crunch big volumes of data, aggregate or summarize it, and store it into relational database systems or other data sources, while databases can be used to perform CRUD (create, read, update, delete) operations, join tables, perform aggregations, and support transactions and concurrency control.
- Pig uses Pig Latin, a scripting language that is similar to SQL but more expressive and flexible, while databases use SQL or its variants, such as HiveQL, which are declarative and standardized languages for data manipulation and analysis.
- Pig can be extended using user-defined functions (UDFs) written in Java, Python, JavaScript, Ruby, or Groovy, while databases may support user-defined functions or stored procedures written in SQL or other languages, depending on the database system.
- Pig has better performance and efficiency than databases for processing large and complex data sets, while databases have better performance and efficiency than Pig for processing small and simple data sets.

### Grunt in Pig

- Grunt is a vocalization made by pigs to communicate with each other.
- Grunt can convey information about the pig's emotional state, personality, and environment.
- Grunt can vary in frequency, duration, intensity, and pitch depending on the context and the individual pig.
- Some examples of grunt meanings are:
  - A low-pitched and long grunt indicates contentment and relaxation.
  - A high-pitched and short grunt indicates excitement and anticipation.

- A fast and rhythmic grunt indicates proactivity and curiosity.
- A slow and irregular grunt indicates reactivity and caution.
- A loud and harsh grunt indicates aggression and dominance.
- A soft and gentle grunt indicates submission and appeasement.
- Grunt can also be used to coordinate social behaviors, such as feeding, mating, and nursing.
- Grunt can be influenced by the pig’s environment, such as the availability of food, water, space, and enrichment.
- Grunt can be analyzed using acoustic recordings and software tools to measure the acoustic features and patterns of the vocalizations.
- Grunt can be used as an indicator of pig welfare and well-being, as well as a tool for improving pig management and husbandry.

### Pig Latin

- Pig Latin is a form of word play that involves altering the spelling and pronunciation of English words.
- The basic rule of Pig Latin is to move the first consonant or consonant cluster of a word to the end and add “ay”. For example, “pig” becomes “igpay”, “latin” becomes “atinlay”, and “smile” becomes “ilesmay”.
- If a word begins with a vowel, the rule is to add “way” to the end. For example, “apple” becomes “appleway”, “egg” becomes “eggway”, and “orange” becomes “orangeway”.
- Some variations of Pig Latin may have different rules, such as adding “yay” instead of “way” for words that begin with vowels, or adding “hay” to words that begin with “h”.
- Pig Latin is not a real language, but a form of code or slang that is used for fun or secrecy. It is mostly used by children, but some adults may also use it for humor or entertainment.
- Pig Latin has no standard spelling or grammar, and different speakers may pronounce or write it differently. However, some common features of Pig Latin are:
  - The stress of the original word is usually preserved in the Pig Latin word. For example, “banana” becomes “ananabay”, not “anabana”.
  - The letter “y” is treated as a consonant if it comes at the beginning of a word, and as a vowel otherwise. For example, “yellow” becomes “ellowyay”, but “happy” becomes “appyhay”.
  - The letter “qu” is treated as a single consonant and moved together. For example, “queen” becomes “eenquay”, and “squeak” becomes “eaksquay”.
  - Some words may have more than one possible Pig Latin form, depending on how the speaker divides the consonant clusters. For example, “school” could be “oolschay” or “oolschay”, and “string” could be “ingstray” or “ingstray”.

## User Defined Functions in Pig

- User defined functions (UDFs) are custom functions that can be written in Java, Python, Ruby, or Groovy and used in Pig scripts to perform specific tasks that are not supported by the built-in functions.
- UDFs can be used to manipulate data, perform complex calculations, call external services, or interact with other systems.
- UDFs can be classified into four types: Eval, Filter, Load, and Store.
- Eval functions take one or more input values and return a single output value. They can be used in expressions, projections, or as arguments to other functions. For example, UPPER is an eval function that converts a string to uppercase.
- Filter functions take a single input value and return a boolean value. They can be used in the FILTER operator to filter out records that do not satisfy a condition. For example, IsEmpty is a filter function that checks if a bag or a map is empty.
- Load functions take a file name or a URI as an input and return a bag of tuples. They can be used in the LOAD operator to read data from various sources, such as files, databases, or web services. For example, PigStorage is a load function that reads data from files in a specified format.
- Store functions take a bag of tuples as an input and a file name or a URI as an output. They can be used in the STORE operator to write data to various destinations, such as files, databases, or web services. For example, PigStorage is a store function that writes data to files in a specified format.
- To use a UDF in a Pig script, the UDF must be registered using the REGISTER statement, which specifies the name and location of the UDF jar file or script file. For example, REGISTER myudfs.jar; or REGISTER myudfs.py USING jython as myfuncs;
- To invoke a UDF in a Pig script, the UDF name must be prefixed with the namespace, which is either the name of the jar file or the alias given to the script file. For example, myudfs.UPPER(\$0) or myfuncs.myfunc(\$0, \$1);
- UDFs can also be defined inline using the DEFINE statement, which specifies the name and the class name of the UDF. For example, DEFINE MyUpper myudfs.UPPER; or DEFINE MyFunc myfuncs.myfunc;
- UDFs can also be tested using the ILLUSTRATE operator, which shows an example of how the UDF works on a sample of the input data. For example, ILLUSTRATE MyUpper; or ILLUSTRATE MyFunc;

## Data Processing Operators in Pig

- Data processing operators are the main tools that Pig Latin provides to operate on the data. They allow you to transform it by sorting, grouping, joining, projecting, and filtering.
- A Pig Latin statement is an operator that takes a relation as input and produces another relation as output.

- There are different types of data processing operators in Pig, such as:
  - Relational operators: These operators perform basic operations on relations, such as loading, storing, filtering, grouping, joining, etc. For example, LOAD, STORE, FILTER, GROUP, JOIN, etc .
  - Evaluation operators: These operators perform various calculations on the data, such as arithmetic, string, date, etc. For example, +, -, \*, /, CONCAT, SUBSTRING, ToDate, etc.
  - Diagnostic operators: These operators help in debugging and testing the Pig scripts, such as printing the schema, the data, or the execution plan. For example, DESCRIBE, DUMP, EXPLAIN, etc.
  - Miscellaneous operators: These operators perform some additional functions on the data, such as ordering, limiting, splitting, etc. For example, ORDER BY, LIMIT, SPLIT, etc.

## Hive

- Hive is a data warehouse software that facilitates querying and managing large datasets residing in distributed storage.
- Hive provides a SQL-like interface called HiveQL that allows users to perform data analysis on structured and semi-structured data .
- Hive can also be used to create, modify, and delete tables and partitions, as well as perform other data management tasks.
- Hive supports various storage formats, such as text, ORC, Parquet, JSON, and Avro, and can integrate with external data sources, such as HBase, Cassandra, and MongoDB.
- Hive can run on top of different execution engines, such as MapReduce, Tez, and Spark, to optimize the performance of the queries.
- Hive can also be accessed through various tools and applications, such as Hive CLI, HiveServer2, Beeline, JDBC, ODBC, and Hive Web Interface.
- Hive is widely used for data warehousing, business intelligence, and analytics applications, especially in big data environments .
- Hive is part of the Apache Hadoop ecosystem and can work with other Hadoop components, such as HDFS, YARN, and ZooKeeper.
- Hive can also leverage cloud storage services, such as Amazon S3, Azure Data Lake Storage, and Google Cloud Storage, to store and access data.
- Hive is an open source software that is maintained by the Apache Software Foundation and has a large and active community of contributors and users.

: <https://learn.microsoft.com/en-us/azure/hdinsight/hadoop/hdinsight-use-hive>  
<https://hive.apache.org/>

## Apache Hive architecture

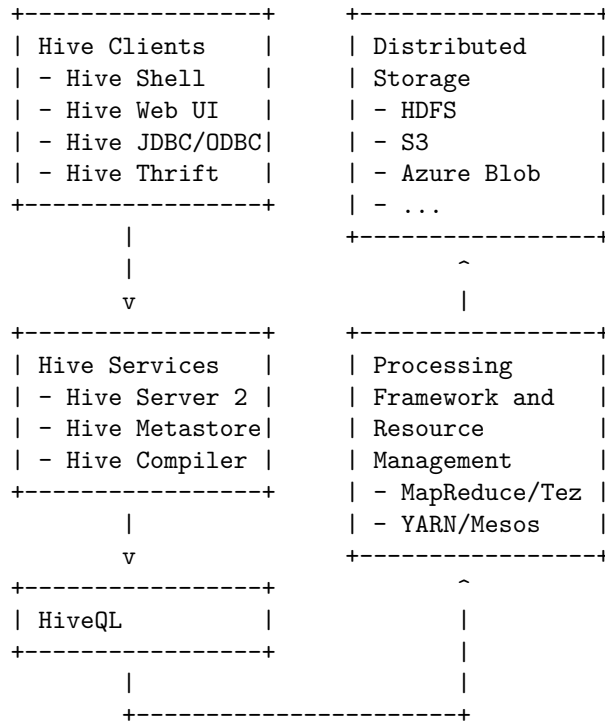
Apache Hive is a data warehouse system that enables analytics at a massive scale on top of Hadoop. It provides a SQL-like query language called HiveQL

that can process structured and semi-structured data. Hive also supports user-defined functions and custom data formats.

The main components of Apache Hive architecture are:

- **Hive Clients:** These are the interfaces that allow users and applications to interact with Hive. They include the Hive Shell, the Hive Web Interface, the Hive JDBC and ODBC drivers, and the Hive Thrift API.
- **Hive Services:** These are the components that provide the core functionality of Hive, such as parsing, compiling, optimizing, and executing queries. They include the Hive Server 2, the Hive Metastore, and the Hive Compiler.
- **Processing Framework and Resource Management:** These are the components that handle the distributed processing and resource allocation of Hive queries. They include the MapReduce or Tez execution engine, and the YARN or Mesos resource manager.
- **Distributed Storage:** This is the component that stores the data and metadata of Hive tables and partitions. It includes the Hadoop Distributed File System (HDFS) or other compatible file systems.

The following diagram illustrates the Apache Hive architecture:



## Installing Hive

- Hive is a data warehouse system that runs on top of Hadoop, a distributed file system that stores large amounts of data.
- Hive provides a SQL-like interface to query and analyze data stored in Hadoop.
- To install Hive, you need to have Java, Hadoop, and Hive binaries installed on your system.
- The steps to install Hive are as follows:
  1. Download the latest version of Hive from <https://hive.apache.org/downloads.html> and extract it to a desired location.
  2. Set the environment variables `HIVE_HOME` and `PATH` to point to the Hive installation directory and its bin subdirectory, respectively.
  3. Create a directory for Hive metastore, which stores the metadata of the tables and partitions in Hive. By default, Hive uses Derby as the metastore database, but you can configure other databases such as MySQL or PostgreSQL.
  4. Initialize the metastore schema by running the command `schematool -initSchema -dbType derby` from the Hive bin directory. You can replace `derby` with the database type you are using for the metastore.
  5. Start the Hive shell by running the command `hive` from the Hive bin directory. You can now execute Hive queries and commands from the shell.

## Hive shell

- Hive shell is a command-line interface that allows users to interact with Hive and execute HiveQL queries.
- Hive shell can be run in two modes: interactive and non-interactive.
- In interactive mode, users can enter HiveQL commands directly in the shell and see the results on the screen. To start the interactive mode, users need to type `hive` in the terminal and press enter.
- In non-interactive mode, users can specify a file that contains HiveQL commands and run them in batch mode. To start the non-interactive mode, users need to type `hive -f filename` in the terminal and press enter, where `filename` is the name of the file that contains the HiveQL commands.
- Hive shell supports various options and commands that can be used to configure and control the shell behavior. For example, users can use `set` to set or view the value of a Hive configuration variable, or use `quit` to exit the shell. To see the full list of options and commands, users can type `help` in the shell and press enter.

## Hive services

Hive services are the components that perform client interactions with Hive.



They allow users to submit queries and commands to Hive and receive the results. Some of the main Hive services are:

- **HiveServer2:** This is the main service that provides a JDBC/ODBC interface for clients to connect to Hive and execute queries. It also supports authentication, authorization, and encryption. HiveServer2 can run in different modes, such as embedded, local, or remote.
- **Beeline:** This is a command-line shell that connects to HiveServer2 and allows users to submit queries and commands to Hive. It is based on the SQLLine tool and supports multiple sessions and output formats.
- **Hive Web Interface (HWI):** This is a web-based graphical user interface that allows users to browse the Hive metadata, execute queries, and view the results. It is deprecated in Hive 0.14.0 and replaced by HiveServer2 web UI.
- **Hive Thrift Server:** This is an older service that provides a Thrift interface for clients to connect to Hive and execute queries. It is deprecated in Hive 0.13.0 and replaced by HiveServer2.
- **Hive CLI:** This is a command-line interface that allows users to interact with Hive directly without using HiveServer2. It is mainly used for debugging and testing purposes. It is deprecated in Hive 2.0.0 and replaced by Beeline.
- **Hive Metastore:** This is a service that stores the metadata of the tables, partitions, columns, and schemas in Hive. It can use different back-end databases, such as Derby, MySQL, PostgreSQL, or Oracle. It also provides a Thrift API for other Hive services and clients to access the metadata.

### Comparison of Hive with traditional databases

Hive is a data warehouse software system that provides data query and analysis on large datasets stored in Hadoop. Hive supports a SQL-like interface called HiveQL, which allows users to perform various operations on the data such as creating tables, loading data, querying, aggregating, joining, etc. Hive is not a full-fledged database, but rather a data warehouse that applies schema on read time, meaning that the data is not validated or transformed until it is queried.

Traditional databases, such as MySQL, PostgreSQL, Oracle, MS SQL Server, etc., are relational database management systems (RDBMS) that store data in structured tables with predefined schemas. Traditional databases enforce schema on write time, meaning that the data is validated and transformed before it is stored in the database. Traditional databases support SQL as the standard query language, and also provide various features such as transactions, indexes, triggers, views, etc.

Some of the main differences between Hive and traditional databases are:

- Hive is designed for batch processing and analytical queries on large datasets, while traditional databases are designed for online transaction processing (OLTP) and operational queries on smaller datasets.

- Hive does not support record-level updates, insertions, and deletions, while traditional databases support these operations using SQL commands such as UPDATE, INSERT, and DELETE.
- Hive does not support transactions, concurrency control, or ACID properties, while traditional databases support these features to ensure data consistency and integrity.
- Hive does not support indexes, triggers, views, or stored procedures, while traditional databases support these features to improve query performance and functionality.
- Hive is very easily scalable at low cost, as it can run on commodity hardware and leverage the distributed storage and processing capabilities of Hadoop. Traditional databases are not very scalable, as they require expensive hardware and complex architectures to scale up or out.
- Hive is based on the Hadoop notion of write once, read many (WORM), meaning that the data is loaded into Hive once and then queried multiple times. Traditional databases support read and write operations on the data at any time.

**References:** <https://sensaran.wordpress.com/2016/01/30/comparison-with-hive-with-traditional-database/>

<https://www.tutorialslink.com/Articles/A-Comparative-of-Traditional-RDBMS-and-HiveQL-in-Hadoop-Environment/1266>

<http://hadooptutorial.info/hive-vs-rdbms>

<https://www.geeksforgeeks.org/difference-between-rdbms-and-hive>

<https://www.quora.com/How-is-the-Apache-Hive-different-from-the-traditional-database>

## HiveQL

- HiveQL is a query language for Apache Hive, a data warehouse system for Apache Hadoop .
- HiveQL allows users to process and analyze structured data in a Metastore, which is a central repository of metadata .
- HiveQL separates users from the complexity of Map Reduce programming, and reuses common concepts from relational databases, such as tables, rows, columns, and schema .
- HiveQL provides the basic SQL-like operations, such as SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, ORDER BY, etc .
- HiveQL also provides built-in operators and functions for data operations, such as arithmetic, logical, relational, string, date, and aggregate operators and functions .
- HiveQL supports storage on various file systems, such as HDFS, S3, ADLS, GS, etc.

- HiveQL aids portability of SQL-based applications to Hadoop, and enables data summarization, querying, and analysis of data .

## Tables in Hive

- Tables in Hive are similar to tables in a relational database management system. They store data in columns and rows and belong to a database.
- Tables in Hive can be created using the `CREATE TABLE` statement with the following syntax:

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
[(col_name data_type [COMMENT col_comment], ...)]
[COMMENT table_comment]
[PARTITIONED BY (col_name data_type [COMMENT col_comment], ...)]
[CLUSTERED BY (col_name, col_name, ...) [SORTED BY (col_name [ASC|DESC], ...)] INTO num_buckets]
[STORED AS file_format]
[LOCATION hdfs_path]
[TBLPROPERTIES (property_name=property_value, ...)]
```

- Tables in Hive can be classified into two types: internal and external.
  - Internal tables: Data is stored in the Hive data warehouse, which is located at `/hive/warehouse/` on the default storage for the cluster. Internal tables are also called managed tables, as Hive manages their data and metadata. When an internal table is dropped, both the data and metadata are deleted. Use internal tables when the data is temporary or exclusive to Hive.
  - External tables: Data is stored outside the Hive data warehouse, on any storage accessible by the cluster. External tables are also called unmanaged tables, as Hive only manages their metadata. When an external table is dropped, only the metadata is deleted, while the data remains intact. Use external tables when the data is shared by other applications or needs to be preserved after dropping the table.
- Tables in Hive can also be partitioned, clustered, and sorted to improve query performance and data organization.
  - Partitioning: Partitioning divides a table into multiple sub-tables based on one or more columns. Each sub-table is stored in a separate directory on the file system. Partitioning allows Hive to scan only the relevant sub-tables for a query, reducing the amount of data to be processed.
  - Clustering: Clustering distributes the data in a table or a partition into multiple files or buckets based on one or more columns. Each bucket is assigned a hash value and stored in a separate file. Clustering allows Hive to perform efficient joins and sampling on the table

or partition.

- Sorting: Sorting orders the data in a table or a partition or a bucket based on one or more columns. Sorting can be done in ascending or descending order. Sorting allows Hive to perform faster queries with filters or range conditions on the sorted columns.

### Querying data in Hive

- Hive is a data warehouse system that allows users to query and analyze large-scale data using a SQL-like language called HiveQL.
- HiveQL is a declarative language that abstracts the complexity of MapReduce, the underlying framework for distributed data processing in Hadoop.
- Hive supports various data formats, such as text, JSON, ORC, Parquet, Avro, etc., and can access data stored in HDFS, HBase, or other external sources.
- Hive provides a schema-on-read approach, which means that the data schema is inferred at the time of query execution, rather than at the time of data ingestion.
- Hive allows users to create tables, partitions, buckets, views, indexes, and functions to organize and manipulate data.
- Hive also supports user-defined functions (UDFs), user-defined aggregate functions (UDAFs), and user-defined table functions (UDTFs) to extend the functionality of HiveQL.
- To query data in Hive, users need to use the Hive shell, the Hive web interface (HWI), or the HiveServer2 (HS2) service, which provides a JDBC/ODBC interface for external applications.
- A typical Hive query consists of the following components:
  - A SELECT clause that specifies the columns or expressions to be returned.
  - A FROM clause that specifies the table or view to be queried.
  - A WHERE clause that specifies the filtering conditions for the rows to be returned.
  - A GROUP BY clause that specifies the grouping criteria for the rows to be aggregated.
  - A HAVING clause that specifies the filtering conditions for the groups to be returned.
  - An ORDER BY clause that specifies the sorting order for the rows to be returned.
  - A LIMIT clause that specifies the maximum number of rows to be returned.
- Hive also supports subqueries, joins, unions, set operations, window functions, analytical functions, and common table expressions (CTEs) to perform complex queries on data.
- Hive provides various built-in functions and operators to perform arithmetic, logical, string, date, and other operations on data.

- Hive also supports various query optimization techniques, such as predicate pushdown, column pruning, partition pruning, map-side join, bucket map join, sort-merge join, etc., to improve the query performance and reduce the resource consumption.
- Hive also supports various query execution modes, such as local mode, mapreduce mode, and tez mode, to choose the appropriate execution engine for the query.

### User Defined Functions in Hive

- User defined functions (UDFs) are custom functions that can be developed in Java, integrated with Hive, and built on top of a Hadoop cluster to allow for efficient and complex computation that would not otherwise be possible with simple SQL.
- UDFs can be useful and very powerful, and yet online documentation is pretty weak.
- UDFs can be classified into three types: simple, generic, and aggregate.
- Simple UDFs are the most common type of UDFs. They take one or more primitive types as input and return a single primitive type as output.
- Generic UDFs are more flexible than simple UDFs. They can take complex types such as arrays, maps, or structs as input or output, and they can also handle null values.
- Aggregate UDFs are also known as user defined aggregate functions (UDAFs). They take a set of values as input and return a single value as output, such as sum, count, or average.
- To create a UDF, one needs to write a Java class that extends the appropriate interface or abstract class depending on the type of UDF.
- For simple UDFs, the interface is `org.apache.hadoop.hive.ql.exec.UDF`.
- For generic UDFs, the abstract class is `org.apache.hadoop.hive.ql.udf.generic.GenericUDF`.
- For aggregate UDFs, the abstract class is `org.apache.hadoop.hive.ql.udf.generic.GenericUDAFEvaluator`.
- The Java class must implement the `evaluate` method that defines the logic of the UDF.
- The Java class must also be packaged into a JAR file and registered in Hive using the `CREATE FUNCTION` command.
- To use a UDF in a Hive query, one can simply invoke it by its name and pass the arguments as required.
- To check which UDFs are loaded in the current Hive session, one can use the `SHOW FUNCTIONS` command.
- To drop a UDF from the current Hive session, one can use the `DROP FUNCTION` command.
- To list the details of a UDF, such as its class name, type, and usage, one can use the `DESCRIBE FUNCTION` command.
- Some examples of built-in UDFs in Hive are `concat`, `substr`, `upper`, `lower`, `trim`, `length`, `round`, `floor`, `ceil`, `abs`, `sin`, `cos`, `tan`, `sqrt`, `log`, `exp`, etc.
- Some examples of custom UDFs in Hive are `decode`, `date_add`, `date_sub`, `datediff`, etc.

## Sorting and Aggregating in Hive

- Sorting and aggregating are common operations in data analysis that involve rearranging or summarizing data based on some criteria.
- Sorting is the process of ordering data in ascending or descending order based on one or more columns or expressions.
- Aggregating is the process of grouping data by one or more columns or expressions and applying some functions to calculate summary statistics for each group, such as count, sum, average, etc.
- Hive supports both sorting and aggregating operations using SQL-like syntax and built-in functions.
- Sorting can be done using the `ORDER BY` or `SORT BY` clauses in a `SELECT` statement.
- `ORDER BY` sorts the entire result set globally and returns a single ordered output file. It is useful when the output needs to be in a specific order, but it can be expensive in terms of performance and memory usage.
- `SORT BY` sorts the data locally within each reducer partition and returns multiple output files. It is useful when the output does not need to be globally ordered, but only within each partition, and it can be faster and more efficient than `ORDER BY`.
- Aggregating can be done using the `GROUP BY` or `CUBE` or `ROLLUP` clauses in a `SELECT` statement, along with some aggregate functions such as `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`, etc.
- `GROUP BY` groups the data by the specified columns or expressions and applies the aggregate functions to each group. It returns one row per group with the aggregated values.
- `CUBE` and `ROLLUP` are extensions of `GROUP BY` that allow generating subtotals and grand totals along multiple dimensions. They return multiple rows per group with different levels of aggregation.
- `CUBE` generates all possible combinations of the grouping columns or expressions, including the empty set, which represents the grand total.
- `ROLLUP` generates a subset of the combinations of the grouping columns or expressions, starting from the most detailed level and ending with the grand total. It follows a hierarchical order of the grouping columns or expressions.

## Joins and subqueries in Hive

- Joins are used to combine data from two or more tables based on a common column or condition.
- Subqueries are used to create temporary tables that can be used in the main query or in another subquery.
- Hive supports different types of joins, such as inner join, left outer join, right outer join, full outer join, cross join, and semi join.
- Hive supports subqueries only in the `FROM` clause, and the subquery has to be given a name or alias.

- Hive supports arbitrary levels of subqueries, and the subquery can also be a query expression with UNION.
- Joins and subqueries can be used together to perform complex queries on Hive tables.

Some examples of joins and subqueries in Hive are:

- Inner join: This join returns the records that are common to both tables based on the join condition.

```
SELECT a.col1, b.col2 FROM table1 a JOIN table2 b ON a.id = b.id;
```

- Left outer join: This join returns all the records from the left table and the matching records from the right table. If there is no match, the right table columns are filled with NULL.

```
SELECT a.col1, b.col2 FROM table1 a LEFT OUTER JOIN table2 b ON a.id = b.id;
```

- Right outer join: This join returns all the records from the right table and the matching records from the left table. If there is no match, the left table columns are filled with NULL.

```
SELECT a.col1, b.col2 FROM table1 a RIGHT OUTER JOIN table2 b ON a.id = b.id;
```

- Full outer join: This join returns all the records from both tables, and fills the missing values with NULL.

```
SELECT a.col1, b.col2 FROM table1 a FULL OUTER JOIN table2 b ON a.id = b.id;
```

- Cross join: This join returns the Cartesian product of the two tables, i.e., every row of the left table is combined with every row of the right table.

```
SELECT a.col1, b.col2 FROM table1 a CROSS JOIN table2 b;
```

- Semi join: This join returns the records from the left table that have a match in the right table, but does not return any columns from the right table.

```
SELECT a.col1 FROM table1 a WHERE a.id IN (SELECT b.id FROM table2 b);
```

- Subquery: This query creates a temporary table that can be used in the main query or in another subquery. The subquery has to be given a name or alias.

```
SELECT a.col1, b.col2 FROM (SELECT id, col1 FROM table1 WHERE col1 > 10) a JOIN (SELECT id,
```

- Subquery with UNION: This query combines the results of two or more subqueries using the UNION operator.

```
SELECT col1, col2 FROM (SELECT col1, col2 FROM table1 UNION SELECT col1, col2 FROM table2) a
```

## HBase

- HBase is a **column-oriented non-relational database management system** that runs on top of **Hadoop Distributed File System (HDFS)** .
- HBase provides a **fault-tolerant** way of storing **sparse data sets**, which are common in many big data use cases .
- HBase is well suited for **real-time data processing** or **random read/write access** to large volumes of data .
- HBase is modeled after **Google's Bigtable**, a distributed storage system for structured data .
- HBase does not have a **fixed database schema** in a non-relational database, which means developers can add new data without conforming to a schema model.
- HBase is ideal for **high-scale real-time applications**, such as a social media app or a streaming application.
- HBase is an **open-source** project developed as part of **Apache Software Foundation's Apache Hadoop project** .
- HBase is written in **Java**.

## HBase concepts

- HBase is a type of NoSQL database and is classified as a key-value store.
- HBase is a column-oriented database that runs on top of the Hadoop Distributed File System (HDFS) .
- HBase is an open-source project and is horizontally scalable.
- HBase is a data model that is similar to Google's Bigtable designed to provide quick random access to huge amounts of structured data.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.
- HBase has a master-slave architecture, where a master node manages the cluster and region servers store portions of the tables and perform the work on the data.
- HBase table schema defines only column families, and each family can have unlimited columns.
- HBase table contains multiple families, and each family can have multiple versions of the same column.
- HBase table values are identified with a key, and both key and values are byte array, which means binary formats can be stored easily.
- HBase table values are stored in key-orders, and values can be quickly accessed by their keys.

## HBase clients

- HBase clients are applications or libraries that can interact with HBase using its API or other protocols.



- HBase clients can perform various operations on HBase tables, such as creating, deleting, reading, writing, scanning, and modifying data.
- HBase clients can be written in different programming languages, such as Java, Python, Ruby, Scala, C#, and PHP.
- HBase clients can use different methods to connect to HBase, such as the HBase shell, the Thrift server, the REST server, or the native Java client.
- HBase clients can use different configuration options to tune their performance, security, and reliability, such as client-side buffering, retries, timeouts, authentication, and encryption.
- HBase clients can use different tools and frameworks to simplify their development and testing, such as the HBase Testing Utility, the HBase MiniCluster, and the HBase Integration Testing Framework.

Some examples of HBase clients are:

- The HBase shell: a command-line tool that performs administrative tasks, such as creating and deleting tables.
- The Cloud Bigtable HBase client for Java: a client library that makes it possible to use the HBase shell to connect to Bigtable, a Google Cloud service that offers a fully managed, scalable, and secure version of HBase.
- HappyBase: a Python library that provides a friendly and easy-to-use wrapper around the Thrift server, which exposes a subset of the HBase API using a cross-language service definition .
- HBase-Ruby: a Ruby library that also uses the Thrift server to communicate with HBase, and provides a DSL (domain-specific language) for manipulating HBase tables and data .
- HBase.Net: a C# library that uses the REST server, which exposes a RESTful web service interface to HBase, and supports basic CRUD (create, read, update, delete) operations on HBase tables and data .
- PHPHBase: a PHP library that also uses the REST server to access HBase, and provides a simple and lightweight API for HBase operations .
- Scala-HBase: a Scala library that uses the native Java client, which is the most feature-rich and performant way of interacting with HBase, and provides a Scala-friendly API for HBase operations .

: <https://cloud.google.com/bigtable/docs/installing-hbase-shell>  
<https://www.oreilly.com/library/view/hbase-essentials/9781783987245/ch06.html>  
<https://subscription.packtpub.com/book/big-data-and-business-intelligence/9781783987245/6>

## HBase example

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS).
- HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases.
- HBase is well suited for real-time data processing or random read/write access to large volumes of data.

- HBase stores data in tables, which consist of rows and columns. Each row has a unique identifier called a row key, and each column belongs to a column family.
- HBase supports CRUD (create, read, update, delete) operations, as well as scan and filter operations, on the tables using a shell command interface or a Java API.
- HBase also supports secondary indexes, coprocessors, replication, snapshots, and security features.

Some examples of HBase use cases are:

- In the healthcare sector, HBase is used for storing genome sequences and disease history of people or a particular area.
- In the field of e-commerce, HBase is used for storing logs about customer search history and it also performs analytics and target advertisement for better business insights.
- In sports, HBase is used to store match details and the history of each match.

An example of HBase table creation is:

- To create a table in HBase with the name 'education' and the column family 'guru99', the shell command is:

```
hbase (main):001:0> create 'education','guru99'
```

- The output will be:

```
0 rows (s) in 0.312 seconds =>Hbase::Table - education
```

- This means that the table 'education' with the column family 'guru99' has been created successfully.

## HBase vs RDBMS

- HBase and RDBMS are both types of database management systems, but they differ in several ways.
- RDBMS stands for Relational Database Management System, and it uses a relational data model, where data is stored in tables with predefined columns and rows. HBase stands for Hadoop Database, and it uses a column-family data model, where data is stored in column families, which contain columns and rows. HBase is often referred to as a NoSQL database because of its non-relational data model .
- RDBMS is more suitable for traditional, transactional applications that require strong consistency, ACID compliance, and structured data. HBase is better suited for big data applications that require horizontal scaling, high-speed processing, and semi-structured or unstructured data .
- Some of the main differences between HBase and RDBMS are:

Table 1: <https://www.geeksforgeeks.org/difference-between-rdbms-and-hbase/> : <https://mindmajix.com/hadoop/difference-between-hbase-rdbms> : <https://data-flair.training/blogs/hbase-vs-rdbms/>

| Feature               | HBase                                         | RDBMS                      |
|-----------------------|-----------------------------------------------|----------------------------|
| Data model            | Column-family                                 | Relational                 |
| Schema                | Schemaless                                    | Fixed                      |
| Data size             | Petabytes                                     | Gigabytes or terabytes     |
| Scalability           | Horizontal                                    | Vertical                   |
| Consistency           | Eventual                                      | Strong                     |
| Speed                 | Fast reads and writes                         | Fast reads and slow writes |
| ACID compliance       | Partial                                       | Full                       |
| Transactions          | Single-row                                    | Multi-row                  |
| Joins                 | Not supported                                 | Supported                  |
| Referential integrity | Not enforced                                  | Enforced                   |
| Indexing              | Built-in                                      | Optional                   |
| Query language        | HBase shell, Java API, REST API, or MapReduce | SQL                        |

### Advanced usage of HBase

HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). It provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

Some of the advanced usage of HBase are:

- **Storing genome sequences and running MapReduce on them.** HBase can be used to store and analyze genome data, such as DNA sequences, gene expressions, and mutations. HBase can handle the large and complex data sets that are generated by genome sequencing technologies, and provide fast and scalable access to them. HBase can also integrate with MapReduce frameworks, such as Apache Spark, to perform parallel processing and analytics on the genome data.
- **Storing logs and performing analytics and target advertisement.** HBase can be used to store and process log data, such as web server logs, application logs, and user activity logs. HBase can handle the high volume and velocity of log data, and provide real-time query and analysis capabilities. HBase can also be used to perform analytics and target advertisement based on the log data, such as customer search history, preferences, and behavior .
- **Storing match details and history of sports events.** HBase can be used to store and retrieve match details and history of sports events, such as scores, statistics, players, and teams. HBase can handle the dynamic

and heterogeneous nature of sports data, and provide fast and consistent access to them. HBase can also be used to perform analytics and predictions based on the sports data, such as performance, trends, and outcomes.

- **Using row key and column key to convey meaning and sorting order.** HBase has two fundamental key structures: the row key and the column key. Both can be used to convey meaning, by either the data they store, or by exploiting their sorting order. For example, the row key can be used to store a composite key, such as `user_id:timestamp`, to uniquely identify a record and sort it by time. The column key can be used to store a qualifier, such as name, age, or gender, to store different attributes of a record and sort them lexicographically.
- **Using filters, coprocessors, and custom comparators to enhance functionality.** HBase provides various features to enhance the functionality and performance of the database. Filters can be used to apply conditions and criteria to the data retrieval, such as row filter, column filter, value filter, and prefix filter. Coprocessors can be used to execute custom logic on the server side, such as triggers, aggregations, and transactions. Custom comparators can be used to define custom sorting order and comparison logic for the data, such as reverse order, numeric order, or case-insensitive order.

### Schema design in HBase

- Schema design in HBase is very different from schema design in a relational database system (RDBMS).
- HBase is a NoSQL database that stores data in a tabular format, where each row has a unique row key and each column belongs to a column family.
- HBase schema design requires careful consideration of the following aspects:
  - Row key: The row key is the primary identifier of a row and determines the physical location and sorting order of the data. The row key should be chosen to support efficient access patterns and avoid hotspots. For example, using a sequential or timestamp-based row key may cause all the writes to go to the same region server, creating a bottleneck. A better approach is to use a hash or a salt prefix to distribute the load evenly across the cluster .
  - Column family: A column family is a logical grouping of columns that share the same compression, encoding, and storage settings. A column family can have any number of columns, which are dynamically added at runtime. Each column family is stored as a separate file on disk, so it is recommended to limit the number of column families per table to avoid I/O overhead. A good rule of thumb is to have no more than two or three column families per table .
  - Column qualifier: A column qualifier is the name of a column within a column family. A column qualifier can be any arbitrary byte array,

and can vary in length and format across rows. A column qualifier can also be used to store additional information, such as timestamps, attributes, or indexes .

- Cell value: A cell value is the actual data stored in a column. A cell value can be any arbitrary byte array, and can vary in size and format across rows. A cell value can also be versioned, meaning that multiple values can be stored for the same row key and column qualifier, distinguished by a timestamp. HBase supports configurable retention policies for cell versions, such as keeping the latest N versions or the versions within a time range .
- HBase schema design should follow the principle of data denormalization, meaning that data should be stored in a way that minimizes the number of joins or lookups required to satisfy a query. This may involve duplicating or pre-aggregating data across tables or columns, trading off storage space for query performance. HBase also supports secondary indexes, filters, and coprocessors to optimize read performance and support complex queries .
- HBase schema design should also take into account the use cases and access patterns of the application, such as the frequency and distribution of reads and writes, the types and ranges of queries, and the expected latency and throughput requirements. HBase schema design is an iterative and experimental process, and may require testing and tuning to achieve the optimal results .

### Advanced Indexing in HBase

- HBase is a column-oriented NoSQL database that runs on top of Hadoop and is modelled after Google's BigTable.
- HBase has only one primary index that is lexicographically sorted on the row key. This means that access to records by any other attribute requires scanning over potentially all the rows in the table, which is inefficient and costly.
- Secondary indexing is a technique to create additional indexes on other attributes of the table, such as column family, column qualifier, or value. Secondary indexes can improve the performance of queries that filter or sort by those attributes.
- However, secondary indexing in HBase is not a built-in feature and requires additional design and implementation considerations. Some of the challenges are:
  - How to maintain consistency between the primary table and the secondary indexes, especially in the face of concurrent updates, failures, and compactions ?
  - How to distribute and balance the load of the secondary indexes across the cluster, avoiding hotspots and bottlenecks ?
  - How to support complex queries that involve multiple secondary indexes, such as joins, aggregations, and range scans ?
- There are different approaches to implement secondary indexing in HBase,

each with its own trade-offs and limitations . Some of the common ones are:

- Client-side indexing: The client application is responsible for creating and updating the secondary indexes as separate HBase tables, and querying them using filters or coprocessors . This approach is simple and flexible, but it requires extra logic and coordination in the client, and it may cause inconsistency or stale data if the updates are not atomic or synchronous .
- Server-side indexing: The HBase server uses coprocessors or observers to intercept the updates on the primary table and propagate them to the secondary indexes . This approach is more efficient and consistent, but it requires custom code on the server, and it may introduce performance overhead or scalability issues if the indexes are not well designed or distributed .
- External indexing: An external system or service, such as Solr, Elasticsearch, or Lily HBase Indexer, is used to create and maintain the secondary indexes, and provide a query interface to the client . This approach is more powerful and scalable, but it requires additional infrastructure and integration, and it may introduce latency or complexity in the data pipeline .
- Secondary indexing in HBase is an active area of research and development, and there are several proposals and projects to improve and standardize it . For example, HBASE-9203 is a Jira entry that outlines a design for a built-in secondary indexing framework in HBase. Apache Phoenix is a SQL layer on top of HBase that supports secondary indexing using global or local indexes.

## Zookeeper

A zookeeper is a person who works in a zoo and is responsible for the care and management of the animals. Some of the duties of a zookeeper are:

- Feeding the animals according to their dietary needs and schedules.
- Cleaning the enclosures and providing fresh water and bedding for the animals.
- Monitoring the health and behavior of the animals and reporting any signs of illness or injury to the veterinarian.
- Enriching the environment of the animals to stimulate their natural behaviors and prevent boredom and stress.
- Educating the public about the animals and their conservation status and needs.
- Assisting with the breeding and research programs of the zoo.
- Handling and restraining the animals when necessary for medical or safety reasons.
- Following the safety and ethical protocols of the zoo and the animal welfare laws and regulations.

To become a zookeeper, one usually needs to have:

- A high school diploma or equivalent.
- A bachelor's degree in zoology, biology, animal science, wildlife management, or a related field.
- Some experience working with animals, such as volunteering or interning at a zoo, wildlife sanctuary, animal shelter, or veterinary clinic.
- A passion for animals and a willingness to learn and work hard.
- Physical stamina, strength, and agility to handle the demands of the job.
- Good communication, teamwork, and problem-solving skills.
- A valid driver's license and a clean background check.

The average salary of a zookeeper in the United States is about \$30,000 per year, but it may vary depending on the location, experience, and qualifications of the zookeeper. The job outlook for zookeepers is expected to grow by 4% from 2019 to 2029, which is about as fast as the average for all occupations. However, the competition for zookeeper positions is high, as there are more applicants than available jobs. Therefore, zookeepers may need to relocate or work part-time or seasonal jobs to find employment opportunities. Zookeepers may also advance their careers by obtaining additional education, certifications, or specializations in specific areas of animal care or management.

### **Zookeeper concepts**

- Zookeeper is a distributed application that provides coordination services for other distributed systems.
- Zookeeper has a simple client-server model, where clients are nodes (machines) that use the services, and servers are nodes that provide the services.
- Zookeeper services include naming, configuration management, synchronization, and group services.
- Zookeeper can be thought of as a distributed file system, where each node can store data in a hierarchical namespace of znodes.
- Zookeeper ensures that the data stored in the znodes is consistent, ordered, and durable across the cluster.
- Zookeeper uses a leader-follower architecture, where one server acts as the leader and the rest are followers.
- Zookeeper uses a consensus protocol called Zab to elect the leader and replicate the data.
- Zookeeper clients can connect to any server in the cluster, and the server will forward the requests to the leader if necessary.
- Zookeeper clients can also watch the znodes for changes and receive notifications from the server.
- Zookeeper is designed to be fast, reliable, scalable, and simple to use.

### **How ZooKeeper helps in monitoring a cluster**

- ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services.
- ZooKeeper hosts are deployed in a cluster and, as long as a majority of hosts are up, the service will be available.
- ZooKeeper helps in monitoring a cluster by providing the following features:
  - **Status:** ZooKeeper exposes the status of each node in the cluster, such as the mode (leader or follower), the state (serving or not), the session count, the latency, and the uptime.
  - **Metrics:** ZooKeeper collects and exposes various metrics about the performance and health of the cluster, such as the number of requests, the number of connections, the data size, the watch count, the outstanding requests, and the JVM usage.
  - **Synchronization:** ZooKeeper provides a consistent view of the data across the cluster, and ensures that any changes are propagated to all the nodes in a timely manner. ZooKeeper also provides mechanisms for distributed locking, leader election, and group membership.
  - **Configuration:** ZooKeeper allows the cluster to store and update the configuration information in a centralized location, and notifies the nodes of any changes. ZooKeeper also provides a hierarchical namespace for organizing the configuration data.
  - **Naming:** ZooKeeper provides a unique and persistent identifier for each node in the cluster, and allows the nodes to register and discover each other. ZooKeeper also supports dynamic and ephemeral nodes, which can join and leave the cluster without affecting the availability of the service.

## How to build applications with Zookeeper

Zookeeper is a distributed system coordinator that provides services such as configuration management, synchronization, naming, and leader election. Zookeeper can help developers to build reliable and scalable distributed applications by simplifying the coordination logic and reducing the complexity of the system.

To build applications with Zookeeper, the following steps are recommended:

- Download and install Zookeeper from the official website or use a package manager such as apt or yum. Alternatively, you can use a containerized or cloud-based deployment of Zookeeper, such as Kubernetes or Amazon Web Services.
- Create a configuration file for Zookeeper that specifies the server ID, data directory, client port, and other parameters. If you are running Zookeeper in a cluster mode, you also need to specify the list of servers in the ensemble and their addresses.
- Start Zookeeper server(s) using the zkServer.sh script or the corresponding



command for your platform. You can verify the status of the server(s) using the `zkServer.sh status` command or the four-letter commands such as `ruok` or `stat`.

- Connect to Zookeeper using a client library or a command-line tool such as `zkCli.sh`. You can use the client to create, read, update, and delete znodes, which are the basic units of data in Zookeeper. Znodes can store data, have children, and have watches and access control lists attached to them.
- Use the Zookeeper API to implement the coordination logic for your application. For example, you can use Zookeeper to store configuration data, create locks and barriers, elect leaders, and register services. Zookeeper provides several recipes and examples for common use cases on its website and in its documentation.
- Test and debug your application using the Zookeeper logs, metrics, and tools. You can also use the Zookeeper JMX interface to monitor the performance and health of the server(s) and the client(s). You can use the `zkCli.sh` or the `zkTxnLogToolkit.sh` tools to inspect the data and transactions in Zookeeper.

## IBM Big Data strategy

- IBM Big Data strategy is a corporate initiative that aims to provide solutions for storing, managing, and analyzing the large volumes of data generated daily by various sources, such as social media, sensors, mobile devices, etc.
- IBM Big Data strategy is part of its Smarter Planet vision, which seeks to leverage the potential of smarter systems to achieve economic and sustainable growth and societal progress .
- IBM Big Data strategy consists of four key components: Big Data platform, Big Data services, Big Data solutions, and Big Data ecosystem .
  - Big Data platform: IBM offers a comprehensive and integrated platform that supports various types of data, such as structured, unstructured, and semi-structured, and various sources, such as cloud, on-premise, or hybrid. The platform includes products such as IBM InfoSphere BigInsights, IBM InfoSphere Streams, IBM PureData System for Analytics, IBM PureData System for Hadoop, IBM PureData System for Operational Analytics, IBM DB2 with BLU Acceleration, IBM Cognos, IBM SPSS, and IBM Watson .
  - Big Data services: IBM provides a range of services to help clients design, implement, and optimize their Big Data strategy, such as consulting, implementation, integration, migration, support, education, and training. The services cover various domains, such as analytics, data management, data governance, data quality, data security, data privacy, and data monetization .
  - Big Data solutions: IBM delivers industry-specific and cross-industry solutions that address the business challenges and opportunities of

Big Data, such as customer analytics, fraud detection, risk management, operational efficiency, innovation, and social media analytics. The solutions are based on the IBM Big Data platform and leverage the IBM Big Data services and ecosystem .

- Big Data ecosystem: IBM collaborates with a network of partners, such as technology providers, solution providers, academic institutions, research organizations, and industry associations, to enhance the value and adoption of Big Data solutions. The ecosystem includes partners such as Cloudera, Hortonworks, MapR, SAS, SAP, Oracle, Microsoft, Teradata, Accenture, Deloitte, PwC, and MIT .
- IBM Big Data strategy aims to help clients gain insights from their data, improve their decision making, enhance their performance, and create competitive advantage. IBM Big Data strategy also supports the development of cognitive computing and artificial intelligence, which are the next frontiers of Big Data analytics .

### **IBM Big Data strategy**

- IBM Big Data strategy is a corporate initiative that aims to provide solutions for storing, managing, and analyzing the large volumes of data generated daily by various sources, such as social media, sensors, mobile devices, etc.
- IBM Big Data strategy is part of its Smarter Planet vision, which seeks to leverage smarter systems and technologies to achieve economic and social progress and sustainability.
- IBM Big Data strategy consists of four key components: Big Data platform, Big Data services, Big Data solutions, and Big Data ecosystem.
  - Big Data platform: IBM offers a comprehensive and integrated platform that supports various types of data, such as structured, unstructured, semi-structured, streaming, etc. The platform includes products such as IBM InfoSphere BigInsights, IBM InfoSphere Streams, IBM DB2, IBM Netezza, IBM PureData System, etc.
  - Big Data services: IBM provides consulting, implementation, and support services to help clients design, deploy, and optimize their Big Data initiatives. The services include data strategy, data governance, data integration, data quality, data analytics, data visualization, etc.
  - Big Data solutions: IBM delivers industry-specific and cross-industry solutions that address various business challenges and opportunities using Big Data analytics. The solutions include customer analytics, fraud detection, risk management, social media analytics, etc.
  - Big Data ecosystem: IBM collaborates with various partners, such as technology providers, academic institutions, research organizations, etc., to foster innovation and adoption of Big Data technologies and solutions. The ecosystem includes initiatives such as IBM Academic Initiative, IBM Innovation Centers, IBM Research, IBM Watson, etc.
- IBM Big Data strategy aims to help clients achieve various benefits, such

as improved decision making, enhanced customer experience, increased operational efficiency, reduced costs, and new revenue streams.

- IBM Big Data strategy follows six steps to design and execute a data strategy that aligns with the business objectives and the data lifecycle. The steps are: understand the business objectives, assess the current state, map out the data strategy framework, define the data governance model, implement the data strategy, and measure and refine the data strategy.

### **Introduction to Infosphere**

- The term infosphere (information + - sphere) is used to describe a meta-physical realm of information, data, knowledge, and communication, populated by informational entities called inforgs (or, informational organisms).
- The infosphere is the whole system of services and documents, encoded in any semiotic and physical media, whose contents include any sort of data, information and knowledge, with no limitations either in size, typology, or logical structure.
- The infosphere can be seen as a global network of interconnected information systems, such as the internet, the web, social media, databases, sensors, etc., that enable the creation, storage, processing, and exchange of information.
- The infosphere can also refer to a specific data integration platform, such as IBM InfoSphere Information Server, that helps users to more easily understand, cleanse, monitor and transform data, and to deliver trusted information to critical business initiatives.
- The infosphere can also be a fictional setting, such as the one depicted in the animated series Futurama, where the infosphere is a giant sphere of pure information that contains the sum of all knowledge in the universe.

### **Introduction to BigInsights**

- BigInsights is a software platform that provides a comprehensive solution for managing and analyzing big data.
- BigInsights is based on Apache Hadoop, an open source framework that enables distributed processing of large and diverse data sets across clusters of commodity hardware.
- BigInsights enhances Hadoop with enterprise-grade features such as security, administration, governance, performance, and integration.
- BigInsights also includes value-added components such as IBM Big SQL, IBM BigSheets, IBM Big R, IBM Text Analytics, and IBM BigInsights Analyst and Data Scientist modules.
- BigInsights can help organizations gain insights from various types of data, such as structured, unstructured, semi-structured, streaming, and geospatial data.

- BigInsights can be deployed on-premises, on cloud, or in a hybrid environment.

### **Introduction to Big Sheets**

- Big Sheets are any sheets that come in bigger dimensions than standard ones.
- They can be made for an extra large king, queen, full, or twin bed.
- They are suitable for huge mattresses that may not fit regular sheets.
- They can be made of different materials, such as cotton, linen, sateen, or cashmere.
- They can have different patterns, colors, or designs to match the bedroom decor.
- They can be purchased online or in stores from various providers, such as Dunelm, Brooklinen, Garnet Hill, or Walmart.
- They may vary in price, quality, and durability depending on the brand, fabric, and size.
- They may require special care, such as washing, drying, or ironing, to maintain their appearance and comfort.

### **Introduction to Big SQL**

- Big SQL is a high performance massively parallel processing (MPP) SQL engine for Hadoop that makes querying enterprise data from across the organization an easy and secure experience .
- Big SQL can quickly access a variety of data sources including HDFS, RDBMS, NoSQL databases, object stores, and WebHDFS by using a single database connection or query.
- Big SQL is built on the world class IBM common SQL database technology and has all the standard RDBMS features such as transactions, concurrency, security, and optimization.
- Big SQL supports the ANSI SQL standard and provides compatibility with many SQL dialects such as Oracle, DB2, and Netezza.
- Big SQL provides connectivity for some applications through either a 32-bit or a 64-bit ODBC driver, on either Linux or Windows, that conforms to the Microsoft Open Database Connectivity 3.0.0 specification.
- Big SQL also provides a command line interface (CLI) and a web-based user interface (UI) for executing and managing SQL queries.
- Big SQL leverages the distributed processing power of the Hadoop cluster and the scalability of the Hadoop Distributed File System (HDFS) to handle large volumes of structured and semi-structured data.
- Big SQL can also integrate with other components of the Hadoop ecosystem such as Hive, Spark, HBase, and Kafka to enable advanced analytics and data processing .